IN-NETWORK ASSISTANCE FOR SECURE TRANSPORT PROTOCOLS

A DISSERTATION
SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE
AND THE COMMITTEE ON GRADUATE STUDIES
OF STANFORD UNIVERSITY
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY

Gina Yuan
July 2026

Abstract

Post-TCP transport protocols such as QUIC now include end-to-end encryption at the transport layer. This enhances security by making their packets opaque to connection-splitting proxies and immune to ossification, but can harm performance. In this dissertation, I will present the Sidekick protocol approach to in-network assistance for secure transport protocols, where proxies and endpoints send information on an adjacent connection about which encrypted packets they have received. Sidekick protocols apply set reconciliation techniques in a novel setting to efficiently refer to encrypted packets in a **quACK**, without using plaintext sequence numbers. In some use cases of the Sidekick protocol, Packrat proxies keep a small cache of packets for possible in-network retransmissions of encrypted packets. This approach allows secure transport protocols to achieve performance benefits similar to those of traditional PEPs, but leaves the protocol unchanged on the wire and free to evolve. Finally, I will present the split throughput heuristic for reasoning about connection-splitting in the context of two recent developments: the BBR congestion control algorithm and the QUIC transport protocol. I use this heuristic in an emulation measurement study and discuss how connection-splitting, despite the ossification it can induce, still offers valuable performance benefits today.

Quacknowledgments

I would like to express my deepest gratitude to my colleagues, friends, and family who have influenced and supported my growth throughout graduate school.

First, I would like to thank my co-advisors—Keith Winstein, David Mazières, and Matei Zaharia. Keith has this gift for storytelling that has become a model for how I want to communicate to the world. I have learned so much from him, and feel so grateful for the series of events that brought us together. David and Matei were especially influential in the early years of my PhD. David exudes this idealism that motivates me to pursue ambitious goals and stay true to my values. Matei has a unique penchant for anticipating how an idea will be received, evolve, and ultimately make an impact, inspiring me to select problems with the same intentionality. My co-advisors have been a constant source of confidence and support, and I am deeply grateful for their mentorship.

I cannot express enough gratitude for the guidance I have received—I am honored to have worked with Michael Welzl, who has sparked so much joy over the years with his passion, knowledge, and excitement about the Internet. The theoretical foundations of this work owe much to the mentorship of Dan Boneh and Mary Wootters. I am thankful for Kate Maher, my orals committee chair. I'd also like to extend my appreciation to Te-Yuan Huang, Wei Wei, Yiannis Yiakoumis, Bruce Spang, Deian Stefan, Robert Morris, Malte Schwarzkopf, Jon Gjengset, Raul Castro Fernandez, and Albert Carter.

This work would not have been possible without Matthew Sotoudeh and David Zhang, whose unique perspectives on research provided just the inspiration needed to connect the dots during a more stagnant period of my PhD. Thea Rossman became my networking compatriot in my final year, and I am thankful for our many brainstorming sessions throughout our lively collaboration. It was also my pleasure to work with these bright undergraduates—Devrath Iyer, Jaylene Martinez, Angel Rivera, Henry Weng—who shaped the way that I approach mentorship.

I am thankful for my labmates in Stanford SNR, Stanford SCS, and the FutureData group for the welcoming and vibrant sense of community—Yuhan Deng, Akshay Srivatsan, Colin Drewes, Yasmine Mitchell, Francis Chua, Sebastian Ingino, Angie Montemayor, Geoffrey Ramseyer, Zachary Yedidia, Firas Abuzaid, Lingjiao Chen, Cody Coleman, Jared Quincy Davis, Trevor Gale, Daniel Kang, Fiodar Kazhamiaka, Omar Khattab, Peter Kraft, Qian Li, Deepak Narayanan, Shoumik Palkar, Liana Patel, Deepti Raghavan, Kexin Rong, Sahaana Suri, Keshav Santhanam, Kai Sheng Tai, Pratiksha Thaker,

James Thomas. I want to extend my special thanks to Deepti, Deepak, and Shoumik, senior student mentors and co-authors whose wisdom and humility I hope to carry on.

Thank you to many others in the Stanford community who have impacted me in some way—the systems and security communities, Aishwarya Mandyam and the Race Condition Running club, the friends I have made through the Stanford Cycling community including Miles Chan, Jack Liu, Claire MacDougall, Basil Saeed, Paddy Simha, Albert Wu, and coaches Adrian Bennett and Isaiah Newkirk.

During the pandemic when I had nowhere to go but outdoors, I picked up road cycling, raced bikes, and met some awesome people—Niky Taylor, Kelly Brennan, and Chloé Mauvais. Our relationships with bikes have evolved over time but our friendships have not. I poured my heart into Alto Velo, and it meant a lot to me to have grown this community with some amazing women—Sue Lin Holt, Robin Betz, Ari Fischer, Emily Schell, Louise Thomas, Steph Hart, Rachel Hwang, Katie Monaghan.

Fully grasping that the Internet didn't exist when my parents were born has made me all the more impressed with their story—immigrating to the U.S., building careers as software engineers, and quietly inspiring Paula and me to pursue computer science. I am secretly thrilled that the four of us are reunited in the Bay Area. I also want to express deep respect for my grandma, Weijiu Zhang, whose memory deepens my appreciation for life and the sacrifices that came before me. Thank you to my parents, Christine Lei and Wei Yuan, for creating the opportunities for me to chase my dreams.

Finally, I feel incredibly lucky to have been able to grow alongside my partner, Jack Liu. His curiosity, spontaneity, and dedication to his pursuits encourage me to approach my own work and life with the same vigor. Thank you for all the happiness and random treats you bring to my life.

Previously Published Material

This dissertation adapts work from the following peer-reviewed conference and workshop papers, and would not have been possible without the contributions of the co-authors listed below:

- Gina Yuan, Thea Rossman, and Keith Winstein. Internet Connection Splitting: What's Old is New Again. In *USENIX ATC*, Boston, MA, July 2025 (to appear).
- Gina Yuan, Matthew Sotoudeh, David K. Zhang, Michael Welzl, David Mazières, and Keith Winstein. Sidekick: In-Network Assistance for Secure End-to-End Transport Protocols. In *NSDI*, Santa Clara, CA, April 2024.
- Gina Yuan, David K. Zhang, Matthew Sotoudeh, Michael Welzl, and Keith Winstein. Sidecar: In-Network Performance Enhancements in the Age of Paranoid Transport Protocols. In *HotNets*, Austin, TX, November 2022.

Descriptions of the IBLT quACK in Chapter 2 and Packrat proxy in Chapter 4 are part of ongoing work in collaboration with Thea Rossman, Michael Welzl, and Keith Winstein.

Contents

Chapter 1

Introduction

The topic of this dissertation is an old idea in the Internet—performance-enhancing proxies, or PEPs. But before I say more about PEPs and all their triumphs and heartbreaks over the last few decades, I think it's important to go through a brief history of the Internet. It will be valuable to understand why certain aspects of the Internet were designed the way they were so that we have the proper context for some of the challenges they bring us today. Let's go back to the very beginning.

1.1 Brief history of the Internet

In the 1960s, the ARPANET was just a burgeoning research project sponsored by the U.S. Department of Defense [54]. The network consisted of four stationary hosts, including the Stanford Research Institute. ARPANET applied the new concept of packet switching where data is broken down into smaller packets and transmitted indepedently over wired links. Once the packets reach their destination, they are reassembled into a message understandable by the endpoint. Two computers communicated with each other for the first time in 1969 [95]. The data transmitted over the network were the letters "lo" for "login", though the system crashed before the researchers could finish.

TCP/IP was introduced in the 1970s to provide a cleaner separation of responsibilities between endpoints and in-network nodes as more and more research institutions joined ARPANET [24]. In this canonical model of the Internet, routers and other network components forwarded IP datagrams without regard to their payloads, while TCP transport was end-to-end and implemented only in hosts [27, 115]. The principles of packet switching and TCP/IP are still present in how we use the network today.

In the 1980s, the early Internet was growing rapidly and the sheer amount of traffic started to overwhelm the links. Network congestion caused packet loss, which subsequently caused retransmissions and even more congestion, which led to congestion collapse and a severe degradation in performance [100]. In response, the first feedback control mechanisms for "congestion control" in

TCP/IP such as additive-increase/multiplicative-decrease (AIMD), slow start, and congestion windows were invented [26, 67]. Implemented at hosts, these "loss-based" congestion control algorithms interpreted packet loss as congestion to significantly decrease the congestion window.

The 1990s marked the beginning of the mainstream Internet with the invention of the World Wide Web [42]. The Internet was no longer just a research proposal, but a global commercial project. Wi-Fi, standardized in 1999, enabled people to move freely inside their homes and workspaces [114], while the iPhone and Android phones of the late 2000s along with the growth of cellular networks increased our mobility elsewhere [82]. Satellite networks have long provided Internet access to remote and rural regions, and are even more accessible today with low-Earth orbit satellites such as Starlink in 2019 [123]. Wireless networks and global connectivity really expanded the reach of the Internet, but it also meant the network became more lossy and high-delay than ever before.

Applications have also demanded more from the Internet over time, evolving from basic services such as e-mail and web browsing to a continuous pursuit for higher throughput and lower latency. There are high-bandwidth applications such as streaming and content sharing (e.g., Netflix, YouTube, Instagram) and interactive, low-latency applications such as video conferencing and live streaming (e.g., Zoom, Twitch). Application goals often conflict—a service like Netflix might prefer large buffers in the network to optimize for throughput, while these same buffers introduce significant packet delays for users of Zoom. When transport is end-to-end, a lossy, high-delay network complicates how effectively hosts can coordinate and optimize performance for the specific needs of their applications.

Triumph: TCP performance-enhancing proxies.

How do our resource-intensive applications deal with lossy, high-delay networks? Let's examine the following scenario: Imagine you are riding the local commuter rail. Your laptop is connected to the Wi-Fi on the train via a lossy, low-latency wireless link to the router. The train's router then connects you to the rest of the Internet via a more reliable, high-latency cellular path, pinging nearby base stations that the train rides past. You upload a large file, perhaps a paper submission to a conference. It's right before the deadline and you find the network to be incredibly slow.

The router divides the network path into two very distinct path segments, but from the perspective of the hosts' transport protocols on either end, there is just *one* lossy, high-delay network path. Because you need to upload the entire file, when the wireless link drops a packet, you want to retransmit the packet as soon as you find out about the loss. However, your laptop can only detect the loss once it has received a signal from the web server on the other end of the high-delay path, even though the loss occurred nearby. Additionally, the "loss-based" congestion control on your laptop interprets the signal as congestion, and decides to conservatively reduce the large congestion window on the high-delay path by a significant amount. This slows your connection down even more.

The solution to this problem in the 1990s when wireless networks first started gaining popularity was performance-enhancing proxies, or PEPs [50]. Now we can formally define PEPs: PEPs are

network middleboxes deployed to improve the performance—most simply, throughput and latency—of the connections for which they are on the path. These middleboxes were deployed widely at base stations, satellite gateways, and even Wi-Fi routers. As of 2019, it was estimated that 20–40% of Internet paths still contained a PEP [41,57]. Most middleboxes provide in-network assistance to TCP, the dominant transport protocol, and commonly as connection-splitting TCP PEPs [17,33,45,55,68]. Other middleboxes might perform header compression, ACK filtering and decimation, or other caching and in-network retransmission functions to save bandwidth and improve performance [8,29,56,96,111].

Your file upload could be much faster with a connection-splitting TCP PEP on the train's router. These PEPs silently interpose themselves in the network path without the knowledge of the endpoints, splitting an end-to-end connection into multiple, independent TCP connections on either side of the proxy. Once the connection is split, the proxy tunes congestion control and other parameters for each, potentially very different, path segment. Returning to the scenario on the train, the TCP transport protocol on your laptop needs only to wait for a signal from the nearby router to detect loss, as opposed to an end-to-end signal. The resulting congestion response is also mild since the nearby congestion window is already small. The result is earlier retransmissions and a significantly faster file upload.

Heartbreak: Protocol ossification and secure transport protocols.

However, even though there was a connection-splitting TCP PEP on the train, your laptop was not using TCP but a different reliable transport protocol called QUIC [65]. QUIC cannot benefit from PEPs made for TCP, and its performance is the same as an entirely end-to-end connection—slow. The irony in this situation is that QUIC is a pretty new transport protocol announced in 2012, 38 years newer than TCP and long after lossy, high-delay networks became the norm. It was invented at Google, one of the world's largest software companies, and is even in wide deployment with 8.4% of websites supporting QUIC including major traffic sources such as Cloudflare [28,128,143]. Yet its performance is so much worse, and for reasons that were very intentional. How can this be explained?

Middleboxes cannot help QUIC because QUIC is what we call a "secure" transport protocol. Even though transport was originally designed to be end-to-end, the TCP transport headers in the payload of an IP datagram are unencrypted. This reveals information such as the packet sequence number and acknowledgment number to curious middleboxes. PEPs intercept the packets of a TCP handshake and use this information to transparently split the connection on initialization. In comparison, QUIC is completely encrypted on the wire. PEPs are only able to forward these QUIC packets as opaque IP datagrams. Many recent transport protocols in addition to QUIC now intentionally encrypt their headers, rendering them unable to receive help from TCP PEPs and harming performance [10,107,133,144].

If exposing transport headers to proxies can help performance, why are protocol designers so deliberate about encrypting them? The answer lies in the sacrifice that TCP made in hindsight

to benefit from in-network assistance. To provide the kind of help that they do, proxies make assumptions about the wire format of the protocol and what each field is used for. They modify and block traffic based on these fields. When protocol designers later repurpose these fields to evolve the protocol or introduce new features, these assumptions break and the "help" can actually harm [12, 13, 74, 80, 89]. With PEPs being so widely deployed, backwards compatibility is challenging to maintain and protocols have no choice but to fix their functionality in place. This phenomenom is called protocol ossification, where the wire format of the transport protocol has become rigid and resistant to change over time [41, 103].

Protocol ossification has derailed the deployment of many otherwise promising extensions to TCP. Despite making it as far as the IETF, ideas like extended TCP options, Multipath TCP, ECN++, and tcpcrypt now lie in a graveyard of unrealized and under-realized proposals [9,57,88,112]. Even innocuously simple fields like the TCP sequence number are not immune to ossification. For example, QUIC uses two sequence number spaces, referring separately to packets and the bytes in the underlying stream [65]. This is not possible with TCP. In one infamous debate, exposing just a single bit in QUIC for passive RTT measurement triggered widespread concern in the IETF about middlebox interference [59]. Due to stories like this, I would expect transport protocols to be secure by default in the future.

An additional reason for encrypting the transport layer is privacy and security. In the early days of the Internet, ARPANET was just a small network of trusted research institutions. Privacy and security were not core design principles in the invention of TCP/IP. Transport protocols these days prefer to limit the exposure of non-essential information to network middleboxes using encryption.

1.2 Overview

This dissertation is about resolving the tension between performance and ossification in PEPs for secure transport protocols. While existing work has previously proposed co-designing PEPs with transport protocols [37, 46, 66, 121], I propose a new approach to in-network assistance that leaves the original transport protocol unmodified on the wire and free to evolve. In this approach, called **Sidekick protocols**, proxies and endpoints send information *about* the packets of the underlying connection on an *adjacent* connection. Using a **Packrat proxy**, the adjacent connection can also generate in-network retransmissions. The information about the underlying connection is called a **quACK**, which applies set reconciliation techniques to a novel setting to efficiently refer to encrypted packets without plaintext sequence numbers. I will show that **the Sidekick protocol approach to in-network assistance for secure transport protocols achieves performance benefits similar to those of traditional TCP PEPs without the possibility of protocol ossification**.

The rest of this work will go as follows: I will start Chapter 2 with quACKs, a tool for efficiently referring to a set of randomly encrypted packets. Chapter 3 will describe how proxies use the Sidekick

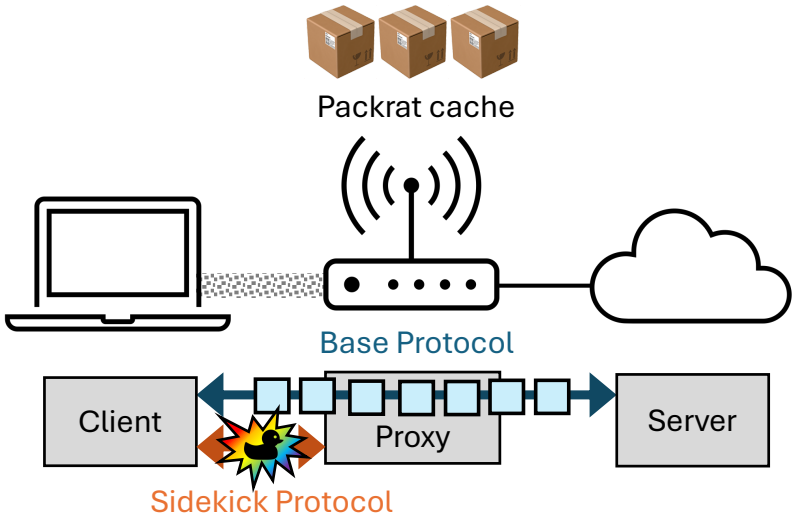


Figure 1.1: An overview of the Sidekick protocol approach to in-network assistance. In this diagram, the client is connected to the server via a lossy link to the proxy. The client and proxy speak the Sidekick protocol on an adjacent connection to the end-to-end connection between the client and server speaking the base protocol. QuACKs are exchanged over the Sidekick protocol, and the proxy also maintains a Packrat cache of packets for potential retransmission.

protocol to send quACKs over an adjacent connection, and how endpoints modify their behavior in response to quACKs to emulate the performance benefits of traditional PEPs. Chapter 4 will describe how endpoints send quACKs to Packrat proxies to receive in-network retransmissions, also achieving improved performance for a variety of secure transport protocols.

Finally, Chapter 5 will take a step back and analyze why the performance benefits of connection-splitting are still relevant today despite recent developments such as the "model-based" BBR congestion control algorithm and the QUIC transport protocol, motivating the need for protocol-agnostic solutions such as the Sidekick approach for helping secure transport protocols. Chapter 6 will discuss considerations for the practical deployment of the Sidekick protocol and future research directions, including the immediate impact on congestion control research, then conclude.

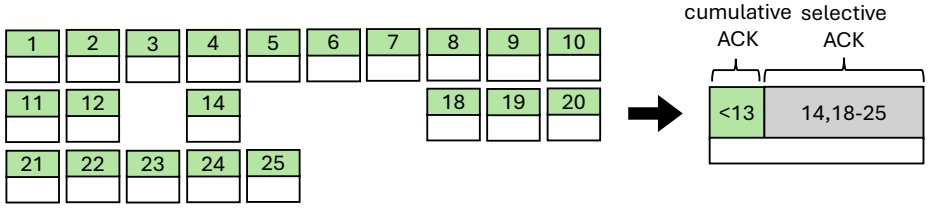
Chapter 2

QuACKs: Referring to encrypted packets without sequence numbers

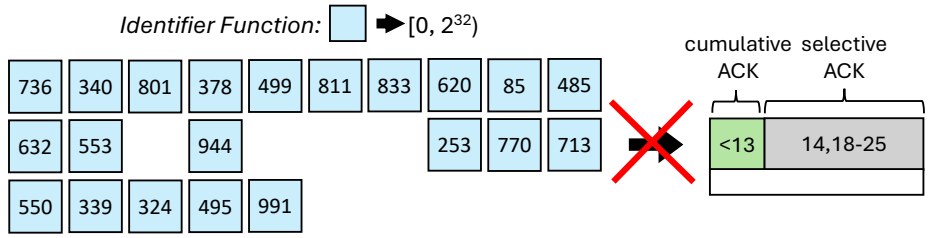
When acting as a PEP for a secure transport protocol, a proxy needs to be able to refer to and efficiently acknowledge a set of packets *even when they are randomly encrypted*. Recall that it is straightforward for a proxy to refer to TCP packets—Since packets in a TCP connection are labeled with cleartext sequence numbers, a proxy can easily refer to ranges of packets using cumulative and selective ACKs. But this problem is technically challenging for packets of secure transport protocols where the headers are randomly encrypted (Figure 2.1). The proxy can't exactly say a certain range of packets has been received, or even if any packets are missing. The protocol could perhaps reveal a sequence number to the proxy on the wire, but then that field risks ossification. This technical problem is the crux to enabling the transport behavior of the Sidekick protocol (Chapters 3 and 4).

In this chapter, we mathematically define the **quACK problem** in the context of network packets sent over a lossy path segment, and discuss how to select *identifiers* to refer to encrypted packets. We analyze strawman solutions to the quACK problem that use either too much space or computation. Then, we present two efficient constructions of the quACK inspired by related theoretical work in set reconciliation [43, 97], as well as coding theory and graph theory [69]. Both constructions require a bound on the maximum number of missing elements, a threshold t, and have different tradeoffs. The *power sum* construction is space-optimal and deterministic, but requires an exact bound. The *invertible Bloom lookup table (IBLT)* construction is more computationally scalable with an approximate bound, but also probabilistic with constant factor overheads.

Concretely realized, the power sum quACK uses 48 bytes to express the equivalent of TCP's cumulative + selective ACK over the 32-bit identifiers of randomly encrypted packets, tolerating up to 10 missing packets before the last "selective ACK." On a recent x86-64 CPU, it takes 26 ns to encode a single packet in the power sum quACK, and 15.5 μs to decode which received packets the



(a) TCP packets with cleartext sequence numbers.



(b) Randomly encrypted QUIC packets.

Figure 2.1: Referring to a set of packets is easy when labeled with cleartext sequence numbers by using a cumulative and selective ACK. It is hard when the packets are randomly encrypted because there is no such concept of a range of packets or even a missing packet. The identifier function (Section 2.1.1) deterministically maps a packet payload to a random 32-bit identifier.

quACK represents. The IBLT quACK takes 42 ns to encode a single packet and 1.3 μs to decode a quACK, using at least 58 bytes. These overheads compare well with several strawman alternatives.

2.1 The QuACK Problem

In the quACK problem, a sender (such as an endpoint) transmits a multiset[1] of elements S. At any given time, a receiver (such as a proxy) has received a subset $R \subseteq S$ of the sent elements. We would like the receiver to communicate a small amount of information to the sender, who then efficiently decodes the missing elements—the set difference $S \setminus R$—knowing S. This small amount of information is called the "quACK". The problem is: **what is in a quACK and how do we decode it?**

2.1.1 Identifiers

How exactly do we refer to the elements in the quACK problem that have been sent or received? In a networking context, these elements are packets. Traditional TCP proxies have been able to interpose their own concise, cumulative and selective acknowledgments using cleartext sequence numbers, but this is not possible with modern, secure transport protocols (Figure 2.1). Even if a connection did expose an unencrypted numerical field, we would not want to refer to that field at risk of ossifying

[1] A "multiset" means the same element can be transmitted more than once.

that protocol. As mentioned before, even a seemingly innocuous field such as the TCP sequence number can be ossified with redesigns of the sequence number space [65].

Instead, we need a function that deterministically maps a packet to a random b-byte *identifier*. The most trivial solution that applies to all base protocols is to hash the entire payload. Another option if the payload is already pseudorandom (e.g., QUIC) is to take the first b bytes from a fixed offset of that payload. Although the latter option would rely on those bytes to remain pseudorandom, it is computationally more efficient because it does not require reading the entire payload.

Collisions.

The main consideration when selecting the number of bytes, b, in an identifier is the tolerance for collisions compared with the extra data needed to refer to these packets on the link. Higher bit widths also correspond to higher computation costs, which is explored in Section 2.4.1. The larger b is, the lower the collision probability but the greater the link and computation overheads.

Define the collision probability to be the probability that a randomly-chosen b-byte identifier in a list of n packets maps to more than one packet in that list. There are $2^8 = 256$ possible values of a byte, thus 256^b possible values of a b-byte identifier. If we assume that the identifiers are uniformly distributed, the collision probability is equal to the complement of the probability that all other $n-1$ packets have a different identifier: $1-(1-1/256^b)^{n-1}$. When $n = 500$, using 4 bytes results in an almost negligible chance of collision while using 2 bytes results in a 0.8% chance (Table 2.1).

Identifier Bytes	1	2	4	8
Collision Probability	0.86	0.0076	1.2e-07	≈0

Table 2.1: Collision probabilities for $n = 500$.

When handling collisions, a sender who is decoding a quACK has a list of n packets it is trying to classify as received or missing (Section 2.2). This is the number of packets that were either not classified or newly transmitted since decoding a previous quACK. Note that collisions are also known to the sender beforehand since it has the list of n packets. If there is a collision between a packet that is received and a packet that is missing, the fate of that identifier is considered indeterminate. Either the protocol can still function with approximate statistics (e.g., congestion control) or it can fall back to an end-to-end mechanism (e.g., retransmission).

2.1.2 Strawman solutions

The quACK problem is simple with cumulative and selective acknowledgments of plaintext sequence numbers, but deceivingly challenging for pseudorandom packet identifiers. Consider the following strawman solutions and their disadvantages:

Strawman 1: Echo every identifier.

Strawman 1a, similar to [78, 86], echoes the identifier of every received packet in a new UDP packet to the sender. Decoding is trivial given that the identifiers are unmodified. This strawman adds significant link overhead in terms of additional packets. Additionally, since the strawman is not a cumulative representation of all the packets received over the connection, losing a quACK means the sender can falsely consider a packet to be lost, creating a congestion event or spurious retransmission.

Strawman 1b echoes a sliding window of identifiers over UDP such that there is overlap in the identifiers referred to by consecutive quACKs. This solution is slightly more resilient to loss, but uses more bytes and is still not guaranteed to be reliable without a cumulative representation. Another variant batches identifiers to reduce the number of packets, but this solution is less resilient to loss.

We also consider a Strawman 1c that echoes every identifier over TCP with `TCP_NODELAY` to send every identifier in its own TCP packet. The `TCP_NODELAY` option ensures each quACK is sent immediately rather than batching the identifiers in the bytestream. The reliability of TCP ensures there are no false positives when detecting lost packets, but adds even more link overhead in terms of TCP headers and additional ACKs from the sender (default every other packet in the Linux kernel).

Strawman 2: Cumulative hash of all identifiers.

Strawman 2 sends a SHA-256 hash of a sorted concatenation of all the received packets in a UDP packet, and the sender hashes every subset of the same size of sent packets until it finds the subset with the same hash (assuming collision resistance). The strawman includes a count of the packets received to determine the size of the subset to hash. As the number of missing packets exceeds even a moderate amount, the number of subsets to calculate a hash for explodes. Consider that a quACK composed of 490 sent packets with 10 missing packets has roughly $\binom{500}{10} = 200$ quintillion possible subsets to try. Thus this strawman is computationally impractical to decode.

One might also suggest the receiver send negative acknowledgments of the packets it has not received. However, unlike sequence numbers where one can determine a gap in received packets, there is no way to tell with random identifiers what packet is missing or should be expected next.

2.2 Efficient constructions of the quACK in packet-scale settings

The problem of providing a concise acknowledgment of encrypted packet identifiers closely resembles a more general problem called set reconciliation. In this problem, two parties, each with a set, communicate a small amount of information to each other to learn the set difference—the elements in their set that is missing in the other's set [43, 97]. There are two well-known classes of solutions to

this set reconciliation problem with different tradeoffs in communication and computation cost. We provide background in the literature on the set reconciliation problem, then describe our efficient power sum and invertible Bloom lookup table (IBLT) constructions of the quACK based on these two classes of solutions.

2.2.1 Background: Set reconciliation

The first class of solutions models the problem as a system of symmetric power sum polynomial equations. This construction is deterministic as the set difference is just the solutions to this system of equations. It is also space-optimal; [97] introduced symmetric polynomials as a solution to the set reconciliation problem with nearly optimal communication and tractable computation cost. [36] improved the computation cost in decoding, but found the problem to be intractable for larger parameters. The power sum solution has mainly been restricted to more theoretical literature.

The second class of solutions encodes the elements in an invertible Bloom lookup table (IBLT), and is more computationally scalable than the first class. The IBLT is a data structure based on the Bloom filter [47], and probabilistically decodes the set difference with a multiplicative factor on the comunication cost. This multiplicative factor has been shown to be small on average: 1.23–1.35× [5, 136][2]. In practice, set reconciliation with a Bloom-filter-like data structure has mainly been applied to larger distributed systems such as blockchain and social media synchronization [125, 136].

The quACK setting is novel compared to previous applications of set reconciliation in several ways. In this setting, the elements in the problem are network packets, or more specifically packet *identifiers*. This means that in the algebraic solution, the elements in the set are not just arbitrary numerical values but have bounded bit widths. Only the receiver communicates information, and the sender performs subset and not set reconciliation on the missing packets. Unlike the blockchain setting, the two parties synchronize on millisecond-RTT timescales compared to seconds. The communication must be encoded in a single network packet, not a stream of packets. In addition, at these smaller timescales, fewer elements (typically tens at most) are reconciled at once, and optimizations for constant factor overheads matter more.

2.2.2 Power sums: A space-optimal and deterministic solution

We describe a construction of the quACK that models the problem as a system of power sum polynomial equations when we have an exact bound on the maximum number of missing elements, a threshold t. Unlike the previous strawmen, this construction is efficient to decode, and its size is proportional only to t. It is also a cumulative representation of all packets ever received.

Consider the simplest case, when the receiver is only missing a single element. Note that since we have bounded bit widths in practice, we must map packet identifiers to a finite field, i.e., modulo

[2]The 1.23× multiplicative factor on the communication cost is derived from $1/\alpha$, where $\alpha \approx 0.81$ in [5].

the largest prime that fits in b bytes, and do all arithmetic operations in that field. The receiver communicates the sum $\sum_{x \in R} x$ of the received elements to the sender. The sender computes the sum $\sum_{x \in S} x$ of the sent elements and subtracts the sum from the receiver, calculating:

$$\sum_{x \in S} x - \sum_{x \in R} x = \sum_{x \in S \setminus R} x,$$

which is the sum of elements in the set difference. In the case of a single missing element where $|S \setminus R| = 1$, the sum is exactly the value of the missing element.

In fact, we can generalize this scheme to any number of missing elements m. Instead of transmitting only a single sum, the receiver communicates the first m *power sums* to the sender, where the i-th power sum of a multiset R is defined as $\sum_{x \in R} x^i = x_1^i + x_2^i + x_3^i + \cdots$. The sender then computes the first m power sums of S and calculates the respective differences d_i for $i \in [1, m]$, producing the following system of m equations:

$$\left\{ \sum_{x \in S \setminus R} x^i = d_i \mid i \in [1, m] \right\}.$$

Instead of transmitting an unbounded number of power sums, the receiver only maintains and sends the first t power sums. The sender also incrementally maintains the same number of power sums, required for decoding. Efficiently solving these t power sum polynomial equations in t variables in a finite field is a well-understood algebra problem [43]. The solutions are exactly $x \in S \setminus R$.

2.2.3 Invertible Bloom lookup table: A computationally scalable approach

Now we apply the invertible Bloom lookup table to describe a probabilistic construction of the quACK in a Bloom-filter-like data structure. The IBLT construction has better computation complexity than the power sum construction, but incurs higher constant factor overheads and is probabilistic. The IBLT data structure consists of a configurable number of cells, where each cell contains an XOR and a count of elements in the cell (Figure 2.2). In the IBLT, t is only an approximate bound on the number of missing elements, so we set the number of cells to some multiplicative factor more than t to increase the probability that we successfully decode.

When the sender or receiver adds an element to the IBLT, we leverage the pseudorandom mapping algorithm of the Rateless IBLT to determine which cells to update [136]. This algorithm maps each element to $O(\log(t))$ cells, with a greater density in the cells with smaller indexes. This resembles rateless error-correcting codes with an infinite stream of coded symbols in that some prefix of a Rateless IBLT with an infinite number of cells can be used to deterministically decode the elements in the data structure. Note that a typical IBLT uses a deterministic hash function to map each element to a small, constant number of cells, which is not conducive to an infinite stream.

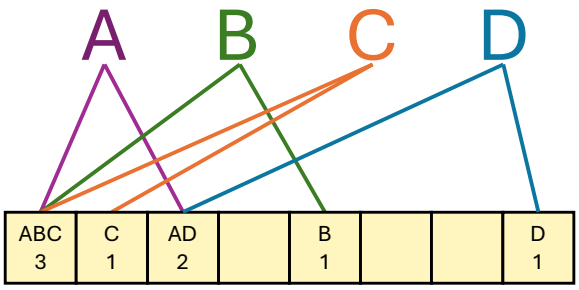


Figure 2.2: The IBLT data structure. Each cell contains an XOR and count of elements in that cell. A deterministic mapping algorithm or hash function maps each element to a small number of cells.

Construction	Description
Strawman 1a	Echo every identifier.
Strawman 1b	Echo a sliding window of identifiers.
Strawman 1c	Echo every identifier over TCP.
Strawman 2	Hash a sorted concatenation of all identifiers.
Power Sums	Encode the identifiers in symmetric power sum polynomial equations.
IBLT	Map the identifiers to cells in an invertible Bloom lookup table data structure.

Table 2.2: Descriptions of the strawman and set-reconciliation-based quACK constructions.

The IBLT also allows the removal of elements and subtraction of two IBLTs with the same number of cells. To find the set difference between the elements in two IBLTs, we first "subtract" the corresponding XORs and counts. To decode the elements encoded in the difference IBLT, we remove elements from cells with a count of 1 until no such cells remain. (Note that the count cannot be negative with subset reconciliation.) Decoding is successful if all counts and XORs are 0 at the end, and can fail probabilistically due to collisions in the mapping algorithm.

2.2.4 Summary

Tables 2.2 and 2.3 summarize the different strawman approaches and set-reconciliation-based constructions of the quACK, and the algorithmic complexities of their costs. Strawman 1 sends an excessive number of packets over the wire while Strawman 2 is impractical to decode when the number of possible subsets of missing packets explodes. Their communication and computation costs scale, respectively, with n, the total number of packets sent over the entire connection. In comparison, the costs of the power sum and IBLT constructions scale only with t, a bounded number of missing packets between quACKs. The IBLT construction is more computationally scalable, but the power sum construction is both deterministic and space-optimal in practice. Next we will explore efficient implementations of the set-reconciliation designs and their practical costs.

	Encode Time	Decode Time	QuACK Size	Cumulative?
Strawman 1a	$O(1)$	$O(1)$	$O(n)$	No
Strawman 1b	$O(1)$	$O(1)$	$O(n)$	No
Strawman 1c	$O(1)$	$O(1)$	$O(n)$	Yes
Strawman 2	$O(1)$	$O(\binom{n}{t})$	$O(1)$	Yes
Power Sums	$O(t)$	$O(t^2)$	$O(t)$	Yes
IBLT	$O(\log t)$	$O(t \log t)$	$O(t)$	Yes

Table 2.3: The algorithmic complexity of the costs of each quACK construction. The encoding time includes inserting an element into the data structure that represents the quACK(s). The decoding time includes finding the elements of either R or $S \setminus R$, given the quACK(s) and S. $n = |S|$ is the total number of sent elements, $t \geq |S \setminus R|$ is an upper bound on the number of missing elements.

2.3 Implementation

The implementation of the quACK should provide a convenient interface to interchangeably use different constructions of the quACK. The two set reconciliation constructions have different tradeoffs that are appropriate for different settings. We define a common API for quACK implementations, shown in Listing 2.1. Our optimized implementations of the power sum and IBLT quACK constructions adhere to this interface and are available as a software library on GitHub [137].

The performance of the implementation must be practical for packet-scale settings. The communication must fit in a single network packet, ideally even less to be considered a control packet for congestion control purposes. For example, a TCP ACK is only 20 bytes on the wire, excluding Ethernet and IP headers. The implementation must also be able to practically decode set differences at millisecond-RTT timescales and encode packets with nanosecond interarrival times. We describe several optimizations in our implementation to meet these performance constraints.

2.3.1 Wire format

The wire format of the quACK reflects its communication cost. Although the actual mechanism in which quACKs are transmitted over the wire is not tied to the design, we assume quACKs to be written in the payload of a UDP datagram, possibly with a small header. Assuming we use 32-bit integers as packet identifiers, the actual quACK consists of three fields:

(i) The 4-byte count of received elements,
(ii) The 4-byte identifier of the last element received, and
(iii) t symbols, where t determines the number of missing elements that can be decoded.

The count and last element received are valuable for efficient decoding in the quACK context. The sender uses the count to calculate the number of missing packets m, which is not known ahead of time, by subtracting the count of the quACK transmitted by the receiver from its own count.

```rust
trait quACK {
    fn count() -> usize;
    fn last_element() -> u32;
    fn encode(element: u32);
    fn remove(element: u32);
    fn sub(rhs: quACK) -> quACK;
    fn decode() -> &[u32];
}
```

Listing 2.1: Pseudocode interface for the implementation of a quACK construction in our software library [137]. An element is an unsigned 32-bit integer referring to a 4-byte packet identifier.

The last element received is an optimization that allows m to represent just the "holes" among the packets being selectively ACKed, excluding the consecutive elements that are in-flight (Section 3.3.3).

The wire format of the symbol depends on the construction of the quACK. In the power sum construction, the symbol is just a power sum of the elements, and uses 4 bytes. In the IBLT construction, the symbol consists of both an XOR of the elements and a count. With 4-byte identifiers, the count uses just one byte for a total of 5 bytes. For the same upper bound on the number of missing elements, one might use more symbols in the IBLT construction to increase the probability of decoding. The total communication cost of the quACK (excluding headers) is either $8 + 4 \cdot t$ or $8 + 5 \cdot t$ bytes, depending on the construction.

Why is a single byte sufficient for the count in an IBLT cell?

Since a quACK needs to fit in a single UDP datagram, this limits us to ≈ 1400 bytes excluding headers. If each XOR is 4 bytes, then there can be at most 350 packets in the set difference. Note that 8-byte identifiers are unnecessary because collisions are unlikely in these small set difference sizes. The count in each symbol needs to be as large as the set difference size. It can also be an unsigned integer because we do subset (and not set) reconciliation, so the differences should always be positive and we can subtract with overflow. An 8-bit integer goes up to 255, which is sufficiently close. Thus the largest IBLT quACK contains 255 symbols or $4 + 4 + 255 \cdot 5 = 1283$ bytes on the wire.

2.3.2 QuACK API

The API for the quACK consists of the basic operations for encoding and decoding elements as described in Section 2.2 (Listing 2.1). The `encode()` and `remove()` functions add and remove a single element, respectively, either by updating all power sums or by updating the XOR and counts in the mapped IBLT cells. The `sub()` function calculates the quACK that represents the set difference between two quACKs by subtracting corresponding symbols, and the `decode()` function returns the decoded elements in that set difference. The values returned by the `count()` and `last_element()`

functions are incrementally updated as elements are added and removed.

2.3.3 Performance optimizations

We explore many optimizations to improve the performance of each quACK. In Sections 2.4 and 2.5, we run microbenchmarks to demonstrate the practicality of our power sum and IBLT quACK implementations, respectively, for in-line packet processing at packet-scale settings. We also evaluate the impact of our design decisions on the computation cost.

Starting with the power sum quACK, we optimize the implementations of modular arithmetic operations for different bit widths. This includes basic techniques like repeated squaring. For 16-bit identifiers only, we precompute power tables that fit in the L3 cache, replacing repeated multiplication with a simple cache access. For 64-bit identifiers, we implement Montgomery modular multiplication [99] to avoid an expensive hardware division for 128-bit integers. Note that the effect of these performance optimizations depend on the CPU processor.

We also implement an alternate decoding method for the power sum quACK that does not require polynomial factorization. Decoding the power sum quACK requires finding the solution to a system of power sum polynomial equations. This boils down to applying Newton's identities (a linear algorithm) and finding the roots of a polynomial equation in a modular field [43]. Factoring a polynomial is asymptotically fast in theory, but the implementation is branch-heavy and complicated [104]. We find that in practice, it is faster to plug in and determine which of the n candidate roots evaluate to zero when there are $n < 40,000$ roots. We sometimes refer to this method as "not factoring".

For the IBLT quACK, we first port the optimized Rateless IBLT [135] to Rust. We simplify the implementation to fit the constraints of the quACK context, such as by reducing the bit widths of the symbols and counts, adjusting the mapping algorithm accordingly, and removing an unnecessary checksum check for doing subset reconciliation. The pseudorandom mapping algorithm uses a square root operation, and is the source of constant factor overheads when t is small even though the IBLT construction has better computational complexity.

2.4 Power sum microbenchmarks

First, we evaluate the computation overheads of the space-optimal and deterministic power sum quACK. We explore different bit widths for the identifier and settle on using 32-bit packet identifiers as the best overall tradeoff. We analyze how the encoding and decoding times of the power sum quACK scale in practice and show that our implementation can achieve practical packet encoding and quACK decoding times in packet-scale settings. Our microbenchmarks use an m4.xlarge AWS instance with a 4-CPU Intel Xeon E5 processor @ 2.30 GHz and 16 GB memory.

Bits	NF	PC	MM [99]	Encode Time	Decode Time	QuACK Size
32				149 ns	3.18 ms	82 bytes
32*	x			151 ns	102 μs	82 bytes
32	x		x	163 ns/pkt	130 μs	82 bytes
16	x			137 ns/pkt	101 μs	42 bytes
16*	x	x		57 ns/pkt	43 μs	42 bytes
16	x	x	x	84 ns/pkt	48 μs	42 bytes
63	x			1.14 μs/pkt	622 μs	162 bytes
63*	x		x	159 ns/pkt	121 μs	162 bytes

Table 2.4: Ablation study of the power sum quACK with different bit widths and optimizations, using $n = 1000$, $t = 20$. The three optimizations are not factoring (NF), pre-computation (PC), and Montgomery multiplication (MM). *Recommended optimizations for that bit width.

2.4.1 Identifier bit widths

The more bits we use in an identifier, the less likely that there will be a collision, but different bit widths have different implications for the computation cost based on which instructions the CPU can use. Modular operations are efficient for 16- and 32-bit integers, fitting within the 64-bit word size (the number of bits that can be processed in one instruction) of most modern CPUs. For example, to multiply two 32-bit integers, we cast them to 64-bit integers, multiply, then take the modulus.

Table 2.4 shows our ablation experiments for the performance optimizations described in Section 2.3.3, as well as our recommended optimizations for each bit width. In the remainder of this dissertation, we use 32-bit (4-byte) identifiers for having the best tradeoff between collision probability, link overheads, and computation cost. The collision probability is too high for 16-bit identifiers and the link overheads too high for 64-bit identifiers, with computation being comparable in all.

2.4.2 Encoding time

The encoding time scales linearly with the threshold of the power sum quACK (Figure 2.3). Encoding a single packet requires updating the t power sums in the power sum quACK. Each power sum can be updated in a constant number of operations, a single modular addition and multiplication, based on the previous power sum. The per-packet encoding time is thus directly proportional to the threshold number of missing packets t at ≈ 3 ns/power sum.

2.4.3 Decoding time

The decoding time of the method of plugging in candidate roots scales linearly with the number of candidate packets n (Figure 2.4a) and the number of missing packets m (Figure 2.4b). Note that this is faster in practice than factoring the polynomial. Decoding takes ≈ 11 ns/candidate/missing,

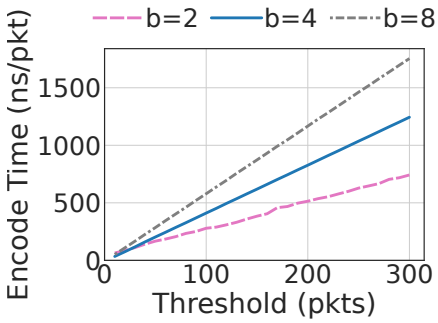
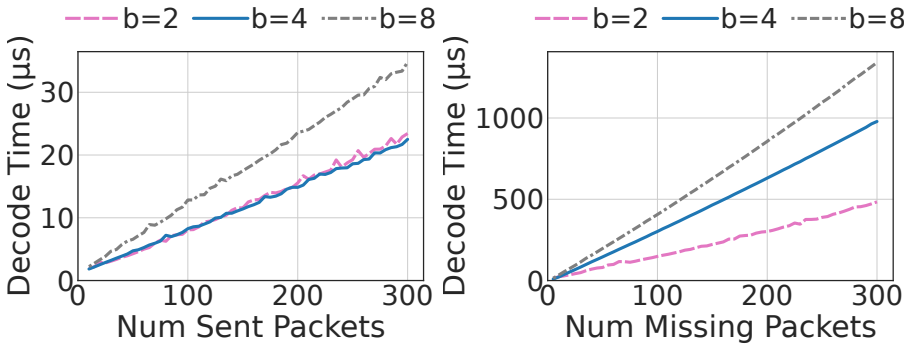


Figure 2.3: The encoding time of the power sum quACK scales linearly with the threshold t, and is generally higher for higher bit widths. If expect the typical threshold to be < 100, then the typical encoding time is on the order of nanoseconds per packet. Average of 100 trials, each trial is the average of 1000 packets.



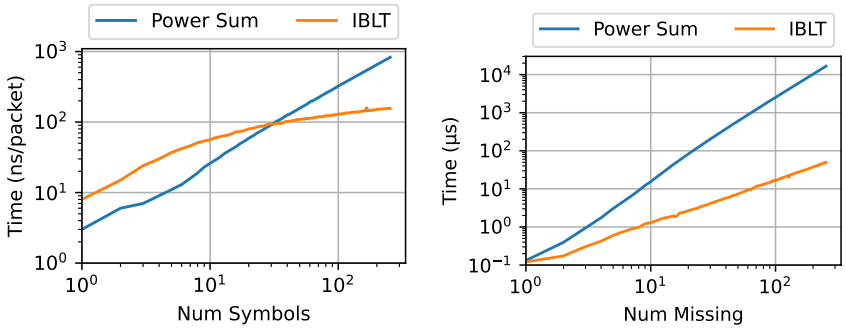
(a) n vs. decode time, where $t = m = 10$. (b) m vs. decode time, where $n = 300$.

Figure 2.4: The decoding time of the power sum quACK scales linearly with both the number of candidate (sent) packets n and the number of missing packets m. Higher bit widths generally indicate higher computation costs. The typical decoding time is in microseconds. Average of 100 trials.

and both n and m are typically a few hundred at most. Thus it is typical for encoding to take a few nanoseconds per packet and decoding to take a few microseconds per quACK.

2.5 IBLT microbenchmarks

The IBLT quACK has better computational complexity than the power sum construction, but how well does this hold in practice? We compare the encoding and decoding times of the two, and find that while the IBLT construction is more computationally scalable, it has non-negligible constant factor overheads at small set sizes due to the mapping algorithm. We also explore how to configure the IBLT quACK based on its probabilistic correctness. We run the microbenchmarks on a single core of an AWS m4.xlarge instance.



(a) Encode time. (b) Decode time.

Figure 2.5: IBLT vs. power sum quACK microbenchmarks. The IBLT quACK is more computationally scalable than the power sum quACK, but the encoding time suffers from constant factor overheads with a smaller number of symbols. The cumulative time of the trials is at least 100 ms.

2.5.1 Encoding and decoding time scalability

The encoding time in the IBLT quACK scales logarithmically with the number of symbols (Figure 2.5a). It is faster than the power sum quACK when there are at least ≈ 30 symbols. This is because each update in the IBLT uses an expensive square root instruction in the pseudorandom mapping algorithm, adding constant factor overheads that impact smaller numbers of symbols.

The decoding time in the IBLT quACK scales roughly linearly with the number of missing packets, and is faster than the power sum quACK for any number of symbols (Figure 2.5b). Recall that the power sum quACK actually uses a decoding method that is linear in the total number of packets sent since the last quACK, not just the number of missing packets m. The IBLT quACK needs to peel each of the m missing packets from $O(\log m)$ cells in the IBLT.

2.5.2 Probabilistic correctness

In the previous microbenchmarks, it is unknown how many symbols are required in the IBLT quACK to decode the same number of m missing packets using power sums. There is a constant multiplicative factor on average (1.23–1.35× [5, 136]), but this is not necessarily representative when m is small, as it often is. Figure 2.6 shows the CDF of the minimum number of symbols required to decode various m as this constant multiplier increases. $m = 1$ trivially requires one symbol, while the multiplier decreases for higher m to achieve the same success rates. This suggests we made need a larger multiplier for small set difference sizes on average.

The sender does not know how many symbols it needs to encode in order to later decode a certain number of missing packets. Using the power sum quACK, the threshold can be set to exactly m symbols. In the IBLT quACK, $4 \cdot m$ symbols have at least a 98.8% success rate for all m evaluated in Figure 2.6. When m is large, the IBLT quACK utilizes more of the link than the power sum quACK,

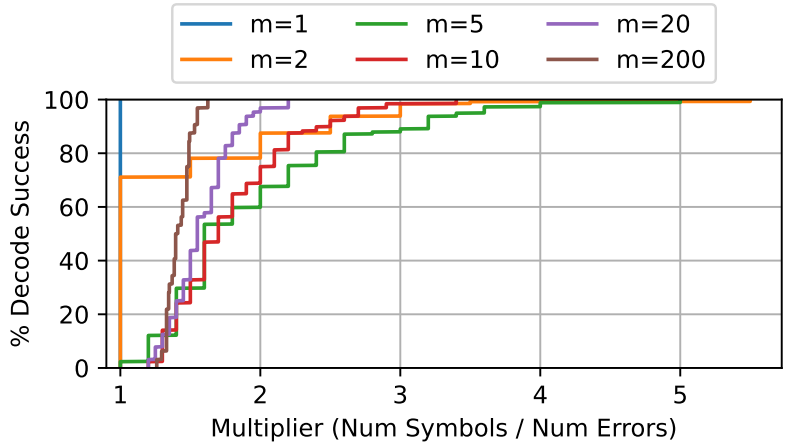


Figure 2.6: The CDF of the minimum number of symbols t needed to successfully decode an IBLT quACK for various numbers of missing packets m, 100000 trials.

	Encode Time	Decode Time	Num Packets	Num Bytes	Protocol
Strawman 1a	N/A	N/A	500	4	UDP
Strawman 1b	N/A	N/A	500	$4 \cdot window$	UDP
Strawman 1c	N/A	N/A	≥ 500	4	TCP
Strawman 2	27 ns/pkt	88 Myears	1	36	UDP
Power Sums	26 ns/pkt	15.5 μs	1	48	UDP
IBLT	42 ns/pkt	1.3 μs	1	≥ 58	UDP

Table 2.5: A concrete realization of the costs of each quACK construction. The computation and communication costs of the power sum and IBLT quACK constructions are appropriate for packet-scale settings, while the strawmen are insufficient in one or more aspects. Parameters: $n = 500$, $t = 10$, and the elements are 32-bit integers.

but if the sender and receiver each maintain more symbols, the sender has a greater worst-case tolerance for errors with more scalable encoding overheads. With rateless quACKs (Section 4.4.3), the receiver can encode much larger numbers of symbols while the sender decodes fewer symbols in the common case.

2.6 Summary

Our set-reconciliation-based constructions of the quACK can efficiently refer to and acknowledge a set of randomly encrypted packets. These constructions can practically decode set differences at millisecond-RTT timescales and encode packets with nanosecond interarrival times, summarized in Table 2.5. They are also cumulative representations of the packets seen by the receiver. The

power sum construction is a space-optimal and deterministic solution, while the probabilistic IBLT construction is more computationally scalable in practice for larger set sizes. Next, we will explore the use of the power sum quACK in the Sidekick protocol in (Chapter 3) and settings where the computational scalability of the IBLT quACK is a desirable property (Chapter 4).

Chapter 3

Sidekick Protocols: Secure in-network assistance for data senders

This chapter describes *Sidekick protocols*: an approach to in-network assistance for secure transport protocols. In this approach, in-network intermediaries help endpoints by sending information adjacent to the underlying connection, which remains opaque and unmodified on the wire. Described in Chapter 2, this information is called a quACK, and the quACK is the key technical tool that allows proxies to usefully refer to a set of packets that they have received even when the transport headers are randomly encrypted. In real-world and emulation-based evaluations, the Sidekick protocol improves performance in several scenarios: early retransmission over lossy Wi-Fi paths, proxy acknowledgments to save energy, and a path-aware congestion-control mechanism we call *PACUBIC*. The path-aware CUBIC congestion control algorithm emulates the congestion response of a "split" connection from the perspective of the data sender at the endpoint, without buffering packets at the proxy.

3.1 Introduction

In the Internet's canonical model, transport is end-to-end and implemented only in hosts; only hosts think about connections, reliable delivery, or flow-by-flow congestion control (Chapter 1). In practice, however, the best behavior for a transport protocol depends on the particulars of the network path. An appropriate retransmission or congestion-control scheme for a heavily-multiplexed wired network wouldn't be ideal for paths that include a high-delay satellite link, Wi-Fi with bulk ACKs and frequent reordering, or a cellular WWAN [49, 80].

By the 1990s, many networks had broken from the canonical model by deploying in-network TCP

accelerators, also known as "performance-enhancing proxies" (PEPs) [50]. TCP PEPs can split an end-to-end connection into multiple concatenated connections [17, 33, 45, 55, 68], buffer and retransmit packets over a lossy link [8, 111], virtualize congestion control [29, 56, 96], resegment the byte stream, and enable forward error correction, ECN, or other segment-specific enhancements. Because TCP isn't encrypted or authenticated, PEPs can achieve this without the knowledge or cooperation of end hosts. Roughly 20–40% of Internet paths cross at least one TCP PEP [41, 57].

While many flows benefit from PEPs, their use carries a cost: protocol ossification [41, 103]. In response, post-TCP transport protocols have been designed to be impervious to meddling middleboxes by encrypting and authenticating the transport header. We refer to these newer transport protocols as "opaque" transport protocols. The most prevalent is QUIC [65], found in billions of installed Web browsers and millions of servers [143]; other secure transport protocols are used in WebRTC/SRTP [107], Zoom [144], BitTorrent [10], and Mosh/SSP [133].

This opacity means that middleboxes can't interpose themselves on a connection or understand the sequence numbers of packets in transit. This prevents PEPs from providing assistance, in some situations reducing the performance of opaque transport protocols [12, 13, 74, 80, 89]. It's possible to co-design protocols and PEPs to permit assistance from credentialed middleboxes [37, 46, 66, 121], but challenging to do so without tightly coupling these components and risks ossification and fragility. The resulting tension between the performance enhancements of PEPs and protocol ossification makes it challenging to provide in-network assistance to secure transport protocols.

In this chapter, we propose a second protocol called the **Sidekick protocol** to be spoken on an adjacent connection between an end host and a PEP. The contents of the adjacent connection are *about* the packets of the underlying "base" connection. Sidekick proxies assist end hosts by reporting what they've observed about the packets of the encrypted base connection, leaving the transport protocol unchanged on the wire: an encrypted end-to-end connection between hosts, opaque to middleboxes and free to evolve. End hosts use this information to influence decisions about how and when to send packets on the base connection, approximating some of the performance benefits of traditional PEPs. No PEPs are credentialed to decrypt the transport protocol's headers.

One key technical challenge in this approach is how Sidekick proxies can efficiently refer to a set of packets in an encrypted base connection when these packets appear random to the middlebox. Referring to a range of, say, 100 encrypted packets in the presence of loss and reordering is not as simple as saying "up to 100" in the case of packets with cleartext sequence numbers. To solve this problem, we leverage the power sum quACK from Chapter 2. The **quACK** is a mathematical tool that concisely represents a selective acknowledgment of encrypted packets when there is a practical bound on the maximum number of "holes" among the packets being ACKed.

A second challenge is how the end host should use information from a Sidekick proxy to obtain a performance benefit for its base connection. Since the performance benefit comes from changing behavior at the end host rather than the middlebox, transport protocols need to incorporate this

information into their existing algorithms for, e.g., loss detection and retransmission, which have gotten increasingly complex over time. To explore this, we designed a Sidekick protocol and integrated it into client implementations with path-aware modifications in three scenarios:

- A low-latency audio stream over an Internet path that includes a lossy, low-latency Wi-Fi path segment, followed by a reliable, higher-latency WAN path segment. Can using the Sidekick protocol reduce the de-jitter buffer delay by triggering earlier retransmissions on loss?
- An upload over the same path. Can a secure transport protocol like QUIC, aided by a Sidekick proxy at the point between these two path segments, match the throughput of TCP over a connection-splitting PEP?
- A battery-powered receiver downloading data from the Internet over Wi-Fi. If the Wi-Fi access point sends quACKs over a Sidekick connection on behalf of the receiver, can it reduce the number of times the receiver's radio needs to wake up to send an end-to-end ACK?

A third technical challenge is how knowledge about *where* loss occurs along a path should influence a congestion-control scheme. The challenge in any such scheme is how to maximize the congestion window to achieve high throughput while sharing the network fairly with competing flows. We present a path-aware modification to the CUBIC congestion-control algorithm [52], which we call **PACUBIC**, that approximates the congestion-control behavior of a PEP-assisted split TCP CUBIC connection while making its decisions entirely on the host.

Summary of results. We evaluated the three scenarios above, integrating the Sidekick protocol with a media client based on the WebRTC standard and an HTTP/3 client using `libcurl` and the Cloudflare implementation of QUIC [28]. In real-world experiments using an unmodified local Wi-Fi network to access our nearest AWS datacenter, the Sidekick proxy was able to trigger early retransmissions to fill in gaps in the audio of a latency-sensitive audio stream, reducing the receiver's de-jitter delay from 2.3 seconds to 204 ms—about a 91% reduction (Figure 3.5). The Sidekick proxy was also able to improve the speed of an HTTP/3 (QUIC) upload by about 50%.

In emulation experiments of the "battery-powered receiver" scenario, the Sidekick proxy was able to reduce the need for the receiver to send ACKs by sending proxy acknowledgments on its behalf—ACKs the sender used to advance its flow-control and congestion-control windows. The receiver only needed to wake up its radio to send occasional end-to-end ACKs, which the sender used to discard data from its buffer (Figure 3.6c).

Also in an emulation experiment, we confirmed that PACUBIC's achieved throughput approximates that of a split CUBIC connection (two TCP CUBIC connections separated by a PEP), responding to loss events proportionally to the delay of the path segment on which the loss occurs (Figure 3.9). The results indicate that the gains from using the Sidekick protocol do not come at the expense of congestion-control fairness relative to a split CUBIC connection.

The rest of this chapter discusses the three motivating scenarios for Sidekick protocols in more detail (Section 3.2). We describe the concrete Sidekick protocol and path-aware sender modifications we built around power sum quACKs (Section 3.3), including path-aware CUBIC congestion control (Section 3.4) and its integrations with two base protocols (Section 3.5). We evaluate (Section 3.6) the use of the protocol in real-world (Section 3.7) and emulation experiments (Section 3.8). Finally, we discuss limitations to our approach (Section 3.9) and conclude (Section 3.10).

3.2 Motivating scenarios

We focus on three scenarios where end hosts benefit from in-network assistance. In each one, a proxy provides feedback in the form of a quACK to an end host: the data sender (Figure 3.1). Recall that a quACK is a "cumulative ACK + selective ACK" over encrypted packet identifiers. The data sender uses this feedback to influence its behavior on the base connection, without altering the wire format.

To be clear: the Sidekick protocol is not tied to a specific base protocol nor to how the end hosts use the quACK information. The base protocol does not need to be reliable, nor to have unique datagrams—we implemented and evaluated the same Sidekick protocol and the same proxy behavior across the different scenarios in this paper.

3.2.1 Low-latency media retransmissions

Consider a train passenger using on-board Wi-Fi to have a low-latency audio conversation, using WebRTC/SRTP [107], with a friend. The end-to-end network path contains a low-latency, high-loss "near" path segment (the Wi-Fi hop) followed by a high-latency, low-loss "far" path segment (the cellular and wired path over the Internet). The friend probably suffers from poor connection quality, experiencing drops in the audio stream or high de-jitter buffer delays from waiting for retransmitted packets to be played in order (Figure 3.5a in the real world, Figure 3.6a in emulation).

In the Sidekick approach, a PEP on the Wi-Fi access point sends quACKs to the media client on the passenger's laptop, assisting the base connection. The passenger's media client uses quACKs to retransmit packets sooner than they would have using negative acknowledgments (NACKs) from the friend. The end result is similar to the effect of PEPs such as Snoop [8] and Milliproxy [111] that leverage TCP's cleartext sequence numbers to trigger early retransmission on lossy wireless paths.

3.2.2 Emulating split congestion control in an HTTP/3 file upload

Consider the same train passenger as before but uploading a large file over the Internet with a reliable transport protocol. If the protocol were TCP, the train could deploy a split TCP PEP at the access point. The split connection allows quick detection and retransmission of dropped packets on the lossy Wi-Fi segment, while opening up the congestion window on the high-latency cellular segment.

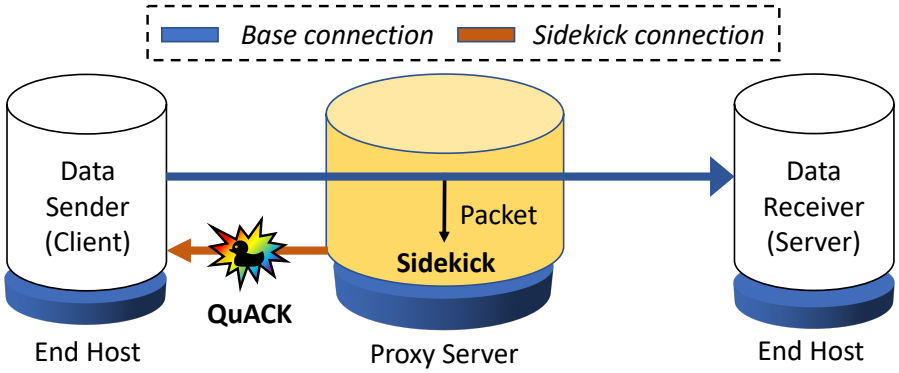


Figure 3.1: The proxy generates quACKs, in-network acknowledgments, based on the encrypted packets it observes in the base protocol. It quACKs to an end host, the data sender, which sends or resends packets on the base protocol as a result. Although we only show one side of the connection, the Sidekick could assist either end host of a bidirectional flow.

However, secure transport protocols like QUIC can't benefit from (nor be harmed by) connection-splitting PEPs. Without a PEP, QUIC relies on end-to-end mechanisms over the entire path to detect losses, recover from them, and adjust the congestion-control behavior. This leads to reduced upload speeds (Figure 3.5b in the real world, Figure 3.6b in emulation).

With help from the same Sidekick PEP, the QUIC client combines information from quACKs and end-to-end ACKs to emulate the congestion-control behavior of a split TCP connection (Section 3.4). The application considers whether packets are lost on the near or far path segments, and adjusts the congestion window accordingly while respecting the opacity of the end-to-end base connection. The application also retransmits the packet as soon as the loss has been detected.

The only guarantee the proxy makes to the client via the quACK is that it has received some packets. To respect the end-to-end reliability contract with the server, the client does not delete packets that may need to be transmitted until it receives an end-to-end ACK from the server, even if the packet has been quACKed by the proxy.

3.2.3 Battery-saving ACK reduction

Now consider a battery-powered device downloading a large file from the Internet. To reduce how often the device's radio needs to wake up, saving energy, the base connection can reduce the frequency of end-to-end ACKs the device sends to the data sender. ACK reduction has also been shown to improve performance by reducing collisions and contention over half-duplex links [31, 85]. The ACK frequency can be configured with a TCP kernel setting or proposed QUIC extension [64].

However, ACK reduction can also degrade throughput [30, 31] (Figure 3.6c in emulation). The data sender receives more delayed feedback about loss, and has to carefully pace packets to avoid bursts in the large delay between ACKs. One proposal has the PEP acknowledge packets on behalf

of the data receiver [71], leveraging cleartext TCP sequence numbers. This proposal does not apply to secure transport protocols without these sequence numbers.

In this scenario, a Sidekick proxy at a Wi-Fi access point or cellular base station quACKs to the data sender on behalf of the battery-powered device. The device still occasionally wakes up its radio to send ACKs, but the data sender uses the more frequent quACKs from the proxy to advance its flow-control and congestion-control windows.

The data sender respects the end-to-end reliability contract by only deleting packets in response to end-to-end ACKs, but disregards the data receiver's flow control by using quACKs to advance the flow-control window. If the data sender only used end-to-end ACKs to advance the window, it would waste time waiting between ACKs to send packets with too small a window, and need to pace sent packets on receiving a large ACK with too large a window.

3.3 Design

This section describes our design for a Sidekick protocol built around quACKs. This includes the setup and configuration of a Sidekick connection, how a data sender detects loss from a quACK, and a path-aware modification to CUBIC called PACUBIC for congestion-controlled base protocols.

3.3.1 PEP discovery mechanism

PEPs have traditionally been deployed as transparent proxies, silently interposing on end-to-end connections when they are on the network path. Endpoints therefore need a way to detect Sidekick proxies and inform them of where to send quACKs. Because of network address translation, all communication to the proxy must be initiated by the endpoint or use the same IP addresses and port numbers of the base connection.

Our design has endpoints signal Sidekick support by sending a distinguished packet containing a 128-byte `sidekick-request` marker along the base connection. Such inline signaling could confuse hosts when only one endpoint supports Sidekick, but this approach target protocols such as QUIC that discard cryptographically unauthenticated data anyway. It would be cleaner to signal support through out-of-band UDP options [127], which we may hope to do once standardized. The proxy replies to a `sidekick-request` packet by sending a special packet from the other endpoint's IP address and port number back to the original endpoint. This packet contains a `sidekick-reply` marker, an opaque session ID, and an IP address and port number for communicating with the proxy.

In systems that explicitly configure proxies, such as Apple's iCloud Private Relay [4] based on MASQUE [75, 77], proxies can simply negotiate the Sidekick connection during session establishment. However, any explicit proxy configuration has the disadvantage that the proxy may not be on the most direct network path. Explicit proxies also make mobility and handoffs more challenging.

```
Init
epoch: u32;
base_conn: [u8; 12];
quack_ty: u8;
num_symbols: u8;
id_offset: u16;
quack_pkts: u16;
quack_ms: u16;
```

```
Reset
epoch: u32;
errno: u32;
```

(a) Client payloads.

```
InitACK
epoch: u32;
udp_port: u16;
errno: u32;
```

```
QuACK
count: u32;
last_element: u32;
code: Vec<Symbol>;
```

(b) Proxy payloads.

Figure 3.2: Sidekick protocol messages to configure and reset the Sidekick connection.

Security.

A malicious third-party could execute a reflection amplification attack that generates a large amount of traffic while hiding its source. This is possible because the endpoint requests quACKs to a different port and (for some carrier-grade NATs) IP address from the underlying session. To mitigate this, each quACK can contain a quota, initially 1, of remaining quACKs the proxy will send as well as an updated session ID. The quota and session ID ensure only the endpoint can increase the quota or otherwise reconfigure the session.

An adversarial PEP could send misleading information to the endpoint. Note that only on-path PEPs can send credible information, since they refer to unique packet identifiers. To mitigate this, the endpoint can consider PEP feedback along with end-to-end metrics to determine whether to keep using the PEP. The endpoint can always opt out of the PEP and fall back to end-to-end mechanisms, and the PEP cannot actively manipulate traffic any more than outside a Sidekick setting.

3.3.2 Sidekick protocol messages

Once the endpoint has an IP address and port for communicating with the proxy, it can establish an adjacent Sidekick connection with the proxy from a different local UDP port. The endpoint can then send various messages to the proxy to configure the connection and reset bad state (Figure 3.2a), while receiving quACKs to decode and adjust the behavior of the base connection (Figure 3.2b).

Configuration.

In the `Init` message, the endpoint configures (i) which quACK construction to use and the number of symbols, (ii) a byte offset into the packet payload at which to compute the 4-byte identifier, and (iii)

the interval at which the PEP should send quACKs. The proxy accepts or rejects these configurations with an `InitACK` and, if accepted, immediately begins to send `QuACK` messages.

The number of symbols represents the upper bound on the number of missing packets between quACKs, in practice the number of "holes" among the packets that are selectively ACKed. This bound depends on the quACK interval, and should be set based on how precise loss detection needs to be and other link qualities. For example, the number of symbols should be larger to detect congestive loss in the queue of a bottleneck link, or smaller to detect transmission error on a lossy link.

The quACK interval is expressed in terms of time or number of packets, e.g., every N milliseconds or every N packets, as in a TCP delayed ACK. The endpoint determines the desired interval based on its estimated RTT of the base connection and its application objectives. For example, the endpoint may want to receive quACKs more frequently for latency-sensitive applications or lower-RTT paths.

Resets.

The Sidekick protocol allows the endpoint to tell the PEP to reinitialize the quACK. The `Reset` message acts as a synchronization point in the base connection in which both the endpoint and proxy disregard old packets and start a new cumulative quACK. This is helpful if the quACK becomes invalid, such as if the number of missing packets exceeds the maximum bound. It is always safe to reset the quACK, or even to ignore the PEP entirely and fall back to the base protocol's end-to-end mechanisms.

3.3.3 Path-aware sender behavior enabled by the quACK

Now we discuss sender-side behaviors that are enabled by the Sidekick protocol and which are helpful across several scenarios: detecting packet loss from a decoded quACK, earlier retransmissions, and path-aware congestion control to emulate the congestion response of a split TCP connection.

Loss detection.

The endpoint knows definitively which packets have been received by the proxy from a decoded power sum quACK. Next, it must determine from the remaining packets which ones have been dropped and which are still in-flight, including if there has been a reordering of packets. In-flight packets are later classified as received or dropped based on future quACKs.

When there is no reordering, the packets that are dropped are just the "holes" among the packets that are selectively ACKed by the quACK. In particular, these are the holes when considering sent packets in the order they were sent up to the last element received, which represents the last selective ACK. To identify these dropped packets, the endpoint encodes t cumulative power sums of its sent packets up to the last element received. The difference between these power sums and the power sums in the quACK represents the dropped packets. The endpoint "removes" the identifiers of

dropped packets from its cumulative power sums, ensuring that the only packets that contribute to the threshold limit are those that went missing since decoding the last quACK.

To account for reordering in loss detection, our design has the Sidekick protocol use an algorithm similar to the 3-duplicate ACK rule in TCP [11, 124]. In TCP, if three or more duplicate ACKs are received in a row, it is a strong indication that a segment has been lost. Our design similarly considers a packet lost only if three or more packets sent after the missing packet have been received. Other mechanisms could involve timeouts for individual packets similar to the RACK-TLP loss detection algorithm for TCP [25].

Earlier retransmissions.

On detecting loss, a data sender can immediately retransmit missing packets. Assuming reliability is a desired property of the base protocol, this allows the data sender to retransmit the packet earlier than if it had waited for an end-to-end ACK.

Path-aware congestion control.

The congestion response of a congestion-controlled base protocol to lost packets that they retransmit due to quACKs determines whether the connection achieves any throughput improvements. Consider what happens if the base protocol treats loss detected from quACKs the same as from end-to-end ACKs. Since loss is a binary signal in "loss-based" congestion control algorithms such as CUBIC [52], the connection would have the same, low throughput as an end-to-end connection. If the base protocol had no congestion response at all, it would not be fair to connections without Sidekick assistance, and in the worst case, retransmissions from quACKs could cause congestion collapse.

Thus congestion-controlled base protocols must have some congestion response to loss from quACKs to ensure friendliness with existing congestion control schemes that do consider the loss. Our benchmark for a fair response is to achieve the same throughput as TCP CUBIC in the presence of a connection-splitting TCP PEP in the same settings. We refer to CUBIC with a split connection as "split CUBIC", which has distinct behavior from "end-to-end CUBIC" without a connection-splitter and has notably higher throughput. However, a path-aware congestion response can only change the behavior at the endpoint, since packets are opaque to the proxy, in order to emulate the split congestion-control behavior of split CUBIC. We describe the algorithmic details of in Section 3.4.

3.4 Path-aware CUBIC congestion control

Here, we propose PACUBIC, or path-aware CUBIC, an algorithm that emulates "split CUBIC" behavior. PACUBIC uses knowledge of where loss occurs along a network path to improve connection throughput compared to end-to-end CUBIC, while remaining fair to competing flows.

3.4.1 Background: CUBIC

Recall that CUBIC [52] reduces its congestion window by a multiplicative decrease factor, $\beta = \beta^* = 0.7$, when observing loss (a congestion event), and otherwise increases its window based on a real-time dependent cubic function with scaling factor $C = C^* = 0.4$:

$$cwnd = C(T - K)^3 + w_{max} \text{ where } K = \sqrt[3]{\frac{w_{max}(1-\beta)}{C}}.$$

Here, *cwnd* is the current congestion window, w_{max} is the window size just before the last reduction, and T is the time elapsed since the last window reduction.

3.4.2 PACUBIC algorithm

While a split CUBIC connection has *two* congestion windows, end-to-end PACUBIC only has *one* window representing the in-flight bytes of the end-to-end connection. Conceptually, we want an algorithm that enables PACUBIC's single congestion window to match the sum of the split connection's two congestion windows.

PACUBIC effectively makes it so that we reduce and grow *cwnd* proportionally to the number of in-flight bytes on the path segment of where the last congestion event occurred. Let r be the estimated ratio of the RTT of the near path segment (between the data sender and the proxy) to the RTT of the entire connection (between end hosts). We use r as a proxy for the ratio of the number of in-flight bytes. If the last congestion event came from a quACK, we use the same real-time dependent cubic function but with the following constants:

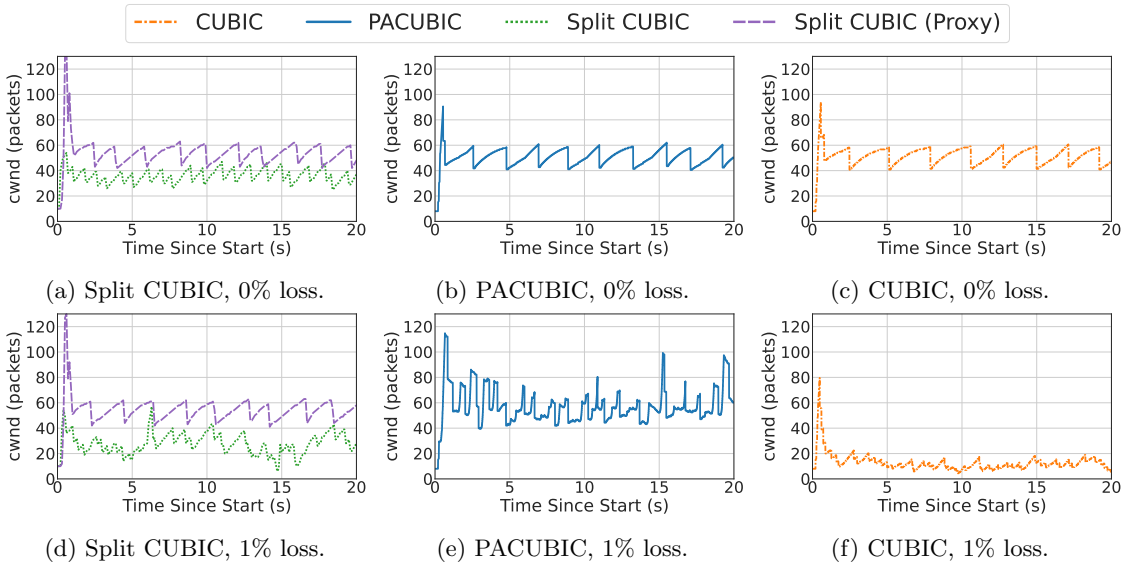
$$\beta = 1 - r(1 - \beta^*) \text{ and } C = \frac{C^*}{r^3}.$$

If the last congestion event came from an end-to-end ACK, then we use the original β and C.

While this algorithm resembles the congestion behavior of split CUBIC, it is simply an approximation. PACUBIC does not know the exact number of bytes in-flight on each path segment, and the sum of the two congestion windows is simply a heuristic for an inherently different split connection. The main takeaway is that knowing where loss occurs can inform congestion control. We generally hope that quACKs can lead to the development of smarter, path-aware congestion-control schemes.

3.4.3 Intuitive correctness analysis

Here, we dive deeper into the intuition behind the PACUBIC constants β and C, including how they were derived and why the PACUBIC algorithm achieves similar congestion behavior to split CUBIC.



(a) Split CUBIC, 0% loss. (b) PACUBIC, 0% loss. (c) CUBIC, 0% loss.

(d) Split CUBIC, 1% loss. (e) PACUBIC, 1% loss. (f) CUBIC, 1% loss.

Figure 3.3: Congestion window of a long-running upload in Scenario #2 (Table 3.2) with 0% and 1% loss on the near path segment. The cwnd is measured at the data sender, except for split CUBIC whose split connection also has a cwnd at the proxy. PACUBIC reacts to every congestion event while keeping the cwnd high. CUBIC performs poorly when there is loss on the near path segment. CUBIC and PACUBIC are implemented in QUIC, while split CUBIC is from TCP with a PEP.

Analysis.

Consider the same network topology as Figure 3.1 in which a data sender uploads a large file to a data receiver, with help from a Sidekick proxy in the middle of the connection. The near path segment connects the data sender to the proxy, and the far path segment connects the proxy to the data receiver. The near path segment is low-delay with varying random loss, and the far path segment is high-delay with no random loss. The far path segment is the bottleneck link in terms of bandwidth. The actual link parameters are the same as in Scenario #2 of Table 3.2.

To conceptually motivate PACUBIC, let's first discuss how split CUBIC would behave in this setting. Consider the congestion windows of each half of the split connection, one taken at the data sender and one at the proxy (Figures 3.3a and 3.3d). The far path segment experiences only congestive loss, leading the window at the proxy to fluctuate around the segment's bandwidth-delay product regardless of the loss on the near path segment. The window at the data sender independently determines whether the packets that reach the proxy will be able to fully utilize the window set at the far path segment. We will see that the data sender is able to achieve this at low random loss rates, but becomes the bottleneck as loss rates increase (Figure 3.9).

While split CUBIC has two windows, PACUBIC only has one window representing the in-flight bytes of the end-to-end connection. PACUBIC considers loss detected from both quACKs and end-to-end ACKs. Conceptually, we want an algorithm that would enable PACUBIC's single congestion

window to match the sum of CUBIC's two congestion windows, or the total number of in-flight bytes.

With no random loss on the near path segment, PACUBIC (Figure 3.3b) behaves the same as end-to-end CUBIC (Figure 3.3c). The congestion window is entirely governed by end-to-end ACKs since the far path segment is the bottleneck link. Note that while the sender may be able to deduce that a loss occurred on the far path segment by combining info from the quACK with the end-to-end ACK, PACUBIC conservatively treats the loss as occurring anywhere on the path.

With some random loss on the near path segment, PACUBIC grows and reduces *cwnd* based on where the last congestion event occurs (Figure 3.3e). Note that if the congestion window *cwnd* represents the bytes in-flight in the end-to-end connection, then $r \cdot cwnd$ represents the proportion of bytes in-flight on the near path segment. At a high level, if the data sender discovers loss on the near path segment via the quACK, it holds the $(1-r) \cdot cwnd$ portion of the "far window" constant while applying the CUBIC algorithm to the remaining $r \cdot w_{max}$ of the "near window", representing the bottleneck link.

Mathematically, instead of reducing w_{max}, the window size just before the last reduction, by $(1-\beta^*) \cdot w_{max}$, PACUBIC reduces it by only $[1-(1-r(1-\beta^*))] \cdot w_{max} = r(1-\beta^*) \cdot w_{max}$. That is r times the original reduction, a *smaller* amount. We use the RTT ratio r (near path segment to end-to-end) as a proxy for the ratio of the number of in-flight bytes.

Similarly, instead of using a cubic growth function with scaling factor C^* and inflection point $K = K^* = \sqrt[3]{w_{max}(1-\beta^*)/C^*}$, we use a larger scaling factor $C = C^*/r^3$ and thus a shorter inflection point:

$$K = \sqrt[3]{\frac{w_{max}(1-\beta)}{C}} = \sqrt[3]{\frac{r \cdot w_{max}(1-\beta^*)}{C^*/r^3}} = r^{4/3} \cdot K^*.$$

The shorter inflection point leads the congestion window to *grow more quickly* since the sender also reacts to feedback about loss more quickly over the low-delay link.

At times, there can be loss detected both in quACKs and in end-to-end ACKs. The end-to-end ACKs have a greater effect since they reduce the congestion window by a larger proportion, until the remaining path segment with loss is the bottleneck link. In this scenario with loss, the bottleneck link at equilibrium is the near path segment. At this point, the quACK primarily determines the congestion window updates. If the far path segment were to become the bottleneck again, the data sender would detect a congestion event via the end-to-end ACK.

Limitations.

PACUBIC has several limitations. Although it beats end-to-end CUBIC, we will see that it still performs worse than split CUBIC, especially at high loss rates (Figure 3.9). Also, it doesn't consider loss on the far path segment any differently than end-to-end CUBIC, unlike split CUBIC which treats the two split connections independently. PACUBIC emulates the congestion control behavior and fairness of split CUBIC fairly well as a heuristic, but would benefit from an analysis in a wider

variety of network scenarios. It would also benefit from a side-by-side fairness comparison against congestion control algorithms such as BBR [23] that perform well in the same scenarios. We'd like a takeaway of PACUBIC to be that knowing where loss occurs can cleverly inform congestion control.

3.5 Implementation

Module	Language	LOC
Media server/client + integration	Rust	478
`quiche` client integration	Rust	1821
`libcurl` client integration	C	1459
Sidekick proxy binary	Rust	833

Table 3.1: Lines of code in the Sidekick protocol implementation.

We now describe our implementation of the Sidekick protocol [138]. We integrated Sidekick functionality with a simple media client for low-latency streaming and an HTTP/3 QUIC client. We used the power sum construction of the quACK from our quACK library in Section 2.3. The total implementation of the proxy and client integrations used 4591 LOC (Table 3.1).

3.5.1 End-to-end applications

The baselines we evaluated against were the performance of two secure transport protocols without proxy assistance, and the TCP friendliness of a split TCP CUBIC connection.

Low-latency media application.

We implemented a simple server and client in Rust for streaming low-latency media based on the WebRTC standard. The client sends a numbered packet containing 240 bytes of data every 20 milliseconds, representing an audio stream at 96 kbit/s. The sequence number is encrypted on the wire. The server receives these packets. If it receives a nonconsecutive sequence number, it sends a NACK back to the client that contains the sequence number of each missing packet. The client's behavior on NACK is to retransmit the packet. The server retransmits NACKs, up to one per RTT, until it has received the packet.

The server's application behavior is to store incoming packets in a buffer and play them as soon as the next packet in the sequence is available. The de-jitter buffer delay is the length of time between when the packet is stored to when it can be played in-order. Some packets can be played immediately.

HTTP/3 file upload application.

We used the popular `libcurl` [87] file transfer library as the basis for our client, and an `nginx` webserver. The client makes an HTTP POST request to the server. Both are patched with

`quiche` [28], a production implementation of QUIC from Cloudflare, to provide support for HTTP/3.

For our TCP baselines, we used the same file upload application with the default HTTP/1.1 server and client in `nginx` and `libcurl`. We used a connection-splitting TCP PEP called PEPsal [17] that intercepts the TCP SYN packet in the three-way handshake, pretends to be the other side of that connection, and initiates a new connection to the real endpoint. Both clients use CUBIC for congestion control.

3.5.2 Client integrations with Sidekick

In each application, we modified only the *client* to speak the Sidekick protocol and respond to in-network feedback. The server remained unchanged. The modifications were in two parts: following the discovery mechanism to establish bi-directional communication with the proxy, and using the information in the quACK to modify transport layer behavior.

Low-latency media client.

The media client has two open UDP sockets: one for the base connection and one for the Sidekick connection. When it receives a quACK, it detects lost packets without reordering and immediately retransmits them. The protocol does not have a congestion window nor a flow-control window. The client also sends `Init` and `Reset` messages over the Sidekick connection.

HTTP/3 file upload client.

The HTTP/3 client similarly has an adjacent UDP socket for the Sidekick connection on which it receives quACKs and sends `Init` and `Reset` messages. The client passes the quACK to our modified `quiche` library, which interprets the quACK and makes transport layer decisions. From the client's perspective, `quiche` tells `libcurl` exactly what bytes to send over the wire.

Our modified `quiche` library uses the quACK to inform the retransmission behavior, congestion window, and flow-control window. The library immediately retransmits lost *frames* in a newly-numbered packet, as opposed to the lost *packet*, similar to QUIC's original retransmission mechanism. We implement PACUBIC, described in Section 3.4. We also move the flow-control window (without forgetting packets in the retransmission buffer), but only in the ACK reduction scenario, when the congestion window is nearly representative of that of the Sidekick connection's path segment.

3.5.3 Sidekick proxy

Our proxy sniffs incoming packets of a network interface using the `recvfrom` system call on a raw socket. It stores a hash table using Rust's standard library `HashMap` that maps socket pairs to their respective quACKs, and incrementally updates quACKs for flows that have requested Sidekick assistance. It also sends quACKs at their configured intervals and listens for configuration messages.

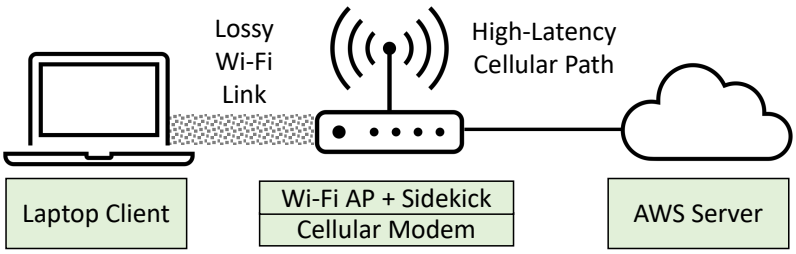


Figure 3.4: Real-world experimental setup.

	Data Sender (Client) ↔ Proxy	Proxy ↔ Data Receiver (Server)	QuACK Interval	Num Symbols
#1 Low-latency media	1ms, 100 Mbit/s, 3.6% loss	25ms, 10 Mbit/s, 0% loss	2 pkts	8
#2 Connection-splitting PEP emulation	1ms, 100 Mbit/s, 1.0% loss	25ms, 10 Mbit/s, 0% loss	30 ms	10
#3 ACK reduction	25ms, 10 Mbit/s, 0% loss,	1ms, 100 Mbit/s 0% loss	15 ms	50

Table 3.2: Experimental scenarios. Link 1 connects the data sender (client) to the proxy, while Link 2 connects the proxy to the data receiver (server). The quACK interval and number of symbols represent our Sidekick configuration in the `Init` message.

3.6 Evaluation methodology

We modeled the scenarios from Section 3.2 using the same m4.xlarge AWS instance as before. We want to understand if the Sidekick protocol is robust in a real-world environment and to answer the following questions in a more controlled emulation environment:

1. Can using the Sidekick protocol improve the performance of secure transport protocols in a variety of scenarios while preserving the end-to-end behavior of the base protocols?
2. Can a path-aware congestion control algorithm match the fairness of split CUBIC?
3. How do the CPU overheads of encoding quACKs impact the Sidekick proxy?
4. What link overheads does the power sum quACK add and how does it compare to the strawmen?

Emulation experiments.

We emulated a two-hop network topology (Figure 3.1) in mininet, configuring the link properties using `tc`. In emulation, we represented each link by a constant delay (with variability induced by the queue), a random loss percentage, and a maximum bandwidth. Table 3.2 describes the parameters for each link to model—e.g., lossy Wi-Fi or a high-latency cellular path—as well as the metrics for success in that scenario. Link 1 connects the data sender (client) to the proxy, while Link 2 connects the proxy to the data receiver (server). The proxy can either be a Sidekick proxy, a connection-splitting

TCP PEP [17], or a basic router.

Real-world experiments.

To test its robustness, we also evaluated the Sidekick protocol over a real-world environment that resembled the scenario on the train (Figure 3.4). In this setup, a Lenovo ThinkPad laptop, running Ubuntu 22.04.3 with a 4-Core Intel i7 CPU @ 2.60 GHz and 16 GB memory, acted as a client to an AWS instance in the nearest geographical region. The ThinkPad used as an access point (AP) a Lenovo Yoga laptop, running Ubuntu 20.04.6 with a 4-Core Intel i5 CPU @ 1.60 GHz and 4 GB memory, with a 2.4 GHz Wi-Fi hotspot. The AP was connected to the Internet via a JEXtream cellular modem with a 5G data plan. The AP ran the Sidekick proxy binary.

We measured the link properties of each path segment to compare to our emulation parameters. We measured delay and loss using 1000 `ping`s over a 100 second period, and bandwidth using an `iperf3` test. On the near segment between the ThinkPad client and the AP, the min/avg/max/stdev RTT was 1.249/37.194/272.168/54.660 ms at 49.8 Mbit/s bandwidth. We observed that loss increased the further away the AP. In our experiments, the client was located roughly 200 feet away in a different room, with 3.6% loss. The far segment between the AP and the AWS server was 48.546/64.381/92.374/6.806 ms with 0.0% loss at 30.9 Mbit/s. In both environments, the cellular link was the bottleneck link in terms of bandwidth, and the corresponding path segments in emulation had similar minimum RTTs and average loss percentages.

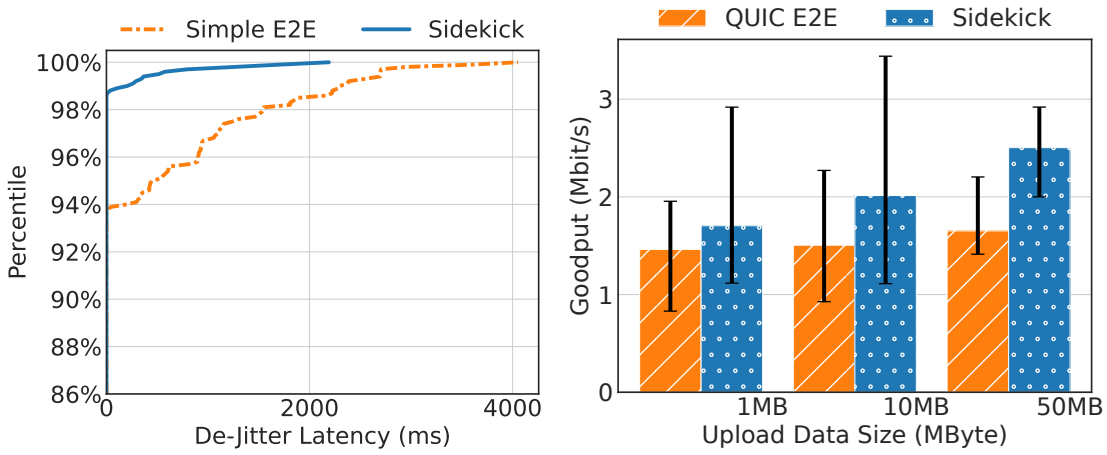
3.7 Real-world results

We discuss the results of our experiments replicating two of our scenarios in the real world, using as context these main differences between emulation and the real-world:

- The RTT is more variable as it depends on interactions in the wireless medium and the shared cellular path.
- Wireless loss can be more variable as nearby 2.4 GHz devices and physical barriers may interfere with the link. Wireless loss also tends to be more clustered in practice.
- The available bandwidth on the shared cellular path is more variable, depending on time of day.

Figure 3.5 shows the results of running the low-latency media and connection-splitting PEP emulation experiments in the real-world. The baseline protocol with a Sidekick is able to reduce the 99th percentile de-jitter latency of an audio stream from 2.3 seconds to 204 ms—about a 91% reduction—and improve the goodput of a 50 MB HTTP/3 upload by about 50%. Although the improvements are more conservative compared to emulation in Figure 3.6a and Figure 3.6b, each case still benefits the base protocol under all circumstances, compared to end-to-end mechanisms alone.

Part of the difference can be attributed to the network setting. When there is no loss on the near path segment, as can occasionally happen in a real Wi-Fi link, we do not expect to see a difference



(a) Low-latency media. CDF of per-packet de-jitter latencies over 10 one-minute trials per protocol.

(b) Path-aware congestion control. Median of 20 trials. Error bars are 1st and 3rd quartiles.

Figure 3.5: Real-world results. Experiments were run in a moderately well-attended office environment over a Friday afternoon. Trials alternate between the baseline and the Sidekick to account for variability in time of day.

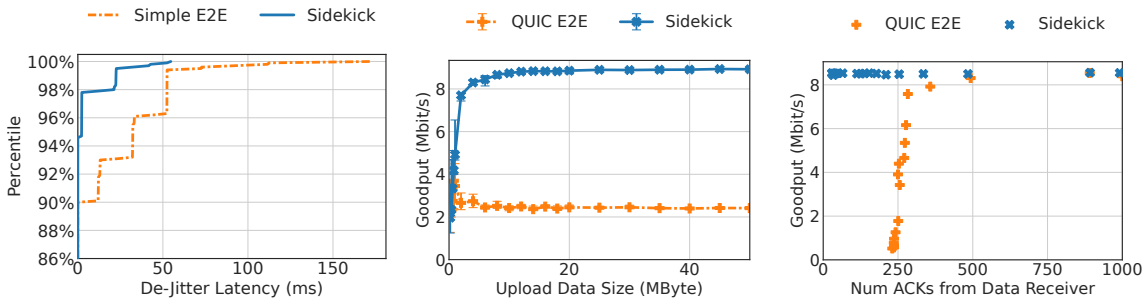
with a Sidekick. When there is more loss on the far path segment, which is variable and depends on the time, we expect the benefit of the Sidekick protocol to be less since this equally affects the performance of the base protocol.

The other part of the difference could be made up by future work that better adapts a Sidekick connection to real-world variability: The client could improve path segment RTT estimation based on when the proxy receives packets, and use this dynamic estimate in the calculation of r used in β and C. The client could also use this estimate to dynamically adjust the quACK interval. Finally, we could analyze theoretically how PACUBIC responds to traffic patterns in the real world.

3.8 Emulation results

3.8.1 Performance of secure transport protocols with Sidekick

We first evaluate Sidekick's main performance goal: In each of the motivating scenarios, we show that using the Sidekick protocol can improve performance compared to the base protocol alone, which would not be able to benefit from existing PEPs. Each scenario has a different metric for success—tail latency, throughput, or number of packets sent by the data receiver (corresponding to energy usage or chance of Wi-Fi collisions)—demonstrating the versatility of the Sidekick protocol.



(a) Scenario #1: Reduced tail latency of de-jitter delay with earlier retransmission in the low-latency media application. 5 minute trials.

(b) Scenario #2: Improved goodput in the connection-splitting PEP emulation. Error bars are the IQR of 20 trials.

(c) Scenario #3: High goodput independent of end-to-end ACK frequency in the ACK reduction scenario. 10 MB upload.

Figure 3.6: Comparing the end-to-end baseline protocol to the same protocol with a Sidekick connection, using the success metrics for the three scenarios described in Table 3.2.

Low-latency media application.

Using the Sidekick protocol can reduce tail latencies in a low-latency media stream, representing fewer drops and better quality of experience. The early retransmissions induced by quACKs reduced the 99th percentile latency of the de-jitter buffer delay from 48.6 ms to 2.2 ms—a 95% reduction (Figure 3.6a). As long as the quACK interval is less than the end-to-end RTT, the connection benefits from the Sidekick protocol.

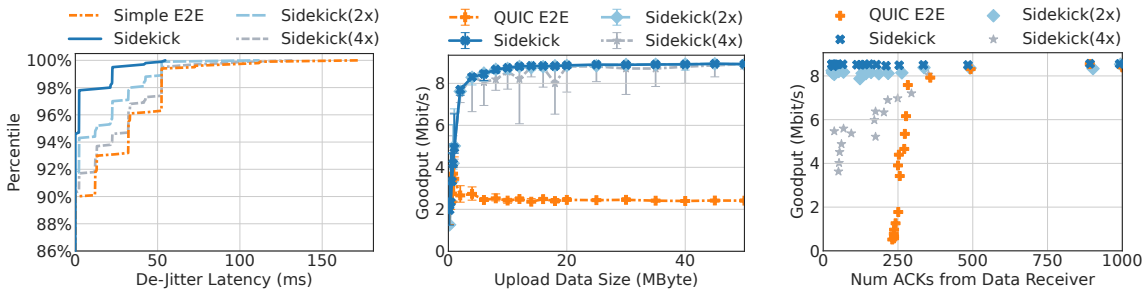
Sidekick is beneficial in this scenario because it enables the client to sooner detect and retransmit lost packets, and the server to sooner play packets from its de-jitter buffer. The end-to-end mechanism takes one additional received packet to notify of the loss and one end-to-end RTT to retransmit and play the packet ($20 + 52 = 72$ms), resulting in three delayed packets (the three "steps" in Figure 3.6a) in most cases. The Sidekick PEP takes up to two additional packets and one near path segment RTT ($20 + 2 = 22$ms or $20 \times 2 + 2 = 42$ms), delaying either one or two packets in comparison. Dropped ACKs and quACKs account for the $< 2\%$ of packets with even greater de-jitter latencies.

Connection-splitting PEP emulation.

Using the Sidekick protocol improves upload speeds when there is a lossy, low-latency link by using quACKs to inform the sender's congestion control. In a scenario with 1% random loss on the link between the proxy and the data sender, the HTTP/3 QUIC client achieves 3.6× the goodput for a 10 MB upload with a Sidekick proxy compared to end-to-end QUIC (Figure 3.6b).

When there is no random loss, the Sidekick protocol does not impact the performance of QUIC. There are no logical changes to the base protocol in this case because all loss is on the bottleneck link on the far path segment, and the CPU overheads of processing quACKs are negligible.

Knowing *where* congestion occurs is an opportunity for creating smarter congestion control. In



(a) Scenario #1. (b) Scenario #2. (c) Scenario #3.

Figure 3.7: Extending Figure 3.6, the Sidekick(Nx) data points show the performance at Nx the quACK interval (sent less frequently) and threshold of the default configurations specified in Table 3.2.

PACUBIC, identifying where the loss occured let the data sender reduce the congestion window proportionally to how many packets were in-flight on each path segment. In Section 3.8.5, we will show that our path-aware congestion control algorithm is as fair as split CUBIC.

ACK reduction scenario.

Using quACKs in lieu of end-to-end ACKs allows the data receiver to significantly reduce its ACK frequency while maintaining high goodput. In our experiment, QUIC with a Sidekick sent 96% fewer packets (mainly ACKs) than end-to-end QUIC before the goodput dropped below 8.5 Mbit/s (Figure 3.6c). The quACK enables the data sender to promptly move the flow-control window forward, as long as the last hop is reliable.

The goodput significantly degrades when reducing the end-to-end ACK frequency without a Sidekick. When end-to-end QUIC reduces the ACK frequency to every 80 ms, the data receiver sends $247/138 = 1.8\times$ the packets at $4.5/8.4 = 0.5\times$ the goodput, worse than QUIC with the Sidekick in both dimensions (Figure 3.6c). Using the Sidekick protocol, the data sender also does not need to change packet pacing to avoid bursts in response to infrequent ACKs, which is why end-to-end QUIC cannot send fewer than ≈ 240 packets.

3.8.2 Configuring the Sidekick connection

In each scenario in Figure 3.7, we also show how with less frequent quACKs ($2\times$ and $4\times$ the interval) and proportionally-adjusted thresholds, the protocol performs worse, or more variably. Table 3.2 shows baseline the quACK interval and threshold we elected for each scenario based on the considerations in Section 3.3.2. Less frequent quACKs means the client reacts later to feedback about the near path segment, and more often has to rely on the end-to-end mechanism. The performance particularly degrades when the quACK interval exceeds the end-to-end RTT. However, even in this case, the base protocol using Sidekick in any capacity performs better than the base protocol alone.

3.8.3 Link overheads from sending quACKs

	Data Sender→		←Proxy		←Data Receiver		
	Pkts	**Bytes**	**Pkts**	**Bytes**	**Pkts**	**Bytes**	**Goodput**
QUIC E2E	1.00×	1.00×	1.00×	1.00×	1.00×	1.00×	1.00×
Strawman 1a	0.96×	1.01×	2.02×	1.56×	1.01×	1.03×	3.33×
Strawman 1b	0.94×	1.00×	2.00×	1.78×	1.00×	1.03×	3.53×
Strawman 1c	1.83×	1.06×	2.01×	1.83×	1.00×	1.03×	3.46×
Power Sum	**0.94×**	**1.00×**	**1.03×**	**1.07×**	**1.00×**	**1.03×**	**3.55×**

(a) Scenario #2: Connection-splitting PEP emulation.

	Data Sender→		←Proxy		←Data Receiver		
	Pkts	**Bytes**	**Pkts**	**Bytes**	**Pkts**	**Bytes**	**Goodput**
QUIC E2E	1.00×	1.00×	1.00×	1.00×	1.00×	1.00×	1.00×
Strawman 1a	0.96×	1.00×	9.94×	4.99×	0.04×	0.08×	1.02×
Strawman 1b	0.96×	1.00×	9.95×	7.13×	0.04×	0.08×	1.02×
Strawman 1c	1.91×	1.05×	9.73×	7.41×	0.04×	0.08×	0.97×
Power Sum	**0.96×**	**1.00×**	**1.09×**	**2.56×**	**0.04×**	**0.08×**	**0.98×**

(b) Scenario #3: ACK reduction.

Table 3.3: Link overheads for a 10 MB upload. The cells represent the multiplier relative to the end-to-end QUIC baseline for each type of quACK. Lower is better for number of packets and bytes sent on a link. Higher goodput is better. The power sum quACK achieves the success metric for each scenario without incurring the link overheads of the strawmen. We did not evaluate the contrived protocol in Scenario #1.

The other cost in terms of using Sidekick protocols is the additional data sent by the proxy to the data sender. Too many additional bytes use up bandwidth, and additional packets use up CPU. Table 3.3 shows the number of packets and bytes sent at each node comparing the strawmen and power sum quACK to no Sidekick connection at all.

Using power sum quACKs increases the packets sent from the proxy to the data sender by 3-9%. These packets either consist mostly of end-to-end ACKs which are sent every packet in `quiche`, or end-to-end ACKs that have been replaced by quACKs in the ACK reduction scenario. We did not evaluate Scenario #1 because it is based on a contrived protocol that lacks many of these features, and the link overheads would not really make sense.

This overhead is representative of the CPU overhead at the client, since quACKs and ACKs take a similar number of cycles to process. In an experiment with Scenario #2 during a period of $\approx$ 90k incoming packets, ACKs took on average 26065 cycles to process while the quACKs took 26369 cycles, 1% more. These cycles come from, i.e., the complex recovery and loss detection algorithms implemented at the end host.

The strawmen have significantly higher link overheads compared to the power sum quACK. The proxy sends up to 10× more packets using Strawman 1a, and also slightly harms the goodput in

the congestion control scenario. The reduced goodput is due to the sender mis-identifying received packets as dropped due to dropped quACKs, since Strawman 1a is not a cumulative representation of all packets received. The proxy achieves higher goodput with Strawman 1b but sends more bytes. Strawman 1c increases the link overheads at both the proxy and the data sender due to larger TCP headers and TCP ACKs. We did not evaluate Strawman 2 due to its impractical decode time.

3.8.4 CPU overheads of encoding at the proxy

	25-Byte Payload		**1468-Byte Payload**	
	Cycles	%	**Cycles**	%
Sniff Packet	22417	97.6	22408	97.5
Table Lookup	247	1.1	251	1.1
Parse ID	23	0.1	22	0.1
Encode ID	74	0.3	69	0.3
Other	213	0.9	225	1.0
Total	*22974*	*100.0*	*22975*	*100.0*

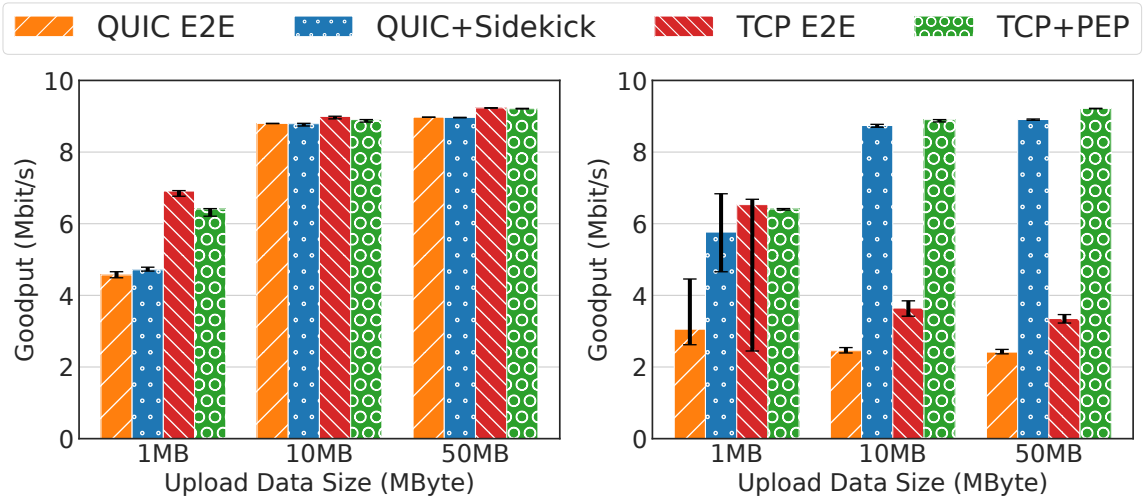
Table 3.4: Breakdown of the CPU cycles spent processing each packet at the proxy. Most cycles are spent on general per-packet overheads as opposed to quACK-specific processing.

The main bottleneck of a Sidekick proxy is the CPU. Table 3.4 shows a breakdown of the number of CPU cycles in each step. The largest overhead was reading the packet contents from the network interface (97.5% of the CPU cycles). Encoding an identifier in a power sum quACK with $t = 10$ used 74 CPU cycles (0.9%). As a calculation of the theoretical maximum on a 2.30 GHz CPU, the proxy would be able to process 31 million packets/second on a single core. The hash table lookup used 251 cycles and parsing the pseudorandom payload as an identifier used 22 cycles.

In practice, we measured the maximum throughput of our Sidekick proxy to be 464k packets/s with 25-byte payloads and 5.5 Gbit/s (458k packets/s) with 1468-byte packet payloads on a single core (assuming 1500-byte MTUs). This experiment used multiple `iperf3` clients to simulate high load until the proxy was unable to keep up with the load. The packet payload size did not seem to affect results.

We find these achieved throughputs acceptable for edge routers such as Wi-Fi APs and base stations. To deploy the Sidekick proxy on core routers, we would need to reduce the overhead of reading packets from the NIC, such as by bypassing the kernel/user-space protection boundary[1] [38,94,129] or using native hardware [14]. We could also scale on multiple cores using symmetric RSS hashing [134].

[1]A kernel-bypass system like Retina [129] can achieve 25 Gbps on 2 cores while processing raw packets with a 1000-cycle callback (Figure 5(a) in [129]). The Sidekick equivalent would be a 500-cycle callback, and assumes all traffic uses Sidekick. Throughput scales almost linearly with the number of cores using symmetric RSS hashing. Thus we don't expect proxy overheads to be an issue with 100 Gbps network speeds even on commodity hardware.



(a) 0% loss.

(b) 1% loss.

Figure 3.8: Goodput for three upload data sizes with 0% and 1% loss on Link 1. IQR of 20 trials. With assistance from a Sidekick proxy or connection-splitting TCP PEP, respectively, both QUIC and TCP at 1% loss match the throughput of the scenario with no loss.

3.8.5 TCP friendliness of path-aware CUBIC

It is easy to improve the throughput of a congestion-controlled base protocol without regard to competing flows; however, we demonstrate that PACUBIC can match the fairness of split TCP CUBIC. We evaluate fairness in Scenario #2 with varying amounts of loss on the near path segment.

QUIC vs. TCP.

We first compare QUIC to TCP without either PEP. As both connections use CUBIC, they exhibit similar congestion control behavior and achieve nearly maximum throughput in the emulated network with no random loss (Figure 3.8a). We attribute the differences to the slightly different retransmission and loss recovery behaviors of QUIC and TCP. The PEPs do not affect the performance.

With even a little loss on the near path segment, both QUIC and TCP dramatically worsen, respectively achieving 28% and 42% of the goodput when there is no loss for a 10 MB upload (Figure 3.8b). In both protocols, CUBIC treats every transmission error as a congestion event, even though no amount of reducing the congestion window affects the error rate. QUIC and TCP perform similarly to each other with proxy assistance and 1% loss on the near path segment.

Sidekick vs. TCP PEP.

Figure 3.9 shows that QUIC with a Sidekick roughly matches—as intended—the behavior of TCP with a PEP-assisted split connection. At higher loss rates, the near path segment becomes the bottleneck

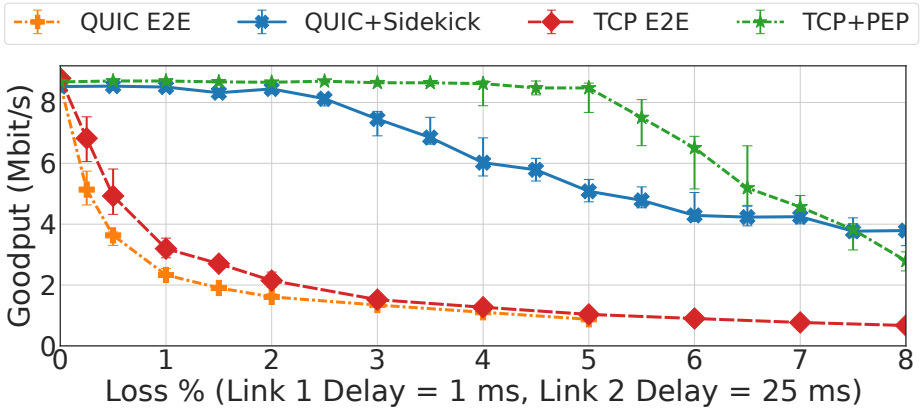


Figure 3.9: Connection-splitting PEP emulation as a function of near-segment loss rate. In this emulation experiment, QUIC+Sidekick (running PACUBIC) performs similarly to TCP+PEP (each connection running CUBIC) and improves goodput compared with end-to-end protocols. The graph shows median goodput of a 10 MByte upload. QuACK interval is 30 ms, threshold is 10. Error bars show IQR of 10 trials.

link even with earlier feedback about loss, causing the throughput of TCP with proxy assistance to drop. QUIC with Sidekick follows a similar pattern because of its path-aware congestion-control scheme (Section 3.3.3). The results indicate that the gains from using the Sidekick protocol do not come at the expense of congestion-control fairness relative to a split TCP connection.

3.9 Limitations

The Sidekick protocol approach, and our experiments, are subject to some limitations, which we describe briefly here.

Multipath scenarios.

We have only considered Sidekick proxies along a single path, and not thought extensively about how quACKs would interact with protocols such as TCPLS [113] that use multiple paths or streams, or even multipath QUIC [34]. To begin thinking about this question, we would have a more complex model of the network: multiple PEPs along a single path, multiple paths each with varying numbers of PEPs, and so on. The proxy can include additional information in the `sidekick-reply` packet to indicate which path the PEP assistance is on, and the sender can infer from the RTT how far along a path each PEP is relative to others. New path-aware algorithms that come from this model could diagnose troublesome paths, or better allocate network traffic in a multipath connection. Existing algorithms could be applied to individual paths as if they were single-path connections.

Even more diverse network scenarios.

Two of the three scenarios we explored consisted of a lossy Wi-Fi link and a high-latency WAN link. Not all scenarios will be favorable to the Sidekick protocol we designed. If the "lossy" section of a network path were on the far path segment from the sender, the sender would not have any more information about the problematic link. To accomodate scenarios like this, Sidekick protocols will need more features. For example, the *proxy* would need some way to receive quACKs from the data *receiver*, as well as a mechanism to buffer and retransmit packets [8, 17].

There are likely other scenarios that could benefit from Sidekick protocols as described, but we did not evaluate them. For example, if we replaced the lossy Wi-Fi link with a modern wireless link that has a fluctuating physical capacity [16, 73, 101], the sender may be able to more quickly adapt and make data available for transmission whenever capacity intermittently becomes available.

Practical deployment.

The implementation of the Sidekick protocol exists as a research system that has been evaluated in emulation and a limited set of real-world scenarios. Since Sidekick protocols require the cooperation of middleboxes and client applications, more work will be needed to standardize the discovery mechanism and wire format of Sidekick messages described in Section 3.3.2, ideally with interest from the IETF. The standards will need to establish several design choices such as how identifiers are computed, how quACKs are transmitted, and the exact mechanisms for security and backwards compatibility. We may also want to standardize sender behavior for specific base protocols, though this could be opaque except to the sender.

The deployment of Sidekick protocols can be gradual and backwards-compatible with parties that are either unaware of or do not want to participate in Sidekick protocols. To migrate existing client applications, one needs to modify the code to discover a PEP and use information in a quACK to inform the base protocol. To migrate middleboxes, they would need to be modified to listen for `sidekick-request` markers, then accumulate and send quACKs for participating connections.

Deeper analysis of path-aware congestion control.

The correspondence between endpoint-driven PACUBIC and "split CUBIC" is good, and both are better than end-to-end CUBIC in Figure 3.9, but not exact. The appropriateness of the PACUBIC heuristic, and in general the idea of path-aware congestion control, needs to be further explored.

3.10 Summary

We presented *Sidekick protocols*: an alternate approach to PEPs that leaves the underlying protocol opaque and unmodified on the wire. We leveraged the power sum quACK (Chapter 2) to enable

proxies to refer to packets of the underlying connection without the ability to observe cleartext sequence numbers. We augmented a streaming protocol and a production QUIC implementation to make use of information arriving from a proxy on a Sidekick connection, including a path-aware congestion control mechanism called *PACUBIC*. In emulation and a real-world evaluation, using the Sidekick protocol was able to improve the performance—tail latency, throughput, energy usage—of these end-to-end base protocols without modifying the wire format or security properties.

Chapter 4

Packrat Proxies: Secure in-network retransmissions for data receivers

The next chapter describes and evaluates the use of *Packrat proxies* for network-originated retransmissions for end-to-end transport connections in lossy settings. Packrat proxies speak the Sidekick protocol over an adjacent connection, but rather than sending quACKs to a data sender, they decode quACKs from a data receiver and identify encrypted packets that need retransmission. Any per-packet overhead incurred at the proxy must be extremely small, and we leverage the IBLT construction of the quACK from Chapter 2 for its efficient decoding operation. With these tools, endpoints can receive network-originated retransmissions appropriate for their desired application metrics—higher throughput, lower latency, and lower link overheads than end-to-end retransmissions—without an additional layer of encapsulation, tunnel, or link-layer retransmissions. Paired with the Sidekick proxies of Chapter 3, Packrat proxies provide protocol-agnostic in-network assistance in both directions, effectively serving as a secure replacement for a variety of traditional PEPs.

4.1 Introduction

The relationship between Internet transport protocols and lossy wireless networks has long been fraught. Three decades ago, the seminal Snoop work [8] addressed an issue that emerged with the rise of wireless Ethernet (Wi-Fi): TCP's end-to-end reliability works poorly over network segments that regularly experience non-congestive packet loss. This is for two big reasons: it's wasteful to require frequent end-to-end retransmissions over a whole network path to address packet loss on one wireless link, and because most congestion-control schemes interpret packet loss as a sign of congestion and will slow down in response—a mistake when the loss wasn't caused by congestion. From its point of view in 1995, the Snoop paper outlined three plausible solutions:

- **Splitting TCP connections** [6, 7]. A proxy at the wireless base station transparently interposes on TCP connections that cross it, creating two concatenated connections: one from "fixed host" to proxy, and from proxy to the "mobile host." This lets retransmission occur over the lossy segment only, and prevents the fixed host's congestion control from seeing non-congestive wireless losses.
- **Link-level acknowledgment and retransmission** [106]. Wireless NICs add their own encapsulation (with their own sequence numbers) to each packet, reply to incoming packets with link-layer acknowledgments, detect apparent losses from the absence of an ACK, and retransmit lost packets before the end-to-end transport protocol concludes they were lost.
- **The Snoop approach**, where an on-path network function transparently interprets TCP acknowledgments in-flight, detects losses, retransmits packets over the lossy link only, and temporarily hides loss from the other host to avoid a duplicate retransmission.

In the intervening years, the first two approaches were widely deployed. At the boundary between the wired Internet and a lossy wide-area subpath (e.g. a satellite or cellular network), network operators use connection-splitting performance-enhancing proxies (PEPs) to accelerate TCP connections crossing their networks. For lossy *local*-area subpaths, Wi-Fi and cellular networks use link-layer sequence numbers, selective "block" acknowledgments, and link-layer retransmissions [25, 44, 61]. To avoid triggering a transport-layer end-to-end retransmission, some receiving NICs hold back received packets to wait for retransmissions of earlier packets so that hosts receive packets in the original order [102]. Because of the practical success of the first two approaches, Snoop's "network-originated retransmissions" didn't seem to be needed in practice.

Until, we propose, maybe now. Post-TCP transport protocols, especially QUIC [65], encrypt and authenticate their packets. This puts some operators in a bind, because they can no longer "split" these connections. But what about link-layer ACKs and retransmissions? This is a usable strategy on low-latency wireless links when one party (or standards body) controls both sides, and losses can be patched before the end-to-end protocol detects them. But when the lossy subpath has higher latency (as for some satellite or experimental networks), losses can't be patched before the transport protocol detects them, and the head-of-line blocking necessary to put packets back in order at the receiver can be ruinous to real-time applications. In any event, such techniques can't be deployed unilaterally by a network operator who doesn't control the receiver or the encapsulation format.

What else could address this? Traffic sources could deploy congestion-control and loss-detection schemes that are less sensitive to loss and reordering [25], but this is also out of a network operator's control. Yet another approach could involve Sidekick proxies (Chapter 3)—but these provide in-network *acknowledgments* of encrypted transport protocols (for hosts to interpret and perhaps trigger retransmission), not in-network *retransmissions* based on the encrypted traffic of hosts. Given all this, what really happens today, at least some of the time, is that satellite and other nontraditional network operators block QUIC traffic, forcing hosts to fall back to TCP that can be split as before.

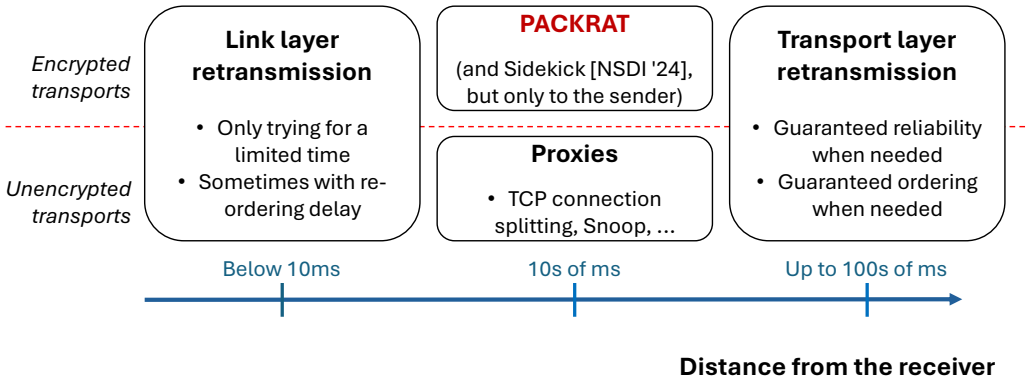


Figure 4.1: The scope of the Packrat proxy compared to other methods for in-network retransmissions.

We think it may be possible to do better by drawing on "Snoop-style" ideas. In this chapter, we describe and evaluate Packrat[1] proxies: a technique that enables network-originated retransmissions for encrypted end-to-end transport connections, yielding performance benefits in lossy settings. We adapt the set reconciliation techniques from Chapter 2 to let the receiver efficiently acknowledge encrypted packets (in an "IBLT quACK") to a network function, without modifying the data sender or the wire format. The data receiver sends quACKs to the Packrat proxy using the Sidekick protocol of Chapter 3. IBLT quACKs are faster to decode, making them appropriate for a deployment where network functions are the ones interpreting acknowledgments from many connections in parallel.

We integrate the Packrat proxy with several applications built on secure transport protocols, and show that it can enable a variety of performance enhancements in settings with a lossy path segment near the data receiver, compared to end-to-end loss recovery schemes:

1. Application 1: Improved throughput for large file downloads using QUIC and HTTP/3.
2. Application 2: Reduced packet tail latency of a low-latency media stream using a simple repetition code.
3. Application 3: Reduced end-to-end retransmissions at the server of a reliable multicast RTP stream.

Figure 4.1 shows the scope of the Packrat proxy compared to other methods of in-network retransmissions over lossy path segments. Unlike link-layer retransmissions, the assistance from a Packrat can be deployed anywhere along the network path and tailored to the performance demands of each base connection. Compared to existing transport-layer solutions, Packrat can be applied to arbitrary transport protocols without ossification.

[1]For "packet rateless retransmission"—and because the Packrat proxy keeps a small cache of recent packets-in-flight for possible retransmission.

Summary of results.

To summarize, we show that it is possible to provide per-connection in-network retransmissions to secure transport protocols along an adjacent connection using Packrat proxies, without involving the data sender nor modifying the wire format of the underlying connection. We start with a brief history of other loss recovery mechanisms in the network (Section 4.2), then describe our contributions:

- A mechanism at the *endpoint* to correct end-to-end reordering signals when there are in-network retransmissions (Section 4.3).
- The use of Packrat proxies with an adjacent Sidekick connection, and optimizations to improve compute scalability based on the co-location of the endpoint and quACK sender (Section 4.4).
- An implementation of the Packrat proxy and integrations with three applications that use secure transport protocols (Section 4.5).
- An emulation study of the performance enhancements, as well as the CPU, memory, and link overheads of using a Packrat proxy. (Sections 4.6 and 4.7).

Finally, we conclude (Section 4.8). With these tools, endpoints can receive in-network retransmissions tailored to their applications without an additional layer of encapsulation, tunnel, or link-layer retransmissions. Sidekick and Packrat together enable protocol-agnostic proxy assistance in both directions, effectively serving as a secure replacement for a variety of traditional PEPs.

4.2 Background: In-network retransmissions

We provide a brief summary of loss recovery mechanisms in the network to motivate the need for lightweight, non-ossifying, in-network retransmissions for secure transport protocols.

End-to-end loss recovery.

Congestion-control schemes, implemented at the endpoints, were traditionally designed to treat all loss as an indicator of congestion [11, 124]. With modern wireless networks and higher bit-error rates, recent developments such as the BBR (Bottleneck Bandwidth and Round-trip propagation time) congestion-control algorithm have de-prioritized packet loss as a congestion signal [22]. This change enables higher throughput but has raised concerns regarding TCP friendliness [108, 130]. In response, the performance of newer versions of BBR has regressed in lossy environments, suggesting a lasting trend in the treatment of loss as a congestion signal regardless of its source [139].

Link-layer loss recovery.

Loss recovery at the link layer [1, 60, 83] can mitigate some of the dramatic reactions of congestion-control algorithms by hiding the existence of random loss from the endpoints. Applications and

transport protocols are oblivious to this functionality, and there is a tradeoff to reliability [71, 72]. More frequent link-layer retransmissions increase the chance of successfully recovering from a loss, but retrying frequently takes more time and may also introduce jitter and reordering between packets [84]. Low-latency applications may prefer to transmit new data instead of retrying, but less frequent link-layer retransmissions also means more loss for other applications. There is no ideal configuration for all the connections that share a link.

Transport-specific mechanisms.

Some transport-layer proxies transparently split a connection into two or more segments and optimize the connection for each segment. In addition to TCP connection splitters [50, 55, 57], selective forwarding units (SFUs) are used for media streams where low-latency is critical, so retransmissions occur closer to the data receiver [3, 132]. As mentioned before, these types of PEPs cause protocol ossification. By making reliability guarantees to the endpoint, connection-splitting PEPs also fate share with the connection.

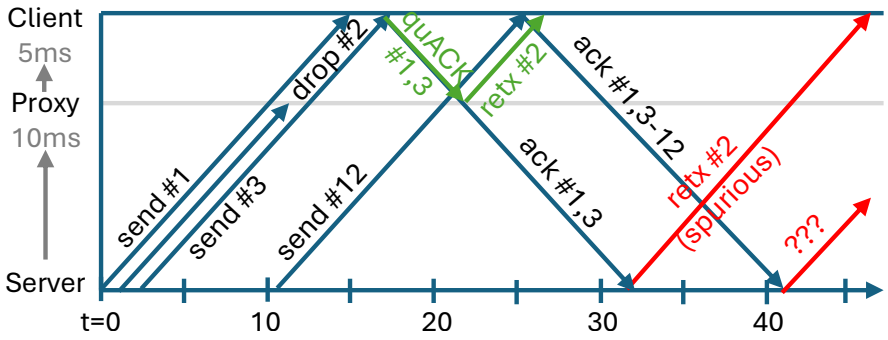
Snoop preserves end-to-end semantics by transparently snooping on TCP acknowledgments to generate retransmissions [8]. However, Snoop modifies transport headers on the wire to prevent spurious retransmissions, which ossifies TCP. Packrat proxies adapt some ideas about in-network retransmissions from Snoop but use mechanisms at the endpoint to preserve end-to-end semantics.

Protocol-agnostic transport assistance.

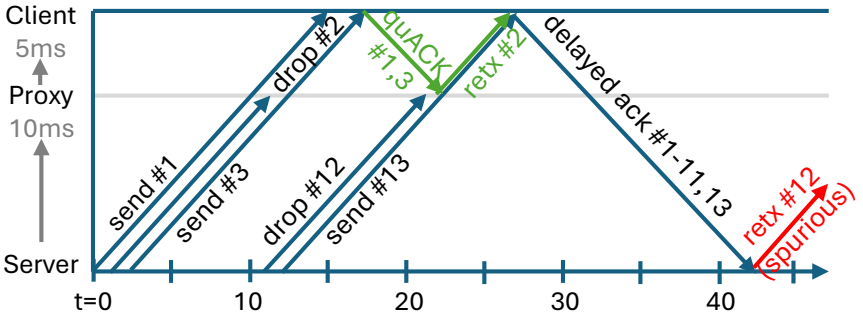
Forward error correction can be used in lossy environments, but this introduces significant overheads as entire packet contents (and not just sequence numbers) must be encoded during transmission [79]. FEC also requires participation from both endpoints.

Mechanisms that encapsulate packets offer potential for reliability without decrypting packet contents, but their primary focus tends to be security. VPNs introduce a costly encryption layer. MASQUE tunnels protocols over HTTP/3, and if used for local retransmissions has the potential for complex nested loss recovery and congestion control interactions [77, 117, 118]. It is intended to function per-application, and the best known implementation from Apple functions like a VPN [4]. These proxies may also not be on-path, which harms performance.

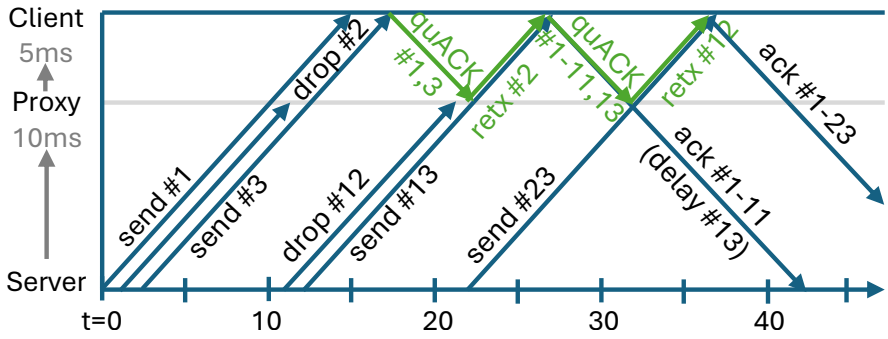
The Sidekick proxy provides protocol-agnostic assistance by sending "quACKs" of encrypted packets to an endpoint, but cannot retransmit packets itself (Chapter 3). This limits its usefulness to when the loss is near the data *sender*. However, client traffic (which is near the lossy last-mile) is typically heavier in the downlink direction. In this chapter, we similarly leverage set reconciliation in quACKs for referring to encrypted packets, but to send network-originated retransmissions.



(a) Unmodified ACK.



(b) Naïve delayed ACK by 10 ms.



(c) ACK with Packrat reorder delay of 10 ms.

Figure 4.2: The reordering problem where in-network retransmissions with millisecond RTTs cause the server to send spurious retransmissions. The end-to-end ACKs constructed with a Packrat reorder delay prevent this problem.

4.3 The reordering problem and spurious retransmissions

Most acknowledgment schemes allow for some reordering but not at the level incurred by in-network retransmissions. For example, TCP fast retransmit requires three duplicate ACKs [11]. The QUIC RFC specifies a threshold of three out-of-order packets before detecting loss [63]. The RACK-TLP algorithm allows for a time-based "reordering window", but still penalizes excessive reordering [25]. Compared to, e.g., Wi-Fi retransmits, the reordering from in-network transmissions is directly related to the RTT to the proxy on the order of tens of milliseconds. We call this the *reordering problem*.

Consider the example in Figure 4.2a using selective end-to-end ACKs, where a gap is considered a missing packet: The server sends 1250-byte packets at 10 Mbit/s, which is 1 packet every ms. The client drops packet #2 but receives #1, #3, #4, etc. The RTT between the client and proxy is 10 ms, and by the time the client receives the in-network retransmission of #2 (from quACKing #1 and #3), it has already ACKed 10 more packets with higher sequence numbers up to #12. The server unnecessarily retransmits #2 and treats the loss as a congestion signal. Spurious retransmissions take up valuable bandwidth and often cause strange behavior in congestion control algorithms [84].

4.3.1 How to solve this problem for TCP

The Snoop protocol faces the same challenges with needing to modify end-to-end signals when sending in-network retransmissions for TCP [8]. The solution here involves manipulating TCP sequence numbers and withholding duplicate TCP ACKs from the server if the proxy has the packet in the cache. This is exactly the kind of ossifying proxy behavior we wish to avoid. In addition, without plaintext sequence numbers, the Packrat proxy is unable to understand which encrypted packets that an end-to-end ACK is referring to, and can only forward packets as intended.

4.3.2 Naïve solutions for secure transport protocols

Another option may be for the client to *delay* sending an ACK until it has allowed one proxy RTT for the proxy to retransmit the packet first [15]. In Figure 4.2b, the client may wait the proxy RTT of 10 ms from detecting the gap in packet #2 before sending an end-to-end ACK, at which point it would have received the retransmission for #2 and become able to ACK the full range from #1-13. This seems promising at first, but in the time the client was waiting, another loss occurred of packet #12, causing the new ACK to have the same problem as an unmodified end-to-end ACK.

Various other solutions exist, such as changing a server configuration for the reordering delay threshold, but it is also undesirable to require certain features in all data senders as they should not have to participate in the Sidekick protocol between the proxy and the data receiver.

4.3.3 Packrat solution with a reorder delay

Our solution is for the end-to-end ACKs constructed by the client to signal *jitter*, defined as variation in packet interarrival times, instead of reordering. This enables the data receiver to process packets as soon as they are received and reduces spurious retransmissions from the data sender, without involving the data sender. The precise manifestation depends on the acknowledgment scheme.

For example, QUIC ACKs consist of a cumulative sequence number, followed by SACK ranges. Using the Packrat solution, the QUIC ACK instead includes the cumulative sequence number, but then only the ranges for packets that have been received for at least the time it takes for the proxy to retransmit. This *reorder delay* includes the RTT to the proxy as well as any delay in sending the quACK. In Figure 4.2c, 10 ms after detecting the gap in #2, the ACK consists of packets #1-2 and any cumulative sequence numbers beyond #2 (up to #11). We assume that most loss is near the client, and that the RTT to the proxy is less than the end-to-end loss detection timeout.

Different protocols use different acknowledgment schemes with similar flavors. The modification for negative acknowledgments of missing packets is to delay sending the NACK for the reorder delay of at least one proxy RTT before deciding whether the NACK is still needed. TCP relies on duplicate ACKs, which need a different modification. Abstractly, packets are held and ordered in a jitter buffer for a certain reorder delay before they can be referenced in the client's underlying acknowledgments.

4.4 Design

The Sidekick protocol in Chapter 3 is spoken between a client and an on-path Sidekick proxy, but in settings using a Packrat proxy, the client is a data *receiver*. In this direction, once the client and proxy have established an adjacent connection for a specific base connection identified by a UDP 4-tuple (Section 3.3.1), the proxy is ready to send in-network retransmissions. The client sends acknowledgements of which encrypted packets it has received, and the proxy retransmits missing packets it previously forwarded on that base connection.

We previously described the *reordering problem* with in-network retransmissions, and a mechanism at the *client* to address the issue (Section 4.3). Now we describe the modified Sidekick protocol messages exchanged between the client and Packrat proxy to establish the adjacent connection and generate in-network retransmissions. We also discuss the behavior of each party in the underlying and adjacent connections. Each message is transmitted in an unreliable UDP datagram.

4.4.1 Sidekick protocol messages with a Packrat proxy

Packrats are distinguished from other types of proxies such as VPNs and transparent PEPs in that the proxy is on the path and the client is knowingly receiving in-network retransmissions. As described in Section 3.3.1, in order to discover proxies on the path, the client sends a magic

```
Init
epoch: u32;
base_conn: [u8; 12];
quack_ty: u8;
num_symbols: u8;
id_offset: u16;
mem_bytes: u32;
```

```
quACK
count: u32;
last_element: u32;
code: Vec<Symbol>;
```

(a) Client payloads.

```
InitACK
epoch: u32;
udp_port: u16;
errno: u32;
```

```
Reset
epoch: u32;
errno: u32;
```

```
Retransmit
udp_payload: Vec<u8>;
```

(b) Proxy payloads.

Figure 4.3: Packrat protocol messages to initialize the connection and generate retransmissions.

`sidekick-request` packet along the base connection 4-tuple, after it has established that connection. Proxies respond with their supported Sidekick configurations as well as their support for Packrat. Clients can choose to accept assistance from a proxy by replying with an `Init` and their configuration request. The proxy completes the handshake with an `InitACK`, and is ready to cache packets and retransmit. On receiving an `InitACK`, the client is free to send quACKs.

The client configuration in `Init` includes several parameters (Figure 4.3a). Similar to the message when initializing a Sidekick connection as the data sender, this includes: the type of quACK, the number of symbols, and how to parse the identifier from the payload. In addition, a configuration unique to the data receiver is the requested size of the Packrat cache. The client may dynamically adjust these configurations with new `Init` packets as it learns more about the network path.

4.4.2 Proxy retransmissions enabled by the quACK

The behavior of the proxy is to decode incoming quACKs and retransmit missing packets, and to cache and evict packets from the base connection for retransmission. The state at the proxy for each base connection is (1) a packet cache and (2) a quACK that represents the cumulative encoding of all packets received by the client that are no longer in the cache.

Decode quACKs and retransmit.

The `quACK` definitively states which packets have been *received* by the client. Based on its knowledge of which packets it has *sent* to the client, the proxy evicts any newly received packets from the cache. The proxy then applies a loss detection policy to determine which of its sent packets may have been dropped. The default policy is any "gap" in the ordered sequence of packets that were previously

cached, but packet timeouts could also apply. The proxy immediately retransmits these missing packets, moving them to the top of the cache.

The retransmission is simply a duplicate of the original packet on the wire. However, the proxy can also explicitly indicate that the retransmission comes from the Packrat by encapsulating the UDP payload in a `Retransmit`. This encapsulation could also include the number of attempts the proxy has made to retransmit this particular packet. The packet would be received by the client on the Packrat socket, which can then forward the payload to the base connection.

Cache and evict packets.

The proxy caches packets with the base connection 4-tuple in the order they are forwarded to the client, up to the pre-configured cache size. It evicts packets if a `quACK` indicates they have been received and no longer need to be retransmitted. Whenever a packet is evicted, it is also encoded in the quACK state of the base connection.

If a packet of the base connection arrives but the cache is full, the proxy *optimistically* evicts the oldest packets to make room. When retransmissions are uncommon, this allows the client to quACK less frequently and the proxy to maintain a significantly smaller cache. Unlike connection-splitters, the proxy does not actually make reliability *guarantees*, and the client can always fall back to end-to-end retransmissions after resetting the Packrat connection, as described below.

Reset the Packrat connection.

If the proxy is unable to decode a quACK for any reason, it sends a `Reset` to the client. These reasons include but are not limited to: an insufficient number of symbols to decode the missing packets, a necessary retransmission was optimistically evicted, a packet corruption caused the encoded identifiers to become out of sync, etc. The `Reset` indicates the start of a new epoch in which the client and proxy encode identifiers into their quACKs.

Base connections that use a Packrat proxy must always be able to fall back to end-to-end mechanisms when the Packrat is unable to send in-network retransmissions. This is unlike typical connection-splitters and VPNs that fate share with their endpoints, or explicitly configured proxies. Similar to Snoop [8], the Packrat style of in-network assistance is better suited to mobility and handoffs if the connection ever changes network paths.

4.4.3 Rateless and sparse quACKing at the client

In addition to modifying end-to-end acknowledgments to avoid sending excessive reordering signals to the server, the primary behavior of the client is to `quACK` at a specified interval. This interval is specified in terms of time or in number of packets, e.g., every 10 ms or every 16 packets. This frequency is typically the same as that of the underlying acknowledgment scheme.

However, an on-path proxy handles many tens to hundreds of thousands of concurrent connections at once. Any per-packet overhead incurred by the Sidekick protocol at the proxy must be extremely small. The Packrat proxy already encodes every packet in the base connection and, unlike the Sidekick proxy, also decodes every quACK. Using the power sum quACK, decoding a quACK can be 600× more expensive than encoding a packet (Table 2.5). To mitigate this issue, we leverage the IBLT construction of the quACK because it is faster to decode.

Another way to reduce the impact of decoding at the proxy is simply to send smaller and fewer quACKs. When the quACK sender is at the proxy, such as in the Sidekick protocol, it doesn't have much flexibility in changing how often it quACKs given the base protocol is completely encrypted. But when the quACK sender is co-located with an application endpoint, the endpoint can leverage its knowledge about the base acknowledgment scheme to dynamically adjust the quACK. We describe two optimizations: *rateless quACKs* and *sparse quACKing*.

Rateless quACKs.

The client configures the number of symbols in the quACK for the worst case, but it is wasteful to consistently send hundreds of bytes over the wire for each quACK if loss is highly variable. Unlike set reconciliation in distributed systems, quACKs are transmitted at millisecond RTT timescales, and the number of symbols cannot be dynamically negotiated between the client and proxy.

In the blockchain setting of the Rateless IBLT [136], the two parties negotiate to determine the number of missing items, and to send additional symbols if the current number is not enough to probabilistically decode. This minimizes the number of symbols sent over the wire, but the quACK ontext does not have the time to negotiate over multiple RTTs. How else can we apply the rateless property of the set encoding to quACKs?

In fact, the client can estimate the number of retransmissions it expects from the proxy and send a smaller quACK, while locally encoding enough symbols for the worst case. For example, the client can count the number of gaps in an ACK, or the packets for which it would send a NACK. Both the power sum and IBLT constructions of the quACK in Section 2.2 have the property that a strict prefix of symbols is sufficient to decode a smaller number of missing elements.

Sparse quACKing.

If the client does not expect it needs a retransmission, there is not much point in sending a quACK. In NACK schemes, where the client detects loss and asks for a retransmission, it can choose to quACK only when it would otherwise send a NACK. Note that the client can omit regular quACKs without the cache exploding in size because the proxy optimistically evicts packets, and these are likely to be received or the client would have NACKed.

4.5 Implementation

We now describe our implementation of Packrat with the Sidekick protocol, which includes a `packrat` library used for three client integrations and a Packrat proxy. We also describe the implementations of the applications and baselines we use to evaluate Packrat.

4.5.1 End-to-end applications

We evaluate Packrat in three applications with different performance metrics to explore the versatility of in-network retransmissions: a high-throughput HTTP/3 file download, a low-latency media stream with forward error correction, and a reliable multicast stream whose server has limited capacity to handle end-to-end unicast retransmissions.

HTTP/3 file download.

The HTTP/3 benchmark uses the default client and server in the Picoquic QUIC implementation [58]. Picoquic is an implementation of QUIC in C based on the IETF standard used for experimentation on extensions to QUIC in the IETF, with simplicity and correctness as its goals. Both endpoints use CUBIC for congestion control.

The goodput is measured by dividing the connection time by the requested data size, excluding headers. The connection time is measured as when the client sends the first byte of its request to when it receives all requested bytes. We use 25 MB which is large enough to achieve stable goodput.

Low-latency media with FEC.

We implement an endpoint for streaming low-latency audio data from the server to the client, using a simple repetition code for forward error correction. The server (a) sends a packet every 20 ms (b) containing audio from the last 40 ms (so there's an overlap in each packet). Each audio packet contains 480 bytes of data, representing an audio stream at 96 kbit/s.

The client begins playback with 40 ms in its buffer, stalling if frames arrive too late and fast-forwarding if behind the target buffer size. If there is a gap in the playback buffer, the client sends a NACK with the sequence number of the missing frame. The client retransmits NACKs, up to one per RTT, until it has received the missing frame. On receiving a NACK, the server immediately retransmits.

The one-way latency is measured from the time the data is produced in the real world to when it is encoded, transmitted, and available in the client's buffer. For example, a frame containing 40 ms of data sent over a network path with 150 ms delay has a minimum one-way latency of 190 ms.

Reliable IP multicast stream.

The IP multicast application is similar to the low-latency media application except the server streams data to a multicast IP address. The server does not use forward error correction, and each packet contains 240 bytes of data. Multiple clients subscribe to the multicast IP address, and if a client is missing data, it sends an end-to-end NACK to the multicast server to receive a unicast retransmission. The primary metric is the number of end-to-end retransmissions requested by the clients. That is, we measure the number of packets that the server must (unicast) retransmit, which captures both server and network load.

4.5.2 Reliable tunnel baseline

We additionally aim to compare Packrat to link-layer loss recovery approaches, as discussed in Section 4.2. To mimic these approaches, we implement a reliable tunnel that encapsulates and retransmits packets similar to Wi-Fi. The tunnel sender encapsulates IP datagrams in a header with a plaintext sequence number, and the tunnel receiver replies with a 64-bit Block ACK [60]. The sender retransmits gaps in the acknowledgment up to 7 times, the default limit in Wi-Fi, and takes care to avoid duplicates. We also implement an option for the receiver to order packets before releasing them to the application, behavior that we believe is unspecified in the Wi-Fi standard but can dramatically impact application performance. We refer to these variants as "ordered tunnels" and "unordered tunnels".

4.5.3 Split QUIC connection baseline

We implement a `picoquic` connection splitter that decrypts and re-encrypts packets in two separate QUIC connections from one endpoint to another. We emphasize that these connection-splitters do not actually exist. To be deployed, the proxy would need to be credentialed with access to the underlying sequence numbers, the antithesis of the transport purists who encrypted these headers in the first place [39, 59]. Since TCP connection-splitters *are* commonly deployed [50, 57], our goal is to understand how much QUIC is "losing out" by not splitting its connection, and how closely Packrat can help QUIC emulate the same performance benefits of a split connection without ossification.

4.5.4 Client integrations with Packrat

Clients must knowingly opt in to receiving assistance from a Packrat proxy, and are also responsible for sending quACKs. Regardless of the base protocol, clients share much of the same functionality: establishing the adjacent connection, maintaining a cumulative quACK of received packets, and determining when to quACK based on a set interval. We implement a library for this shared functionality (Listing 4.1) and integrate it into the three clients of our end-to-end applications. The library uses $\approx$700 lines of Rust and includes C bindings.

```
struct Packrat {
    /// Serialize a Packrat message to send
    fn send_init(params: ...) -> SendBuf;
    fn send_quack(); -> SendBuf;
    fn send_quack_rateless(m: usize); -> SendBuf;
    /// Handle the incoming message and return the
    /// payload if it is a `Retransmit` message.
    fn recv_payload(buf: RecvBuf) -> &[u8]
    /// Manage quACK state
    fn is_ready() -> bool;
    fn insert(id: u32) -> bool;
    fn reset();
}
```

Listing 4.1: Pseudocode interface for the `packrat` library. The library consolidates shared functionality at the client for sending quACKs. The client is responsible for reading and writing to the actual UDP socket for the adjacent connection.

The library can also serialize and deserialize messages in the adjacent connection, but the client is responsible for reading and writing to the actual socket. Each application uses a packet loop to interact with a socket for the base connection. We incorporate the Packrat socket into the same loop, which allows us to consider hints from the application for rateless and sparse quACKing (Section 4.4.3). We also incorporate a reorder delay in the base connection (Section 4.3).

4.5.5 Packrat proxy

We implement the Packrat proxy as a network bridge that uses raw sockets to read and write packets between two interfaces. The proxy needs to be on-path and intercept the packets, as opposed to sniffing them, so its `InitACK` and `Reset` messages can be ordered along with the data packets. The proxy inspects each packet for discovery packets, quACKs, and those that match the 4-tuples of the base connections it is helping. It handles each packet as described in Section 4.4.2. We use our `quack` library to encode and decode quACKs (Section 2.3). The proxy uses ≈1000 lines of Rust.

Multicast proxy.

We also implement an extension to the Packrat proxy for IP multicast, or more generally, multiple clients that share a base data stream. The proxy maintains a fixed-size cache for the multicast 4-tuple with optimistic eviction only. For each client, the proxy maintains a quACK and a *virtual cache*. The virtual cache contains (1) a global index in the base cache of the client's first unacknowledged packet and (2) pairs of global indexes that indicate which packet to insert and where to insert it because it was retransmitted to the client. This allows the proxy to maintain state proportional in size only to the number of outstanding retransmissions per client.

	Data Receiver (Client) ↔ Proxy	Proxy ↔ Data Sender (Server)	QuACK Interval	Num Symbols	Cache Capacity	Reorder Delay
HTTP/3	2ms, 50 Mbit/s, 4% loss	30ms, 20 Mbit/s, 0% loss	10ms, 16pkts	160+ rateless	48 kB	30 ms
Media	50ms, 50 Mbit/s, 10% loss	100ms, 20 Mbit/s, 0% loss	On NACK	40+ rateless	20 kB	110 ms
Multicast	10ms, 50 Mbit/s, 10% loss	30ms, 20 Mbit/s, 0% loss	On NACK	40+ rateless	64 kB	30 ms

Table 4.1: Experimental configuration of each benchmark. The network settings represent scenarios with loss near the data receiver where the proxy is located behind a Wi-Fi access point, LEO satellite ground station, and cellular base station, respectively. The number of symbols is configured for the IBLT quACK, which in practice sends only a short prefix of symbols over the wire.

4.6 Evaluation methodology

The primary baseline is just the end-to-end protocol, which is the default for secure transport protocols that cannot receive transport-layer assistance from proxies. We also compare Packrat to link-layer loss recovery modeled by our reliable tunnel, and a "true" split HTTP/3 connection.

Network configuration.

We run emulation experiments in `mininet` to evaluate the Packrat proxy in a controlled environment. We generally use a linear, two-segment network topology. The host nodes correspond to the client, proxy, and server. Each segment has a bridging node to emulate network properties on the link. In the multicast application, we replicate the client and link it to the same bridging node.

We parameterize each path segment in three dimensions: delay, bandwidth, and random loss. We configure the network properties on the bridging nodes' egress interfaces, using `tc-netem` to set delay and random loss, and `tc-htb` to set bandwidth. Additionally, we use `tc-qdisc` to configure the queues to use `red`, emulating congestive loss in our single-flow environment. The links are symmetric.

In the end-to-end setting, the proxy simply runs a Linux transparent bridge. Otherwise, the proxy runs a binary—the Packrat proxy, the connection splitter, or the tunnel—that manually bridges traffic between the two interfaces. In the tunnel proxy case, we add an additional proxy node near the client to transparently decapsulate traffic from the other proxy.

Experimental configuration.

Table 4.1 describes the default experimental configuration for each application. The network settings model scenarios with a lossy path segment near the data receiver. The delay on the lossy link approximates the location of the proxy relative to the data receiver—behind a Wi-Fi access point, low-Earth orbit satellite ground station, or cellular base station. The other link represents a reliable,

but higher-delay path segment in the network core. The bottleneck bandwidth is on this higher-delay link, representing the congested backhaul. Importantly, the random emulated loss is attributed to non-congestive factors such as wireless interference.

The remaining configurations in Table 4.1 are for the Sidekick protocol and the Packrat proxy. We set the quACK interval to be the same as the protocol's underlying acknowledgment scheme. The number of symbols determines how many missing packets can be decoded between each quACK. Though the IBLT quACK needs more symbols because of its non-determinism, choosing a large number of symbols minimally impacts the encoding overhead, which scales logarithmically, and the link overhead, because the sender can transmit only a prefix of symbols with ratelessness. The Packrat reorder delay is a client configuration that must be at least the RTT to the proxy, which the client can learn from the Sidekick protocol. We also select a cache size that is small enough to evaluate optimistic eviction in Section 4.7.4.

Measurement methodology.

We define spurious retransmissions as the number of packets received by the client containing duplicate data frames. To measure link overheads, we record the values of `/sys/class/net/<iface>/statistics/<metric>` for each emulated network interface immediately before and after the benchmark. Cache memory usage is measured by logging cache updates at the Packrat proxy and parsing the time-series logs. We use `rdtsc()` to measure CPU cycles in the proxy. Unless otherwise specified, the HTTP benchmark results are the IQRs of 20 trials, and the media and multicast benchmark results use 3-minute trials.

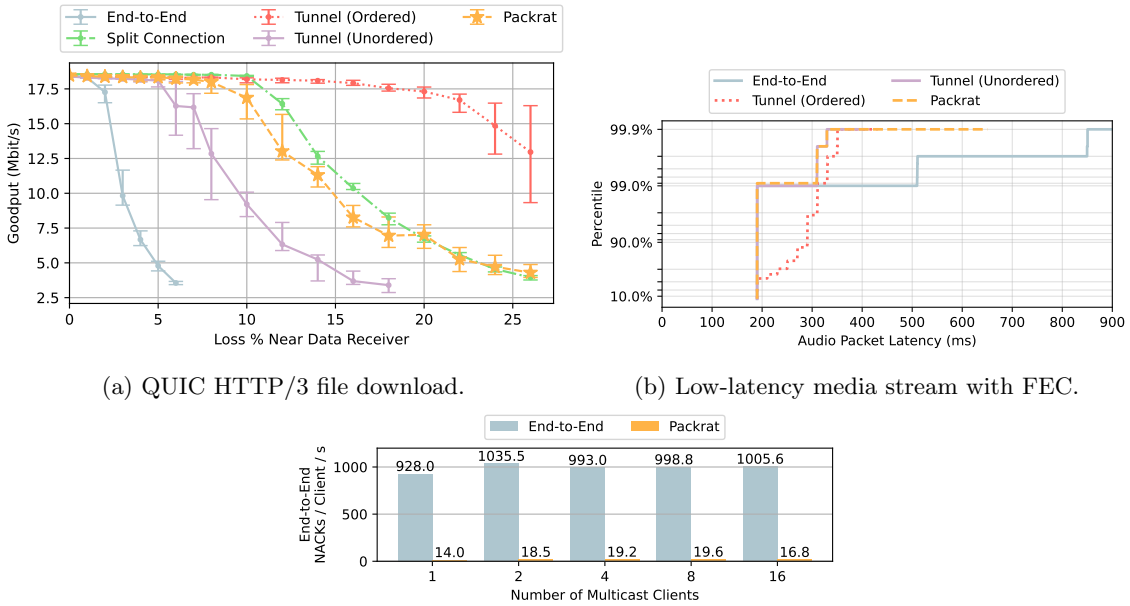
Hardware configuration.

Experiments are performed on an AWS EC2 m4.xlarge instance with 16 GB of RAM and 4-CPU Intel Xeon E5 processor at 2.30 GHz. The instance runs Ubuntu 22.04.2 LTS with Linux 6.80-1024-aws.

4.7 Emulation results

We evaluate our implementation of the Packrat proxy with the Sidekick protocol in an emulation study and answer the following questions:

1. What are the performance advantages of Packrat over the end-to-end, connection-splitting, and tunnel baselines in network settings with a lossy path segment near the data *receiver*?
2. How do the rateless and sparse quACKing optimizations in Section 4.4.3 impact the link overheads of using Packrat?
3. How much memory does the cache use at the Packrat proxy?
4. How much traffic can a CPU-constrained Packrat proxy handle?



(a) QUIC HTTP/3 file download.

(b) Low-latency media stream with FEC.

(c) Reliable IP multicast stream.

Figure 4.4: In-network retransmissions from the Packrat proxy enable applications to achieve better throughput, latency, and link overheads in lossy network settings compared to end-to-end retransmissions. Applications using the tunnel differ in whether they benefit based on the tunnel configuration, which is a disadvantage of link-layer retransmissions.

4.7.1 Performance of secure transport protocols with Packrat

We find that in-network retransmissions via the Packrat proxy enable a variety of performance enhancements—higher throughput, lower latency, and lower link overheads than end-to-end retransmissions—for a variety of secure transport protocols when the data receiver is near a lossy path segment (Figure 4.4). Our link-layer tunnels both harm and help performance depending on their configurations, but they are transparent to the applications that share the link.

HTTP/3 file download.

Packrat improves the throughput of a large HTTP/3 file download 2.7× compared to end-to-end with 4% loss on the near path segment (Figure 4.4a). The CUBIC congestion-control scheme in end-to-end QUIC, in line with other studies on "loss-based" congestion control, achieves poor link utilization in even minimally lossy environments. While the performance of congestion-control schemes differ by implementation, `picoquic` CUBIC has been shown to be one of the more aggressive ones [139].

QUIC with a Packrat proxy achieves similar throughput to QUIC with an explicit connection splitter as loss increases. They both utilize nearly all available bandwidth until ≈10% loss, before gradually deteriorating. In the split connection, this behavior is due to the congestion window on

the near path segment being able to compensate for high loss with a short delay, up to a certain threshold. With Packrat, this is because the reorder delay initially hides loss from the congestion control algorithm at the data sender. As the loss increases, we observe that the proxy fails to decode the quACK more often and resets the connection. Each time there is a reset, the receiver falls back to end-to-end retransmissions, signaling more loss (and congestion) to the data sender. This suggests that the congestion response to loss with Packrat can be both as good and as fair as QUIC with a connection splitter, without fate sharing or decrypting the connection at the proxy.

For the HTTP/3 download in this low-latency, low-loss network setting, the ordered tunnel outperforms QUIC with a Packrat proxy. The unordered tunnel does not perform as well because the retransmissions appear to arrive out-of-order. The end-to-end ACKs subsequently cause the server to send spurious retransmissions (Figure 4.5a). The congestion response to spurious retransmissions is not well-defined in standards, and `picoquic` makes some attempt to revert the congestion window that leads to this result, which is a similar result we observe when using Packrat with no reorder delay. The ordered tunnel does not run into the same issues because its aggressive retransmissions, when ordered, hide most loss the server could interpret as a congestion signal.

Low-latency media with FEC.

Packrat enables fewer and shorter stalls in a low-latency media stream over a high-delay satellite network (Figure 4.4b). In this network setting, the minimum one-way latency of a packet is already 190 ms (40 ms encoding + 150 ms delay). For reference, 150 ms is the normal latency for VoIP, and anything above produces a noticeable drop in quality. Thus network-generated retransmissions are crucial in lossy, high-latency settings.

Both Packrat and the end-to-end baseline benefit from forward error correction, but Packrat enables shorter stalls when a frame still needs to be retransmitted. In this transport protocol, a frame needs to be retransmitted if two packets are lost in a row. Thus at 10% loss, the 99th percentile delay of both mechanisms is the minimum of 190 ms. In the remaining percentile, one more out-of-order packet arrives after 20 ms, draining the last of the buffer and generating a quACK or NACK. The length of the stall is then the 100 or 300 ms RTT with the proxy or server. The application exhibits a similar behavior using the unordered tunnel and the Packrat proxy.

Unlike the HTTP/3 benchmark, this transport protocol gets none of the benefits of forward error correction with the *ordered* tunnel. When a retransmission is required, the tunnel buffers at least 6 more packets until it receives the retransmission and releases them to the client. The buffering causes unnecessary stalls. If the tunnel had just released the packets, the repetition code would have masked instances with a single packet loss.

Reliable IP multicast stream.

Caching packets in the network like a CDN is inherently beneficial because it can reduce the congestion in the core network. This is one of the idealistic draws of IP multicast, but it is rarely used over the Internet in practice because a reliable stream may require an overwhelming number of unicast retransmissions from the server. While CDNs and SFUs have helped with content distribution, these are not options for secure transport protocols without embedding trust in the network.

The Packrat proxy can handle in-network retransmissions for a large number of multicast clients. Each client using the Packrat proxy sends on average 16.8 end-to-end retransmissions/second, a 59× reduction from 1005.6 retransmissions/second/client using end-to-end unicast retransmissions (Figure 4.4c). This reduction scales with the number of clients. In-network retransmissions reduce both the load at the server and congestion in the network.

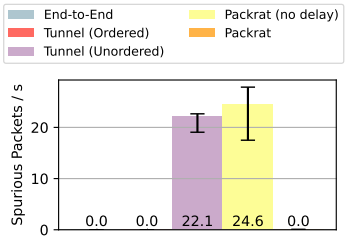
The behaviors of the QUIC and media transport protocols with various tunnels represent the challenges of configuring link-layer retransmissions independently of the applications that share the link. A large download over QUIC performs poorly when there is excessive reordering, while the media transport protocol prefers to receive packets right away. Disabling link-layer retransmissions introduces loss, which can be even more detrimental for performance.

4.7.2 Link overheads using rateless and sparse quACKing

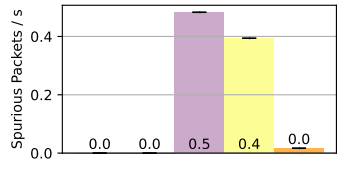
In this section, we analyze how sending quACKs with the optimizations in Section 4.4.3 impacts the link overheads in the network. Figure 4.6 displays the number of packets and bytes sent on each link between the client, proxy, and server in the HTTP and media applications. As a proportion of the volume in bytes sent by both endpoints of the application, the HTTP client makes up 1.3% of the traffic with Packrat and 0.7% without. The media client makes up 8.2% of the traffic with Packrat and 1.0% without. We find that that we can reasonably think of quACKs as control packets in the common case.

Rateless quACKs.

Using the rateless property to send smaller quACKs reduces the number of bytes the client sends by 3.6× (Figure 4.6a) and 1.8× (Figure 4.6c) compared to Packrat without the optimization. This property allows clients to configure the number of symbols less conservatively when initializing the Sidekick connection while being able to dynamically adjust to changing loss conditions, particularly in less controlled non-emulation environments.

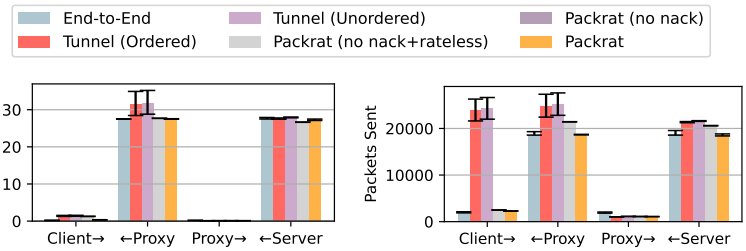


(a) HTTP/3 download.



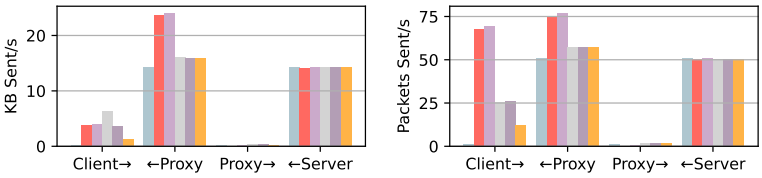
(b) Low-latency media.

Figure 4.5: The unordered tunnel and Packrat without modifying the end-to-end reorder delay cause the client to receive spurious retransmissions. Adding a reorder delay with Packrat significantly reduces them.



(a) HTTP/3 download (bytes).

(b) HTTP/3 download (packets).



(c) Low-latency media (bytes).

(d) Low-latency media (packets).

Figure 4.6: The number of bytes (left) and packets (right) sent between the client, proxy, and server. The statistics are captured directly from the network interface. The link overheads from quACKs using the Packrat proxy are comparable to those of the underlying acknowledgment scheme. The client sends fewer bytes with rateless quACKs, and fewer packets with sparse quACKing. The tunnel aggressively ACKs and retransmits.

Sparse quACKing on NACK.

Sending a quACK only when the client would otherwise send a NACK reduces the number of packets the media client sends 1.9× (Figure 4.6d). Note that without sparse quACKing, we configured the media client to quACK every other packet to balance latency with frequent quACKing. The configurations of the media client that use Packrat send more packets than end-to-end because although it only NACKs when there is a missing *frame*, it quACKs when there is a missing *packet* because the frames that were recovered using FEC are opaque to the proxy.

Base connection improvements.

Even though the HTTP client quACKs at a similar frequency to which it ACKs, the client sends a similar number of packets with and without the Packrat (Figure 4.6b). The proxy in the end-to-end scenario sends nearly twice as many ACKs to the server because the number of ACKs depends on the length of the connection. Packrat improves the throughput so much that the number of net packets sent by the client are similar.

The IP multicast application demonstrates how Packrat can reduce congestion closer to the server by caching retransmissions in the middle of the network (Figure 4.4c). This is especially impactful if

the data is shared by multiple clients.

Note that the tunnels aggressively ACK and retransmit in both directions. The client sends 12-66× additional packets compared to end-to-end and the proxy sends 33-51% more to the client. Even though the block ACKs are small, it can be costly for radios to switch often between sending and receiving modes, and for proxies with per-packet overheads.

4.7.3 CPU overheads of decoding at the proxy

In this experiment, we analyze the number of cycles that the proxy needs to process each packet in the HTTP/3 benchmark, and use the per-packet overheads to extrapolate the maximum processing capacity of a CPU-constrained Packrat proxy. These overheads include both quACK operations and transport-protocol logic for, e.g., packet triaging, loss detection, and cache management.

Each data packet on the base connection takes on average 1002 cycles/packet. The main overhead comes from adding the packet to a ring buffer used as the packet cache.

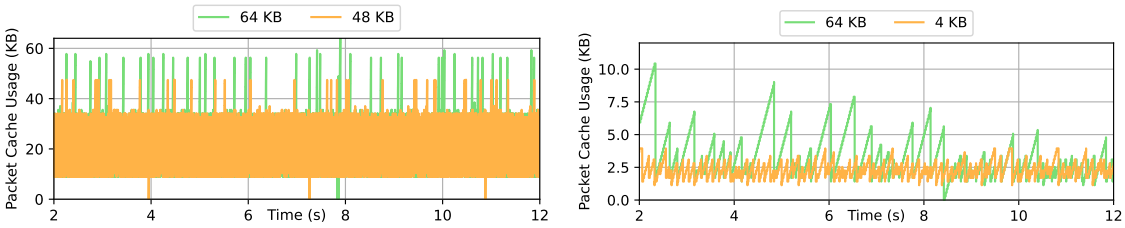
Each quACK on the Packrat connection takes on average 27716 cycles/packet. The proxy determines which packets to retransmit based on the quACK and evicts packets. Loss detection involves encoding a delta of packets since the last quACK into the existing state, as well as decoding the received quACK. Encoding uses a total of $7156/27716 = 25.8\%$ of the cycles and decoding $3560/27716 = 12.8\%$. With a total of 38.6% of the processing time spent on IBLT operations, this implies that both encoding and decoding are valuable to optimize.

In this application, there is approximately one quACK for every 16.3 data packets. If each data packet uses a full MTU of 1500 bytes, this means a single core of a 2.4 GHz CPU on a proxy would theoretically be able to handle 10.7 Gbit/s[2]. However, this estimate does not account for the substantial CPU overhead involved in reading and writing to the NIC—these can be significantly reduced using kernel bypass frameworks like DPDK or with specialized networking hardware.

4.7.4 Memory overheads of the proxy cache

We find that the Packrat proxy needs no more memory than similar TCP PEPs, and that the optimistic eviction policy is an effective fallback for when the cache exceeds its memory limits. Like any in-network retransmission mechanism, the most significant memory usage at the proxy comes from the packet cache. Figure 4.7 shows the number of bytes in the packet contents of the cache over a 10-second period of each application in our chosen network settings. The maximum cache size at the Packrat proxy is 64kB per connection, similar to a typical TCP window size.

[2] 2.4 Gcycles/s $\times$ (16.3 data packets $/(27716 \cdot 1 + 1002 \cdot 16.3)$ cycles) $\times$ (1500 bytes/data packet) $\times$ (8 bits/byte) = 10.7 Gbit/s.



(a) HTTP/3 file download.

(b) Low-latency media with FEC.

Figure 4.7: The number of bytes in the cache over a 10-second period with both bounded (48 KB or 4 KB) and unbounded (64 KB) cache sizes. The proxy uses optimistic eviction if an incoming packet causes the cache to exceed its capacity.

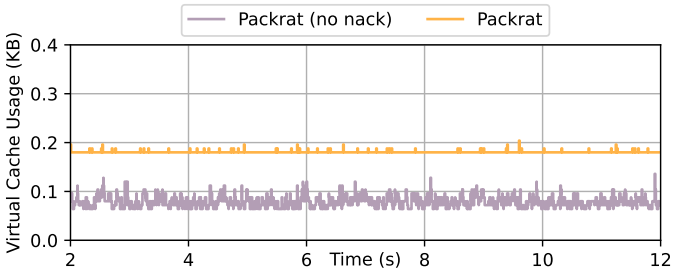


Figure 4.8: The sum of the virtual cache sizes of all 16 clients in the multicast application. Each client uses ≈12 bytes each for a single insertion pair. The variations are due to individuals having slightly more or less than one insertion pair. Without sparse quACKing ("no nack"), clients more often have empty virtual buffers. The base connection's packet cache (not pictured) is always 64 KB.

HTTP/3 file download.

The cache grows quickly because each data packet is large, but evicts packets every 10 ms when it receives a quACK (Figure 4.7a). The minimum cache size is ≈10 kB, the size of the outstanding packets between the client and proxy. A spike occurs when a quACK is dropped, at which point the proxy optimistically evicts. The proxy resets the connection if there is an error, which is more common with a smaller cache, and drops the cache size to 0.

Low-latency media with FEC.

The media application relies on optimistic eviction to keep the cache small since the client sparsely quACKs (Figure 4.7b). The line plateaus at the maximum cache size because the proxy assumes these packets have been received. The cache size only needs to fit 1-2 BDPs of packets between the client and proxy.

Reliable IP multicast stream.

The multicast application has a fixed packet cache size of 64 kB, and we analyze the memory overheads of the per-client virtual caches (Figure 4.8). In particular, we calculate the overhead of each virtual cache as 4 bytes for the first global index and then 8 bytes for each insertion pair (Section 4.5.5). When there are 16 clients, the proxy uses on average 0.178 KB in the virtual caches for a very low overhead of 12 bytes/client. That is exactly one insertion pair, and in general the overheads of the virtual caches are proportional to the number of outstanding retransmissions.

4.8 Summary

In this chapter, we described and evaluated the Packrat proxy for sending network-originated retransmissions when the end-to-end transport is encrypted. We used set-reconciliation techniques based on the Rateless IBLT to let the receiver efficiently acknowledge encrypted packets, yielding performance benefits in lossy settings without an additional layer of encapsulation, tunnel, or link-layer retransmissions.

Chapter 5

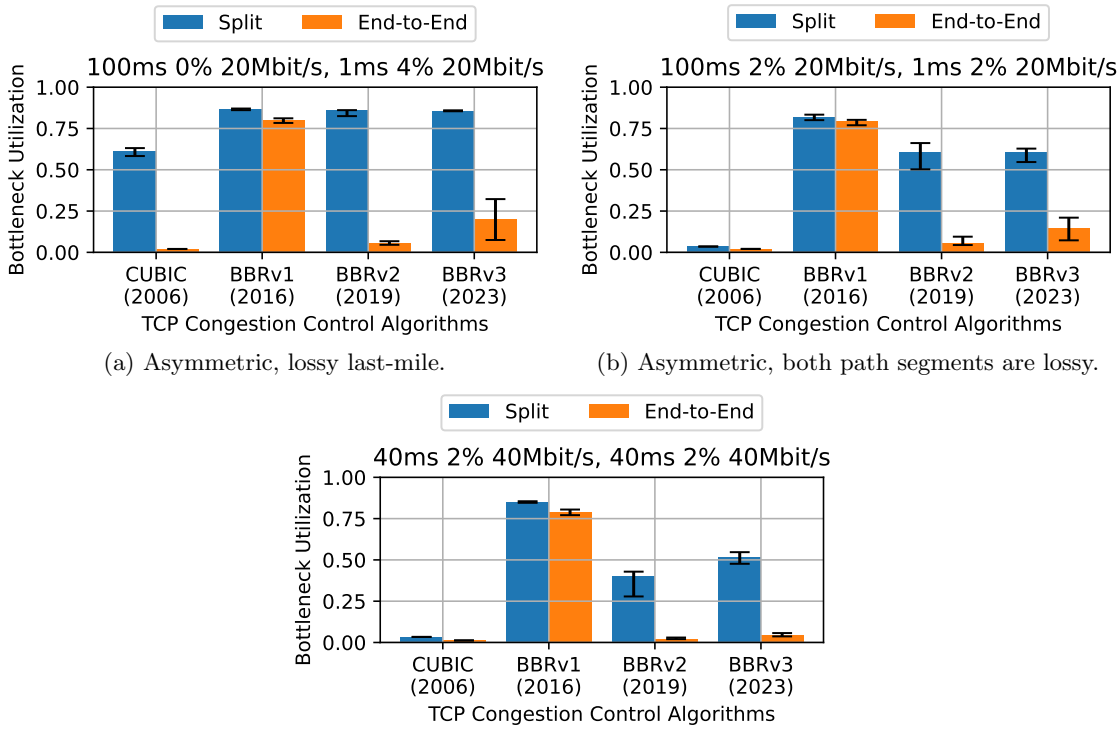
The future of connection-splitting with BBR and QUIC

Much of this dissertation is based off the premise that traditional connection-splitting PEPs still help endpoints achieve performance benefits that we'd like to emulate for secure transport protocols (see PACUBIC in Section 3.4 and the split QUIC connection baseline in Section 4.5). While this may have been true when PEPs were first deployed in the 1990s, two recent developments first announced in the late 2010s have cast doubts on their relevance: the BBR (Bottleneck Bandwidth and Round-trip propagation time) congestion-control algorithm, which de-emphasizes loss as a congestion signal, and the QUIC transport protocol, which prevents transparent connection-splitting yet empirically matches or exceeds TCP's performance in wide deployment, using the same congestion control.

In light of this, are PEPs obsolete? This chapter presents a range of emulation measurements indicating: "probably not." While BBR's original 2016 version didn't benefit markedly from connection-splitting, more recent versions of BBR *do* and, in some cases, even *more* so than earlier "loss-based" congestion-control algorithms. We also find that QUIC implementations of the "same" congestion-control algorithms vary dramatically and further differ from those of Linux TCP—frustrating head-to-head comparisons. Notwithstanding their controversial nature, our results suggest that PEPs remain relevant to Internet performance for the foreseeable future. We hope these findings motivate the development of solutions such as the ones in Chapters 2 to 4 that balance the performance benefits of traditional PEPs with the freedom to evolve transport over time.

5.1 Introduction

A stylized history of connection-splitting over the Internet could go roughly as follows: In the 1970s, TCP was designed as a host-to-host transport for flow-controlled reliable byte streams. In the late



(a) Asymmetric, lossy last-mile.

(b) Asymmetric, both path segments are lossy.

(c) Symmetric, both path segments are lossy.

Figure 5.1: Three classes of network settings in which the throughput of a long-lived data transfer using BBRv3 benefits significantly from connection-splitting. While the throughput of TCP using BBRv1 may not have benefitted from a PEP, as the BBR algorithm has evolved over time, it has also made connection-splitting pertinent again. In two of these classes, split BBRv3 even achieves high bottleneck link rate utilization where split CUBIC does not. IQR of $n = 20$.

1980s, TCP implementations added congestion control [67] to respond to network-path limitations and contention, with indications of packet loss as the primary congestion signals. As Internet use exploded in the 1990s, it became apparent that contemporary TCP implementations performed poorly over long paths with packet loss. By the late 1990s, many network operators deployed commercial "TCP accelerators," also known as "performance-enhancing proxies" [50, 57], to improve customers' TCP throughput. These proxies transparently split a TCP connection in two, providing an intermediate point of acknowledgment and retransmission and transforming an end-to-end TCP connection into two independent connections.

PEPs were marketed as improving users' throughput, especially where a PEP splits a high-delay, lossy path into two segments, each with less delay or less loss. Past surveys found connection-splitting PEPs deployed on large numbers of wireless and satellite networks [50, 57]. But for almost as long as these PEPs have existed, purists have criticized their alleged benefits as exaggerated or unfair, sensitive to the details of an endpoint's congestion-control scheme, contrary to end-to-end

arguments [115], and likely to lead to ossificiation [41, 103].

Two more-recent developments have put PEPs on the back foot. The QUIC transport protocol, standardized in 2021 [65], is encrypted end-to-end and, unlike TCP, its connections can't be split by a transparent proxy—but its wide deployment by major traffic sources (particularly Google) in the 2010s didn't produce a measured reduction in performance, compared with TCP connections that do benefit from PEPs in the field [81]. If PEPs had really been aiding TCP connections all this time, QUIC could have been expected to experience lower sustained throughput over such paths (compared with TCP connections between the same endpoints, using the same congestion-control schemes).

Furthermore, the BBR congestion-control scheme [22], first released in 2016, now controls a large percentage of Internet traffic (both TCP and QUIC) [131] and puts less emphasis on packet loss as a congestion signal than older schemes in wide deployment, such as CUBIC. Because much of PEPs' benefit comes from splitting up long, lossy paths for the benefit of "loss-based" schemes, it has been suggested that BBR has led to a sharp decline in the use of PEPs [70].

Do BBR and QUIC make connection-splitting obsolete? In this chapter, we present findings that suggest that neither of these developments should be regarded as reliable; Connection-splitting may yet play a performance-enhancing role on the Internet, despite its controversial nature.

We conduct a measurement study in emulation to investigate what kinds of congestion control schemes, network scenarios, and transport protocols may still experience increased sustained throughput from connection-splitting. We aid our analysis with a *split throughput heuristic* for predicting the throughput of a split connection, based on the measured end-to-end throughput of each segment on the split path. We aim to (1) explore a wide parameter space of network settings and congestion control schemes and (2) reason about encrypted transport protocols (QUIC) that we cannot directly measure.

We present three major findings:

Finding 1: Splitting has become significantly more beneficial to TCP BBR since it was initially released in 2016.

We confirm that BBR's initial "v1" release does not benefit markedly from splitting, unlike earlier congestion-control schemes. But as the BBR algorithm has evolved into BBRv2 in 2019 and now BBRv3 in 2023, it has also evolved to behave more conventionally in the sense that it benefits from being split—just like more traditional, loss-based congestion control schemes such as CUBIC (Figure 5.1).

The regression in split throughput follows directly from the regression in end-to-end throughput, via the heuristic. While BBRv1 nearly always achieves full utilization, BBRv3 sees lower utilization in lossy settings, leaving more to gain from splitting. The end-to-end regression is because BBRv3 has increased its loss sensitivity to coexist fairly with "loss-based" schemes such as Reno and CUBIC [20, 130, 142]. Thus, we find that BBR and its split behavior continue to evolve, and that the

model-based BBR algorithm has *not* made the performance enhancements of connection-splitting obsolete.

Finding 2: There exist classes of network paths where TCP BBR would significantly benefit from splitting but TCP CUBIC would not.

Splitting not only benefits BBRv3 in all the same scenarios as CUBIC (in terms of long-lived throughput), it also benefits BBRv3 in *new* scenarios. In particular, there exist classes of network settings where the CCA achieves a very low bottleneck link rate utilization *without* a PEP, but a high bottleneck link rate utilization *with* a PEP.

More specifically, in addition to edge deployments with a lossy last-mile (Figure 5.1a), BBRv3 can also benefit from splitting in scenarios where there is loss on both path segments (Figure 5.1b), and when the connection-splitter is located farther from the edge (Figure 5.1c). This suggests that in these network classes, simple split BBRv3 can replace the proprietary protocols that have traditionally been used to address middle-mile loss, particularly in satellite and wireless ad-hoc networks [13, 50, 105]. We should continue to evaluate the behavior of split CCAs in new types of networks, especially in space.

Finding 3: Implementations of the "same" congestion-control schemes in QUIC vary significantly, and further differ those of Linux TCP—frustrating attempts to directly compare performance between QUIC and TCP.

In an initial study of three QUIC implementations, we find that the end-to-end behaviors of the same congestion control schemes vary significantly by implementation. The implementations vary in both baseline performance and sensitivity to loss and delay. By applying the split throughput heuristic, we find that some CUBIC implementations can actually benefit in the same classes of network paths as TCP BBRv3. We also find that BBR is challenging to implement, with various BBRv3 implementations exhibiting non-uniform end-to-end behavior and thus no clear trend in split behavior. We argue that the benefits of in-network assistance should be considered along with not just a CCA, but their specific implementations.

Our initial evaluation suggests that the performance improvements of connection-splitting remain relevant in the context of today's model-based congestion control algorithms, encrypted transport protocols, and satellite and wireless ad-hoc network settings. As congestion-control schemes continue to evolve, we urge researchers to refer to them by algorithm/implementation/version, not just "BBR" or even "QUIC BBRv1". We urge the community to create a performance test suite for an implementation to be able to claim that it conforms to a particular congestion-control standard. Finally, we hope that this work motivates the community to pursue protocol-agnostic PEPs that achieve the performance benefits of in-network assistance without the ossification downsides.

5.2 Background: BBR and QUIC

In this section, we provide additional context on some concepts that we refer to throughout the text.

Performance-enhancing proxies.

The idea of in-network assistance has long brought pain to the hearts of Internet researchers, protocol designers, and network operators. In particular, the word "middleboxes" is often associated with NATs, firewalls, and other policy enforcers that modify or inspect data in ways that break end-to-end behavior. A substantial percentage of Internet paths are affected by feature or protocol-breaking policies of middleboxes [41].

In contrast, we refer specifically to the type of assistance provided by performance-enhancing "proxies", or PEPs, that transparently split a TCP connection in two [50]. While PEPs still interfere with end-to-end transport mechanisms, their goal is to enhance the user experience by maximizing performance, as opposed to enforce security or routing policies.

Connection-splitting PEPs can enhance performance in several ways. By providing an intermediate point of acknowledgment and retransmission, PEPs reduce the length of the feedback loop for signals of loss and congestion. The PEP also allows each side of the split connection to better optimize congestion control and flow control for network conditions local to the path segment. We can expect PEPs to impact a variety of performance metrics, including startup time, packet jitter, retransmission overheads, and more. In this chapter, we focus only on the impact of connection-splitting on the *sustained throughput* of long-lived data transfers.

Congestion control.

Congestion control was originally deployed in the 1980s to manage the explosive growth of the Internet [67]. Foundational algorithms such as slow start and fast retransmit eventually became part of the Tahoe congestion control scheme. Variants such as NewReno and CUBIC emerged over time to improve performance with fairness [52]. CUBIC is the default congestion control module for TCP in the Linux kernel, and widely deployed. It is the primary "loss-based" congestion control scheme that we evaluate.

The word "TCP" is sometimes used synonomously with the congestion control algorithms that it implements. In reality, the two are distinct, though deeply intertwined. For example, the QUIC transport protocol implements many of the same algorithms as TCP to manage network traffic, even though it runs over UDP [65]. We refer to "TCP" and "QUIC" as the transport protocols, "Linux TCP" as the implementation of TCP in the Linux kernel, and congestion control schemes as the particular manifestations of a congestion control algorithm in a transport protocol implementation.

BBR.

Today, the focus on congestion control has shifted towards BBR. BBR is sometimes referred to as a "model-based" congestion control scheme because it relies on a fundamentally different approach of modeling network bandwidth and round-trip time. It does not use packet loss as the primary signal for congestion. Google initially presented BBR in 2016 as addressing the problems of CUBIC in high-speed networks [21].

Over time, BBR encountered controversy for its unfairness towards loss-based schemes [18,109,130]. In response, "v1" of BBR has now evolved into BBRv2 in 2019 and BBRv3 in 2023, driven by efforts at Google. Today, Google recommends that BBRv1 and BBRv2 be deprecated in favor of BBRv3 [20].

BBR is also widely deployed [62, 131]. Google has contributed BBR implementations to the Linux kernel, and Amazon, Akamai, Meta, and Cloudflare CDNs all use BBRv1 for TCP. At Google, BBRv3 is used in all google.com and YouTube public Internet traffic with TCP, and is currently being A/B tested against BBRv1 in Google QUIC traffic [20]. It is estimated that 40% of traffic used BBR in 2019 [98].

Since 2017, there have been ongoing efforts to standardize BBR in the IETF [23]. However, progress has been slow. The Internet draft goes years or months at a time without an update at the mercy of Google contributors. While companies have been quick to adopt BBR in Linux TCP, adoption in QUIC has been slower. Anecdotally, some have resorted to reverse engineering the Linux implementation, and some such as Cloudflare, Meta, and Cisco are actively experimenting with variants of BBRv2+ in their QUIC stacks [62].

QUIC.

The QUIC transport protocol was originally developed by Google in 2012 to improve performance for HTTPS traffic and to enable the rapid evolution of transport mechanisms. QUIC is implemented over UDP, and moves congestion control development from kernel space to user space.

As of 2021, QUIC is now a standards-track RFC [65] with widespread deployment and interoperability testing [120]. It is estimated that 8.4% of global websites support QUIC [128], many of them major traffic sources such as Cloudflare. Still, many high-speed flows remain on TCP due to better TCP hardware capabilities and "per-origin" caching, which complicates multi-CDN deployments that mix protocols.

Evaluations of QUIC at the Internet scale generally show improved performance compared to TCP, except in satellite networks. Features like zero-RTT connection establishment and stream multiplexing enable measured improvements in search latency, rebuffer rate, and other video playback metrics at Google [81]. Satellite network operators, on the other hand, experience strife when it comes to QUIC. Several studies have measured the negative impact of encryption on performance in split satellite environments [13, 74, 80, 89].

5.3 The Split Throughput Heuristic

When initially considering which network scenarios to evaluate in our emulation study, we discovered it to be non-trivial to pick scenarios that would lead to a general understanding of the throughput of a congestion control scheme in a split setting. It is challenging to select which network settings and combinations of path segments to evaluate, as the parameter space for combinations of path segments can be quite large. Also, not all CCA implementations can be trivially split.

For this chapter, we refer to *throughput* as the throughput of a long-lived data transfer in an emulated network, using a single flow. We do not evaluate other metrics such as startup time, retransmission overheads, or short-lived data transfers, which have been of interest to PEPs and could also be impacted by connection-splitting. When discussing throughput, we refer to *end-to-end throughput* as the throughput of an end-to-end connection, and *split throughput* as the throughput of the same connection when there is a connection-splitting PEP.

We realized that while split throughput is not necessarily well understood for any combination of CCA, transport protocol, and network setting, the end-to-end throughput often *is*. We make the following insight based on an ideal setting:

> **Split Throughput Heuristic:** We can estimate the long-lived "split throughput" of a connection by measuring the long-lived "end-to-end throughput" on each segment of the split path and taking the minimum.

Figure 5.2: A heuristic for evaluating split behavior from end-to-end behavior in an emulated network.

Thus if we know the throughput of the much smaller parameter space of end-to-end connections, we can easily derive:

1. The split throughput of a CCA on a network path composed of two path segments,
2. The end-to-end throughput on the network path,
3. The expected throughput benefit of using a connection-splitting PEP on the network path.

Furthermore, it is impossible to quantitatively evaluate the split throughput of encrypted transport protocols without creating a custom and explicit connection-splitting PEP. As a result, it is challenging to establish a baseline for the throughput that is potentially achievable by QUIC with in-network assistance, leaving many studies to compare QUIC only to split TCP [13, 126, 140]. The split throughput heuristic allows us to estimate the throughput of encrypted transport protocols using knowledge about its end-to-end throughput, without actually splitting the connection.

In Section 5.6, we discuss potential sources of error in the heuristic, including the lack of consideration of the queue configuration on the proxy or the placement of the bottleneck link relative to the data sender. However, we believe there is a design tradeoff in accuracy versus simplicity.

```
class NetworkModel:
    def __init__(self, delay, bw, loss):
        self.delay = delay
        self.bw = bw
        self.loss = loss

def compose(s1: NetworkModel,
            s2: NetworkModel) -> NetworkModel:
    delay = s1.delay + s2.delay
    bw = min(s1.bw, s2.bw)
    loss = s1.loss * (1-s1.loss)*s2.loss
    return NetworkModel(delay, bw, loss)

def throughput(s: NetworkModel) -> float:
    return run_emulation(s)
```

Listing 5.1: An interface for modeling a network path and estimating throughput. The `compose()` function models an end-to-end network path from two path segments. The `throughput()` function obtains a throughput measurement for a path segment.

```
def pred_split_throughput(s1: NetworkModel,
                          s2: NetworkModel) -> float:
    return min(throughput(s1), throughput(s2))

def pred_e2e_throughput(s1: NetworkModel,
                        s2: NetworkModel) -> float:
    s = compose(s1, s2)
    return throughput(s)
```

Listing 5.2: Functions that apply our simplified network model and the split throughput heuristic (Figure 5.2) to predict split and end-to-end throughput, using the interface in Listing 5.1.

In the rest of this section, we first describe how to use this heuristic to analyze the split throughput benefit in a single network setting, without having to run an emulation with a connection splitter. Then we describe how we cache emulation results for an end-to-end parameter space to enable a more open-ended analysis of CCAs over a variety of networks. Finally, we discuss the limitations of our methodology using the heuristic, and propose possible extensions.

5.3.1 Analyzing split throughput benefit in a single network setting

Given a network model of the two path segments that compose an end-to-end network path, we would like to be able to estimate the expected throughput benefit of using a connection-splitting PEP between the two segments. In our emulation study, our simplified network model consists of three parameters: delay, bandwidth, and loss.

We explicitly define the interface we use to query split and end-to-end throughput in Listings 5.1 and 5.2. In addition to the network model, we implement functions to `compose()` a model of an end-to-end network path from two path segments and to query the end-to-end `throughput()` of a segment.

The `compose()` function.

Since our network model consists of three parameters, we describe how to compose each of these parameters to model the end-to-end network path. Bandwidth is the minimum of the two bandwidths, which is the bottleneck. (We may also refer to "bandwidth" as "link rate.") Delay is just additive. If we think of loss as independent random loss, then the composition of the two is $loss = loss1 + (1 - loss1) \cdot loss2 = loss1 + loss2 - loss1 \cdot loss2$.

Note that many combinations of path segments can compose to the same end-to-end network path. This makes sense because regardless of where on the network path loss occurs or which path segment has the bottleneck bandwidth, from an end-to-end perspective, the network properties look the same.

The `throughput()` function.

This is the throughput of a long-running HTTPS connection in a `mininet` emulation. The emulated network is parameterized by the network model of a single path segment, and the HTTPS implementation uses the congestion control scheme of interest. We describe additional details about the emulation in Section 5.4.

Calculating the split improvement.

Listing 5.2 demonstrates how we can estimate the split throughput benefit using the interface in Listing 5.1. We `pred_split_throughput()` by taking the minimum measured throughput on each network path segment, and `pred_e2e_throughput()` by composing the path segments into an end-to-end network path and using the measured throughput of that network path. The split throughput benefit is just how much the split throughput has improved (if it has) relative to the end-to-end throughput.

5.3.2 Caching throughput measurements for analysis

While we can now predict split throughput for congestion control schemes which are not easily splittable, evaluating split throughput on a variety of network settings still requires running emulations for each query. In order to speed up our analysis, we select a parameter space of delay, bandwidth, and loss within our network model and cache the end-to-end emulation results for this parameter space. We describe our choice of parameter space and other modifications as follows:

Parameter	Values	Unit
Bandwidth	[10, 20, 30, 40, 50]	Mbit/s
Delay	[1, 20, 40, 60, 80, 100]	ms
Loss	[0, 1, 2, 3, 4]	%

Table 5.1: Network parameter space for the $5 \cdot 6 \cdot 5 = 150$ combinations of network settings that we analyze in Section 5.5. These values attempt to realistically reflect network settings in which we'd see a PEP.

Selecting a parameter space. We select a range of values we thought would realistically reflect network settings in which we'd see a PEP (Table 5.1). We select propagation delays from 1 ms for a Wi-Fi last hop to 100 ms to reflect the longer RTTs of a satellite connection. Bottleneck bandwidths range from 10 to 50 Mbit/s for a single connection, and are within the CPU constraints of emulation. Random loss rates range from 0% for a stable connection to 4% for random loss caused by e.g., wireless interference and extraterrestrial weather.

Modifying the `compose()` function.

We make some subtle changes to the compose function to keep composed network models within our parameter space. In some sense, each parameter e.g., delay, can be thought of as an algebraic group where the operator function is defined in `compose()`.

For path segments with small delays (1 ms), we consider the composed path segment to just have the delay of the longer segment, so we can have segments with 1 ms delay. For loss, note that when loss rates are small, $loss1 \cdot loss2$ is negligibly small. In our case, the loss is always below 4%, so we omit the term in the composition and loss becomes additive.

Collecting data efficiently.

For some CCAs, we expect a large number of network settings in the parameter space to have extremely low throughput. Thus we set a utilization threshold of 5% of the link rate below which we are not interested in the exact throughput of that network setting.

To more efficiently build the cache, we run a search algorithm on the parameter space to explore faster network settings first. The initial network setting we evaluate has the lowest delay, bandwidth, and loss. We explore adjacent data points if and only if the current data point has not timed out.

5.3.3 Limitations

Although our methodology is more efficient for evaluating many network settings and enables the exploration of theoretical scenarios, caching measurements in emulation still takes non-negligible time. A much faster method to estimate sustained throughput would be to use a theoretical model,

such as the TCP macroscopic model for AIMD schemes [93]. However, this method does not reflect the nuances of implementation, and as Mathis states himself, new models are needed for BBR and modern paradigms [91, 92].

Another limitation is our simplified network model and emulation testbed. One could incorporate more parameters into the network model, such as the fluctuating bandwidths of cellular networks [55], or how BBR incorporates ACK aggregation [19]. Another idea is to generate end-to-end measurements from real testbeds instead of emulation. We believe these challenges are not limited to emulation studies on split performance, and hope to use the much more well-understood space of end-to-end behavior to extrapolate split behavior.

We acknowledge that we only test connections in a single-flow environment. This limits what we can say about inter-flow fairness, but we can still infer some aspects from how congestion-control schemes respond to loss and delay in isolation, a classical concept known as "TCP friendliness" [53]. While split connections reduce retransmissions by buffering on the path, their high throughput may also make them unfair. There may also be different fairness implications when combining split with end-to-end connections. If every connection is split, then we refer to other studies for understanding fairness at scale for each side of the concatenation [109].

We also acknowledge that TCP is not limited to bulk data transfers, and that it would be valuable to study other metrics that could be impacted by connection-splitting, such as startup time, packet jitter, and retransmission overheads.

Finally, our heuristic makes the simplified assumption that we can identify the bottleneck path segment based on the minimum measured throughput. In reality, the ability of a data sender to sustain that throughput depends on its ability to saturate the send buffer. This is particularly true for data senders farther along the network path, as their behavior will depend on the queue configuration and the burstiness of previous senders. We explore this more in Section 5.6.

5.4 Measurement methodology

We want to evaluate different congestion control schemes in a variety of network settings. To do this, we run emulation experiments in `mininet` with simple HTTPS clients and servers to measure the throughputs of long-lived data transfers. In this section, we describe our emulated network configurations, HTTPS endpoints and PEP, and other specifications.

Network configuration.

We used two linear network topologies: a one-segment topology for caching end-to-end measurements and a two-segment topology for evaluation with a connection-splitting PEP (Figure 5.3). Both have a client and server node at each end. The two-segment topology additionally has a router node in between. Each path segment has a bridging node to emulate network properties on the link.

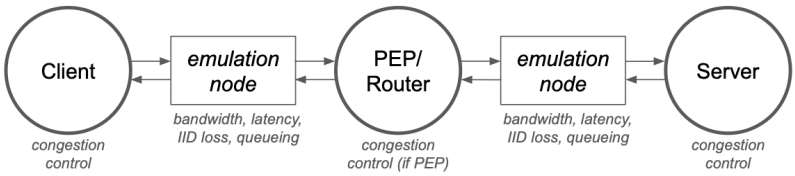


Figure 5.3: Two-segment network topology in `mininet`. The middle node splits the path into two segments with different properties.

CCA	Implementation
BBRv3	Linux TCP v6.4.0+ `google-bbr/v3` fork
BBRv2	Linux TCP v6.4.0+ `google-bbr/v2alpha` fork
BBRv1	Linux TCP v5.15.0-122-generic
CUBIC	Linux TCP v5.15.0-122-generic
BBRv3	Google `quiche` v131.0.6728.1
BBRv1	Google `quiche` v131.0.6728.1
CUBIC	Google `quiche` v131.0.6728.1
BBRv2	Cloudflare `quiche` v0.14
BBRv1	Cloudflare `quiche` v0.14
CUBIC	Cloudflare `quiche` v0.14
BBRv2	IETF `picoquic 29c7c53`
BBRv1	IETF `picoquic 29c7c53`
CUBIC	IETF `picoquic 29c7c53`

Table 5.2: The congestion control schemes and transport protocol implementations we evaluate in the measurement study.

We parameterize each path segment in three dimensions: delay, bandwidth, and a random loss rate. We configure the network properties on the bridging nodes' egress interfaces, using `tc-netem` to set delay and random loss, and `tc-htb` to set bandwidth. Additionally, we use `tc-qdisc` to configure the queues to use RED[1], which is the source of congestive loss. Each link is symmetric in the uplink and downlink directions. For some versions of BBR, we set an `fq` qdisc on the host nodes' egress interfaces for pacing.

Host configuration.

We create simple HTTPS wrappers around each transport protocol implementation to evaluate the congestion control schemes in Table 5.2. The TCP endpoints use the Python `http` module. For the QUIC implementations, we modify comparable client/server applications from each repository. With the exception of enabling TCP pacing where required by BBR, we use "default values" for tunable

[1]We apply RED and not droptail because it gives more continuous feedback about loss for congestion control, and is commonly used in core routers. We also do not need the multi-flow and low-delay properties of other queue disciplines. We use RED in `adaptive harddrop` mode with a maximum queuing delay of ≈ 1 BDP. Exact parameters are available in the code.

parameters, considering these to be part of each implementation.

In TCP, we set the CCA by using a specific Linux kernel version and loading the congestion control module. We use the default kernel's implementations of CUBIC and BBRv1, and Google's kernel fork for BBRv2 and BBRv3 (Table 5.2).

In QUIC, we simply select the CCA as a command-line argument to the user space implementation. We evaluate Google `quiche` [48], Cloudflare `quiche` [28], and `picoquic` [58]. Google and Cloudflare `quiche` are their production implementations of the same name, and `picoquic` is a minimalist implementation based on the IETF spec. All three include CUBIC and BBRv1 implementations, as well as some form of BBRv2 or BBRv3, which is still undergoing standardization.

For the transparent, connection-splitting TCP PEP, we use PEPsal [17]. PEPsal intercepts the SYN packet during the three-way handshake and forms separate TCP connections with each endpoint, copying data between the two sockets. Note that discussions of split QUIC are based only on the split throughput heuristic and do not use an actual splitter.

Experiment specification.

In each experiment, the HTTPS client requests a specific number of bytes to be transferred from the server in the HTTPS payload. This number corresponds to the amount of data that can be transmitted through the bottleneck link over a 10-second period. We have found this to be sufficiently large to reflect sustained throughput.

In practice, we report the "single-stream goodput" calculated as the number of application-layer bytes received (excluding HTTPS headers) divided by request completion time (from when the client sends a request to when it receives a complete response). Due to header overhead and request latency, this metric will never equal the link rate.

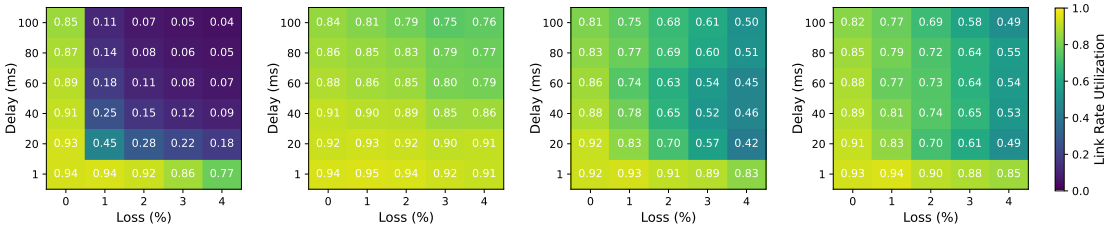
Machine specification.

All experiments use CloudLab [40] x86_64 rs630 nodes in the Massachusetts cluster running Ubuntu 22.04.2. The nodes use the default Linux kernel v5.15.0-122-generic, except for TCP BBRv2 and BBRv3.

5.5 Results

In our emulation measurement study, we aim to answer three questions to gain a more comprehensive understanding of the relevance of connection-splitting with modern networks:

1. Has the model-based BBR algorithm made the throughput enhancements of connection-splitting obsolete?



(a) TCP CUBIC. (b) TCP BBRv1. (c) TCP BBRv2. (d) TCP BBRv3.

Figure 5.4: The link rate utilization calculated as the ratio of achieved goodput to link rate for various TCP CCAs in an emulated network path, at various loss rates and one-way delays, at 10 Mbit/s. CUBIC is the most sensitive to loss and delay, while BBRv1 is the most aggressive and achieves the highest utilizations; BBRv2 and BBRv3 are more sensitive to loss than BBRv1 and utilization starts to suffer (left to right). CCAs tend to achieve lower utilizations, though higher absolute goodputs, as the link rate increases (Section 5.8). Median of $n = 20$ trials.

2. Are there classes of network settings where BBR benefits significantly from splitting but CUBIC does not?
3. How does CCA implementation impact end-to-end behavior, and therefore split behavior?

To recap our measurement methodology, we have a method for measuring the throughput of TCP connections in emulation both with and without a transparent PEP. We also have the ability to estimate throughput both with and without a generic connection splitter, for both TCP and QUIC, based on knowledge of the network model of each path segment, and the measured end-to-end throughput on each segment.

For our analysis, we cache measurements for the end-to-end network parameters in Table 5.1 and the congestion control schemes in Table 5.2. We also run experiments in the two-segment network topology to validate some of these predictions. Raw data for the end-to-end behavior of each CCA implementation are available in Section 5.8. Our emulation benchmarks are also publicly available on GitHub: `https://github.com/StanfordSNR/connection-splitting`.

5.5.1 Finding 1: TCP BBR over time

Has the model-based BBR algorithm made the throughput enhancements of connection-splitting obsolete? We find this line of thought has some truth with the initial release of TCP BBRv1 in 2016. But as the BBR algorithm has evolved into BBRv2 in 2019 and now BBRv3 in 2023, it has also evolved to behave more conventionally in the sense that it benefits from being split—just like more traditional, loss-based congestion control schemes such as CUBIC.

Figure 5.1 shows three different network settings in which BBRv2 and BBRv3 show significant throughput gains from connection-splitting. When BBRv1 was released, its throughput both with and without the connection-splitter were roughly the same, and nearly achieved the bottleneck link

rate. With BBRv2 and BBRv3, the end-to-end throughput drastically deteriorated, behaving more like CUBIC than BBRv1. However, the split throughput remained relatively high, suggesting that BBRv3 today would still benefit from connection-splitting.

Analysis.

To explore why CUBIC and BBRv3 benefit from splitting but BBRv1 does not, we analyze end-to-end throughputs of each CCA and apply the split throughput heuristic (Figure 5.2). As described in Section 5.3, this methodology allows us to study split settings by measuring the much smaller parameter space of end-to-end connections. We find that connection-splitting is likely to improve throughput on lossy paths for CUBIC, BBRv2, and BBRv3 connections, but not for BBRv1.

Figure 5.4 visualizes our cached end-to-end measurements as link rate utilization heatmaps of loss vs. delay. Here is an example for how to interpret these graphs using Figure 5.4d. The top-left cell represents a 100 ms 0% segment with 0.82 utilization of the 10 Mbit/s link rate, and the bottom-right cell represents a 1 ms 4% segment with 0.85 utilization of the 10 Mbit/s link rate. The predicted split utilization of a network path composed of these two segments is just 0.82, the minimum. The two cells compose to the top-right cell, which represents a 100 ms 4% 10 Mbit/s segment with an end-to-end utilization of 0.49. Since $0.82 > 0.49$, we say that splitting has improved the throughput of this network path.

BBRv1 achieves high link rate utilization in all settings (Figure 5.4b), showing it has little to gain from splitting. In fact, previous studies have shown that BBRv1 achieves $\approx$85% link utilization regardless of loss (same as our findings) before it reaches a cliff point at around 20% loss [18, 22]. This may be why it appears that splitting in lossy settings is now obsolete with BBR. However, one should note that BBRv1's high throughput has long been attributed to its aggressiveness and unfairness to legacy algorithms [18, 130], which is what led to the changes in BBRv2 and BBRv3.

While BBRv2 is a large departure from BBRv1, BBRv3 has been described as BBRv2 with bugfixes and performance tuning [20], which supports why the two are so similar. We focus on BBRv3 since Google hopes to now deprecate BBRv2 [20].

BBRv3 is more sensitive to loss than its previous versions (Figure 5.4d), and more similar to CUBIC (Figure 5.4a) in that there exist scenarios where end-to-end throughput suffers. These reflect existing findings of lower utilization in BBRv2 and BBRv3 [32, 122, 142]. Based on the heuristic, it is clear that in some lossy networks such as the ones empirically evaluated in Figure 5.1, connection-splitting can significantly increase the throughput of BBRv3 connections. It is possible that the BBR algorithm continues to evolve in this direction given that BBR's unfairness remains contentious today [32, 142].

Summary.

While BBR may not have benefited from splitting with the release of "v1" in 2016, BBRv2 and now BBRv3 have evolved to behave more conventionally—similar to traditional, loss-based CCAs such as CUBIC—in the sense that they *do*. Even so-called "model-based" congestion control algorithms seem to now react to loss as a congestion signal, as the BBR algorithm continues to evolve.

5.5.2 Finding 2: TCP BBRv3 vs. TCP CUBIC

Are there new classes of network settings where BBR benefits significantly from splitting but CUBIC does not? We want to understand the network settings in which a congestion control scheme is not able to achieve practical bottleneck link rate utilizations end-to-end, but is with a connection-splitter.

We find that while splitting benefits BBRv3 in all the same scenarios as CUBIC, it also has the potential to benefit BBRv3 in many *new* scenarios. In addition to edge deployments with a lossy last-mile, BBRv3 also benefits from splitting in scenarios where there is loss on both path segments, and when the connection-splitter is located farther from the edge. We partition these scenarios into three classes of network settings:

I. Paths with asymmetric delay and a lossy last-mile,
II. Lossy paths with asymmetric delay,
III. Lossy paths with more symmetric delay.

Figures 5.1a to 5.1c represent network settings in each class, respectively. "Asymmetric" refers to the delays on the two path segments. These emulations empirically demonstrate that CUBIC only benefits in the first class, while BBRv3 benefits in all three. BBRv1 does not need splitting in any context.

Analysis.

To identify which network paths benefit from connection-splitting and where along the paths PEPs should be deployed, we apply the heuristic (Figure 5.2). For each CCA, we conduct an exhaustive search of the $15 \cdot 21 \cdot 25 = 7875$ combinations of settings within our parameter space (Table 5.1), and efficiently predict the split and end-to-end throughputs.

We filter on the predicted throughputs for network settings where splitting improves end-to-end throughput by at least $3\times$, and where the split connection utilizes at least half the bottleneck link rate (Table 5.3). BBRv1 does not meet these criteria is any scenarios, and the theoretical connection-splitter is unable to improve the throughput of BBRv1 by even 50%. CUBIC and BBRv3 meet these criteria in 942 and 188 scenarios, respectively. CUBIC benefits from splitting in more scenarios because its end-to-end utilization is more frequently low, so it more frequently has a large split improvement.

Filter	BBRv1	CUBIC	BBRv3
Initial	7875	7875	7875
Split improvement $> 3\times$	0	2231	234
Split utilization > 0.5	0	942	188
Asymmetric, last-mile	0	942	38
Asymmetric, lossy	0	0	72
Symmetric, lossy	0	0	78

Table 5.3: An exhaustive search of network paths and their PEP locations that benefit from splitting for each CCA, and the number of filtered settings that belong to each class.

Since the distribution of network paths in our parameter space does not reflect any meaningful real world distribution, we are more interested in the *classes* of network settings that benefit from splitting. We realized that *all* of the relevant network settings for CUBIC can be clustered into Class I, as network paths where one path segment has 1 ms delay and non-zero loss, and the other has > 1 ms delay and 0% loss. However, Class I only accounts for 21% of the relevant network settings for BBRv3. We identify Class II, which is the same as I, except both path segments have non-zero loss. Class III is the same as II, except both path segments have > 1 ms delay. We used the results to select three representative network settings to empirically evaluate in Figure 5.1.

Intuitively, we can understand why BBRv3 benefits more from splitting in lossy scenarios than CUBIC based on how it reacts to loss and delay (Figure 5.4d). BBRv3's sensitivity to loss and delay is more gradual than abrupt, so it is more likely to benefit when splitting a lossy, high-delay network path in any way. In comparison, CUBIC's throughput falls off a cliff for many of these segmentations.

Discussion.

Do these results reflect where PEP deployments have been useful in the real world? Connection-splitters have traditionally been found in satellite, cellular, and Wi-Fi networks with a wireless link or rate policer [41, 57]. This resembles Class I, where the network path consists of a lossy last-mile, and a reliable Internet path segment. It makes sense then for PEPs to be traditionally located at the edge to address the issues of loss-based schemes [45, 50, 105]. We expect these PEPs to similarly benefit BBRv3 in the same locations.

In Classes I and II, asymmetric delay can be severe in low-resource networks in addition to wireless last-mile links; Consider, for example, regions with no IXPs in which a significant proportion of Internet traffic travels internationally.

For Classes II and III, satellite (and also wireless ad-hoc) networks are known for having lossy "middle-miles" [13, 80, 105, 110]. This can be due to bad weather, fast-moving satellites, and long-distance radio waves, etc. Since CUBIC does not trivially benefit from splitting in these scenarios, the traditional solution has been to split the connection at multiple points and to use an FEC-based

or other proprietary protocol in the satellite backhaul [13, 50, 105]. Our results suggest such an invasive solution may not be necessary for BBRv3.

How could this inform the deployment of connection-splitting PEPs? With the caveat that futher exploration is required to understand how the heuristic extrapolates to the real world, one idea is to determine where to deploy PEPs along an existing network path for maximum benefit especially as congestion-control schemes evolve. Another idea is given the location of a PEP, determine which connections going through the PEP would most benefit based on knowledge of each connection's network path. We leave network operators to decide how best to model their networks and apply the heuristic to evaluate potential PEP deployments.

Summary.

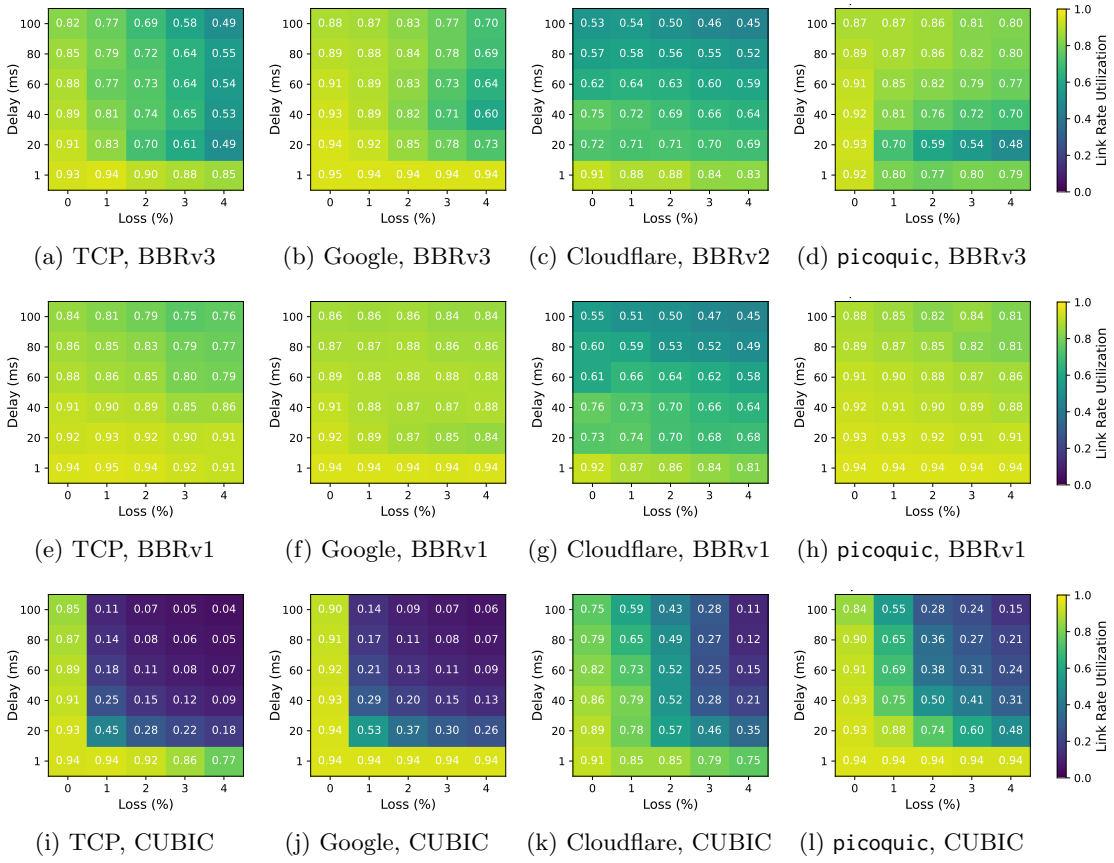
While TCP CUBIC only benefits from connection-splitting when the PEP is located at the lossy last-mile, TCP BBRv3 can also benefit when there is loss on both sides of the PEP and when there are longer propagation delays. This suggests that TCP connections using BBRv3 should benefit from splitters at the same locations as for CUBIC, and also that traditional methods used to address loss in the middle-mile could use simple connection-splitting instead. We believe this method of analyzing useful network paths and PEP placements for connection-splitting can be extended to model new types of networks, especially in space.

5.5.3 Finding 3: QUIC congestion control

How does CCA implementation impact end-to-end behavior, and therefore split behavior? Might claims about "split throughput" depend not just on the CCA, but also the *implementation* and/or the transport protocol on top of it?

For our initial study, we compared end-to-end throughput for four open-source implementations each of CUBIC, BBRv1, and BBRv2/3; one using TCP and three using QUIC. We find that the end-to-end behavior of each CCA varies by implementation in both baseline throughput and sensitivity to loss and delay. To evaluate the split behavior of QUIC, instead of creating a custom and explicit connection-splitting PEP for each QUIC implementation, we apply the split throughput heuristic and argue that these implementations will likewise respond differently to connection-splitting PEPs.

Figure 5.5 visualizes the end-to-end behavior of these schemes. We highlight that the Cloudflare and `picoquic` CUBIC implementations are less sensitive to loss than Google or TCP; the former may benefit from splitting in more classes of network settings than the latter. Additionally, their BBR implementations exhibit non-uniform behavior, suggesting a non-uniform response to connection-splitting. These variations indicate that the benefits of in-network assistance should be considered along with not just the CCA but its specific implementations.



(a) TCP, BBRv3 (b) Google, BBRv3 (c) Cloudflare, BBRv2 (d) `picoquic`, BBRv3

(e) TCP, BBRv1 (f) Google, BBRv1 (g) Cloudflare, BBRv1 (h) `picoquic`, BBRv1

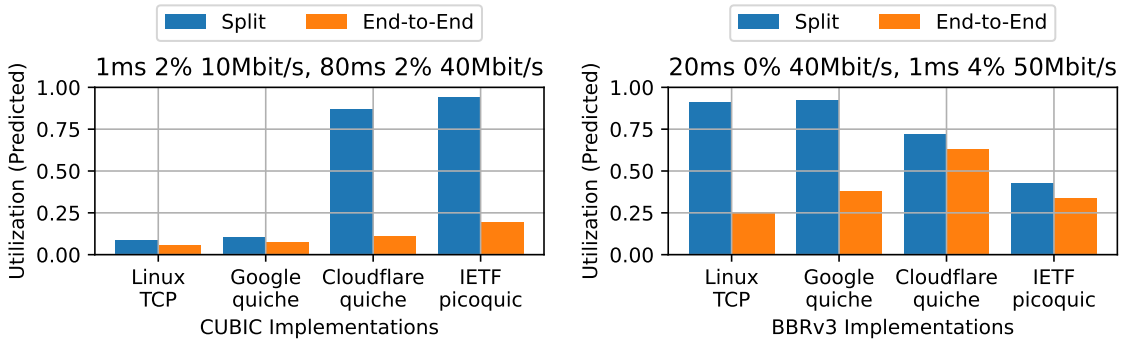
(i) TCP, CUBIC (j) Google, CUBIC (k) Cloudflare, CUBIC (l) `picoquic`, CUBIC

Figure 5.5: Heatmaps for three QUIC implementations of BBRv3 (or BBRv2), BBRv1, and CUBIC showing link rate utilization calculated as the ratio of achieved goodput to link rate, compared to Linux TCP. The heatmaps are shown at various loss rates and one-way delays with a fixed link rate of 10 Mbit/s. User-space QUIC is not CPU-limited, achieving high utilizations at 1 ms delay and 0% loss. The QUIC implementations are Google `quiche`, Cloudflare, and a minimalist implementation based on the IETF spec called `picoquic`. Median of $n = 20$ trials.

Analysis.

The Google QUIC (Figures 5.5b, 5.5f and 5.5j) and TCP (Figures 5.5a, 5.5e and 5.5i) implementations are most similar to each other for each CCA. This is reasonable if we take the community-based CUBIC implementation in Linux to be the standard, and considering that Google contributed to the Linux BBR implementations. In general, Google QUIC achieves slightly higher utilization than Linux TCP.

The Cloudflare QUIC BBR implementations (Figures 5.5c and 5.5g) exhibit profoundly different behavior from Linux TCP in baseline performance. Note that Cloudflare uses BBRv1 for TCP but their use of BBR in QUIC is experimental. Anecdotally, a Cloudflare employee has expressed difficulty



(a) Some QUIC CUBIC implementations can benefit in new network classes where TCP CUBIC could not.

(b) BBRv3 implementations have non-uniform end-to-end behavior and no clear resulting split behavior.

Figure 5.6: Predicted bottleneck link rate utilizations calculated from the predicted end-to-end and split throughputs of the TCP and QUIC implementations, on two different network path segments. End-to-end behavior of each CCA varies significantly by implementation.

making their BBR implementation performant, having to reverse engineer the Linux kernel [62]. Given the wide adoption of BBRv1 for TCP at many CDNs [131], we expect it to be desirable yet challenging for these same companies to correctly incorporate BBRv3 into their QUIC stacks in the coming years.

The `picoquic` BBR implementations (Figures 5.5d and 5.5h) are more similar to Linux TCP, although its BBRv3 implementation seems to have a contradicting reaction to delay. We note that `picoquic` is intended for experimental use in the IETF [58]. It is important then to understand its congestion control behavior if it is to be used to evaluate IETF proposals. We believe its differences from Linux TCP warrant further exploration, but perhaps also that the ongoing standardization efforts of BBR in the IETF [23] indicate that there is no monolith yet of "the BBR algorithm."

The Cloudflare QUIC and `picoquic` implementations of CUBIC (Figures 5.5k and 5.5l) interestingly both exhibit a more gradual degradation in response to loss and delay than Linux TCP (Figure 5.5i). We find this harder to explain, given that TCP CUBIC has been around since 2006, and perhaps can be attributed to transport protocol mechanisms in QUIC. Nevertheless, this indicates that it is important to understand the behavior of a CCA in the context of its entire implementation.

Figure 5.6 applies the heuristic to show how split behavior can vary for CCA implementations with different end-to-end behaviors. Some QUIC CUBIC implementations benefit in new network settings where TCP CUBIC does not, while the various QUIC BBRv3 implementations exhibit non-uniform end-to-end behavior and thus no clear trend in split behavior.

Discussion.

Why do we believe we can extrapolate split behavior from the end-to-end behavior of QUIC? Previous studies explore the effects of QUIC's transport protocol mechanisms in such a case [74, 126]. They

find that the effects of zero-RTT connection establishment with regards to long-lived throughput to be minimal, and stream multiplexing to be mutually beneficial in both scenarios. Further studies can clarify the interactions between CCAs and transport mechanisms.

How do we know that the difference in behavior is due to the congestion control implementation and not other transport protocol mechanisms? Well, we don't, and there is known to be significant variance in the features supported by different QUIC implementations [90]. However, we find it intuitive that congestion control would be a major factor in the measured sustained goodput of a bulk file transfer.

The divergence in QUIC implementations is not so dissimilar from that of TCP at a similar state of evolution [2], but there are still some fundamental differences. With QUIC implementations in userspace, there is a larger diversity of implementations that are easier to tune for a specific application metric, as opposed to correctness or fairness from a congestion-control point of view. These algorithms are also highly parameterized, with no standard nor test suite, so it's not surprising that the implementations differ.

If there is reason to believe that QUIC is losing out on throughput by not connection-splitting, then there will continue to be research on how to achieve the same benefits without ossification [76, 77, 140, 141]. The heuristic helps us understand the theoretical achievable throughput with a simple connection-splitter, or even by combining multiple end-to-end congestion control schemes. In addition to research, this could motivate privacy-minded proposals in the Internet standards community to also view themselves as potential deployment opportunities for private PEPs [75, 116, 117, 119].

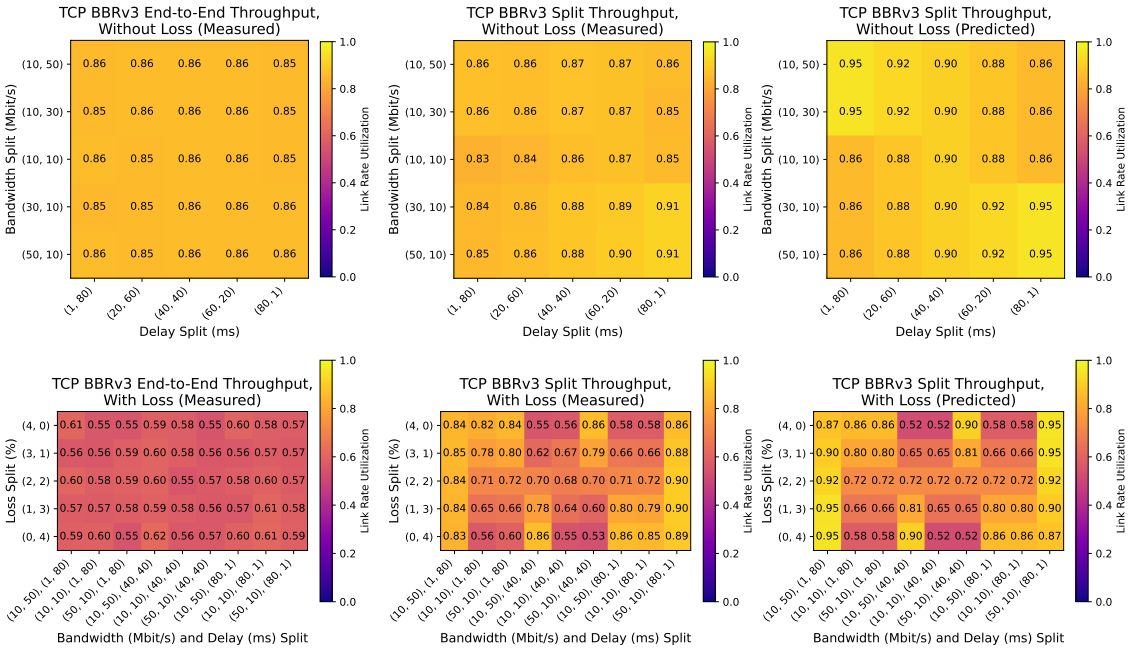
Another application of analyzing CCA implementations is to analyze the Linux TCP implementations of BBR *within* a major version over time. We did this analysis with BBRv1 on 13 Linux kernels between 2016 and 2024, but found no significant variance. However, given the dynamic nature of BBR and the Linux networking stack, it is possible there will still be changes to the split behavior of TCP in the future.

Summary.

We believe it is important to understand the end-to-end behavior of a congestion control scheme in the context of its entire implementation. Our results suggest that BBR is challenging to implement, and that even CUBIC implementations can vary based on context. Whether the prevailing wisdom is that a specific CCA or transport protocol has made in-network assistance undesirable, these results suggest that it is valuable to consistently re-evaluate these claims.

5.6 Accuracy analysis

Our analysis of connection-splitting for TCP and its extensions to QUIC rely on the accuracy of the split throughput heuristic. In this section, we are primarily concerned with how accurate our



(a) E2E throughput accuracy.

(b) Split throughput accuracy.

Figure 5.7: Heatmaps of the measured and predicted BBRv3 throughputs for various splits of delay, bandwidth, and loss, both without (top) and with (bottom) loss. The *end-to-end throughput* predictions (not pictured) are the same for all cells at 0.86 utilization without loss and 0.58 utilization with loss, because they represent the same network path, so end-to-end prediction errors are roughly uniform. The *split throughput* predictions err slightly on the side of overestimation, but they accurately reflect trends in higher or lower throughputs for measurements on different splits of the same network path. Median of $n = 40$ trials.

predictions, which are based on measurements from a one-segment topology, are for measurements from a two-segment topology in emulation, without and with a connection-splitting TCP PEP:

- Does the `compose` function accurately represent the combined network path in the end-to-end throughput?
- Does the split throughput heuristic accurately predict split throughput?

Most importantly, we find the heuristic to be able to usefully predict *trends*. In terms of absolute predictions, we find the `pred_e2e_throughput()` and `pred_split_throughput()` functions to be correct within a reasonable tolerance, with a slight tendency to overestimate.

Orthogonally, we do not evaluate the accuracy to which emulation studies reflect the real world with multi-flow settings and more complex network properties, nor how the accuracy would extrapolate to QUIC connections with custom connection splitters. It may be interesting to explore how to incorporate such factors into the network model and heuristic.

Methodology.

Recall that for a given network path composed of two path segments, we can obtain both the predicted end-to-end and split throughputs, and the ground truth throughputs in an emulated network with and without a TCP PEP. Then for a network setting, we can compute the accuracy as the percent error in predicted vs. measured throughput.

We perform an empirical accuracy analysis of BBRv3 for two end-to-end network paths with identical bandwidth and delay, both without (0%) and with (4%) loss. We test various splits for the bandwidth, delay, and loss and analyze the accuracy trends. In particular, we select delay splits such that the end-to-end delay is 80 ms, bandwidth splits such that the bottleneck bandwidth is 10 Mbit/s, and loss splits such that the total loss is either 0% or 4%. We parameterize the network path segments to use the cached measurements from Table 5.1.

5.6.1 End-to-end throughput accuracy

Each of our splits composes to the same end-to-end network path, so we predict the same end-to-end throughput for each. Our experimental results in Figure 5.7a show that the measured end-to-end throughputs are also roughly uniform, especially without loss, indicating that our method of composing network path segments in emulation represents the same end-to-end network path.

5.6.2 Split throughput accuracy

The split throughput predictions accurately reflect trends in loss, delay, and bandwidth (Figure 5.7b). For example, split throughput is generally higher when the high-bandwidth link is paired with high delay (the yellow-est cells). It is also lower when the lossy link is paired with high delay (columns 1 and 2) or low bandwidth (columns 3-5).

The split throughput predictions tend to slightly overestimate, but we think the level of error is small enough to be helpful for informing real PEP deployment. The maximum error is ±14%, and on average ±4%. This dwarfs the measured gains in some situations, and rules out a large gain in others.

One factor that may lead to overestimation of split throughput is the queue behavior and the burstiness of the sender. With small queues and bursty sending, we would expect the far path segment from the data sender to sometimes be limited by the send buffer. This could subsequently affect how the far connection probes for and utilizes available link rate capacity.

Another factor is the proximity of the bottleneck link to the sender. While our heuristic does not account for this, the real split throughputs for symmetric pairs of network paths (e.g., the left three and right three columns in Figure 5.7b), show that we slightly overestimate when the low-bandwidth bottleneck link is far from the sender. Based on our reasoning about queues, this would suggest that the far path segment, already the bottleneck, is even further under-saturated.

Overall, we find our end-to-end and split throughput predictions to usefully reflect relative trends and absolute throughput within a reasonable tolerance. They are not intended to make claims about exact achievable throughputs nor about the immediate utility of PEPs in the real world, but simply to reason about how connection-splitting may impact long-lived throughput in a simplified network model.

5.7 Summary

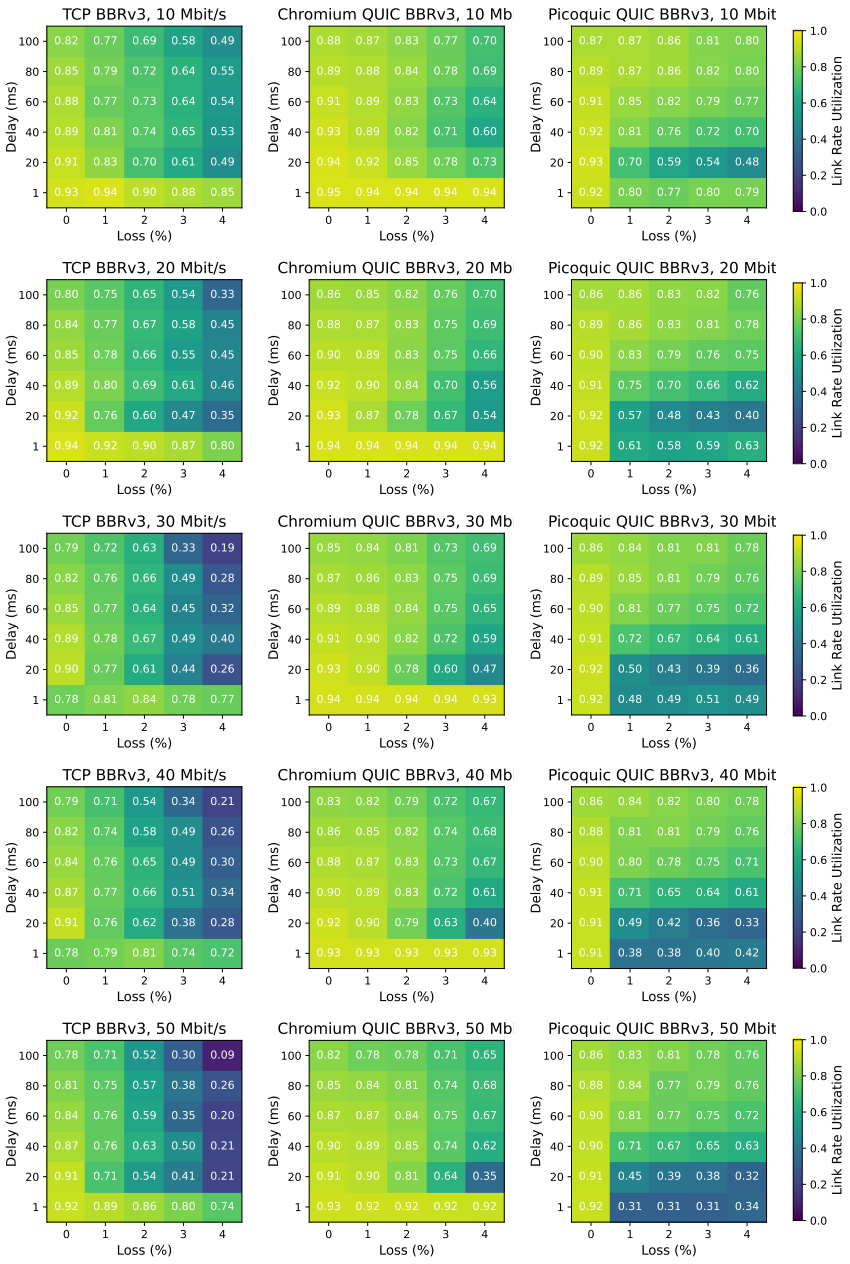
We performed an emulation measurement study on the impact of connection-splitting PEPs on sustained throughput in the context of recent developments such as BBR and QUIC. We found that TCP BBR benefits more from splitting today than when it was first released, and its current version benefits in settings where TCP CUBIC does not. QUIC congestion-control implementations exhibit substantial variability both within QUIC and with Linux TCP.

In the short term, we urge researchers to refer to congestion-control schemes by algorithm/implementation/version, not just "BBR" or even "QUIC BBRv1". We urge the community to create regression and acceptance tests—possibly including our heatmaps with "permissible zones"—for a scheme to call itself an implementation of the XYZv1 algorithm.

For connection-splitting to truly be relevant again, there must be clear scenarios where PEPs offer significant benefits but end-to-end solutions fall short. This work is a first step, but real-world studies are needed. Next, even though the problem is old, the solution need not be to "just do the same things." We hope this chapter motivates the community to pursue protocol-agnostic approaches to in-network assistance and destigmatize the controversial nature of PEPs.

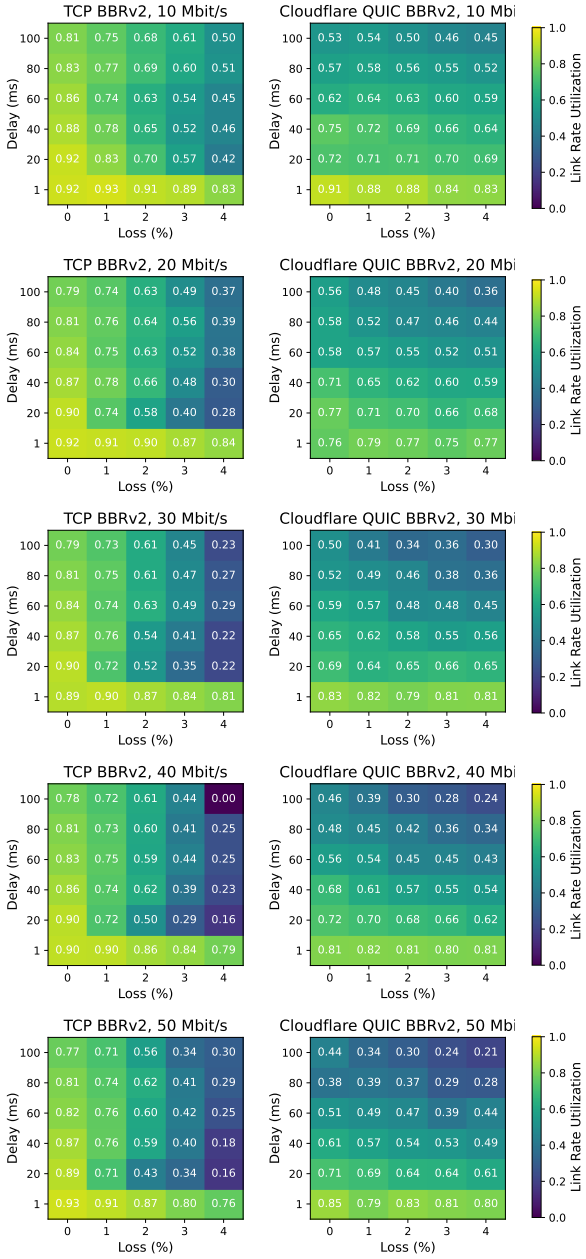
5.8 Appendix: Congestion-control heatmap data

We present the raw heatmap data for our characterizations of each congestion control scheme: BBRv3, BBRv2, BBRv1, CUBIC, and Reno.



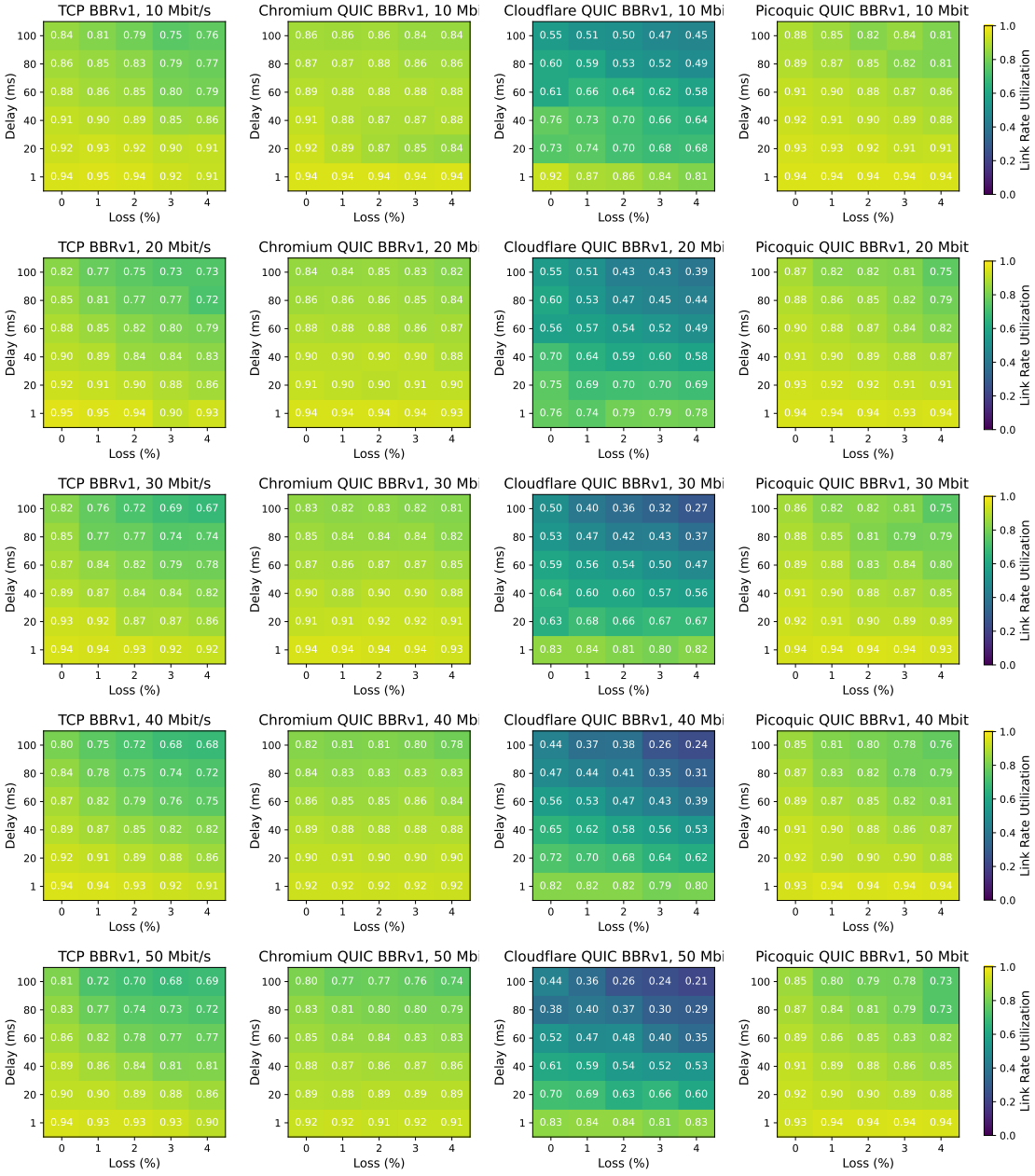
(a) Linux TCP. (b) Google `quiche`. (c) `picoquic`.

Figure 5.8: BBRv3.



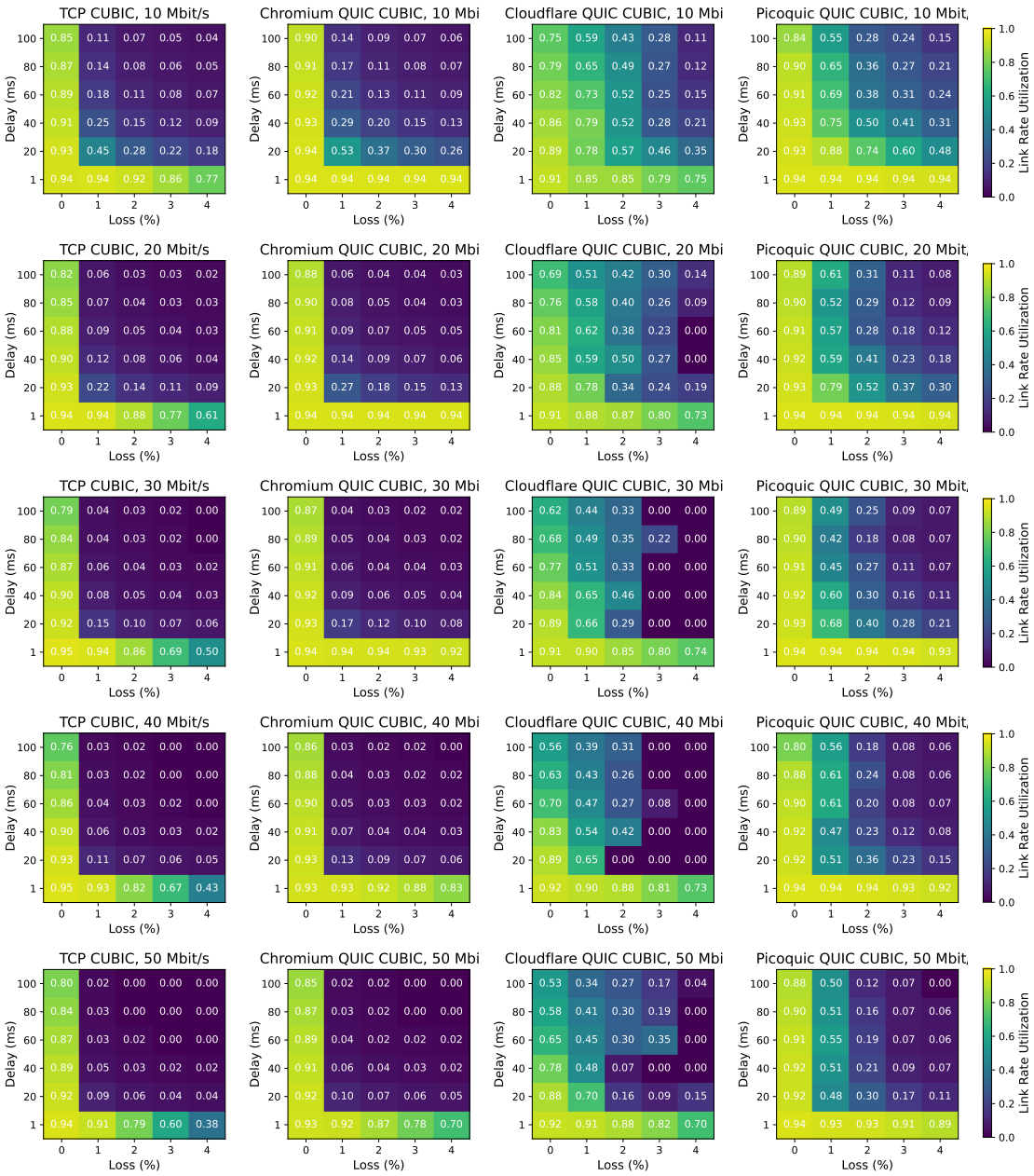
(a) Linux TCP. (b) Cloudflare `quiche`.

Figure 5.9: BBRv2.



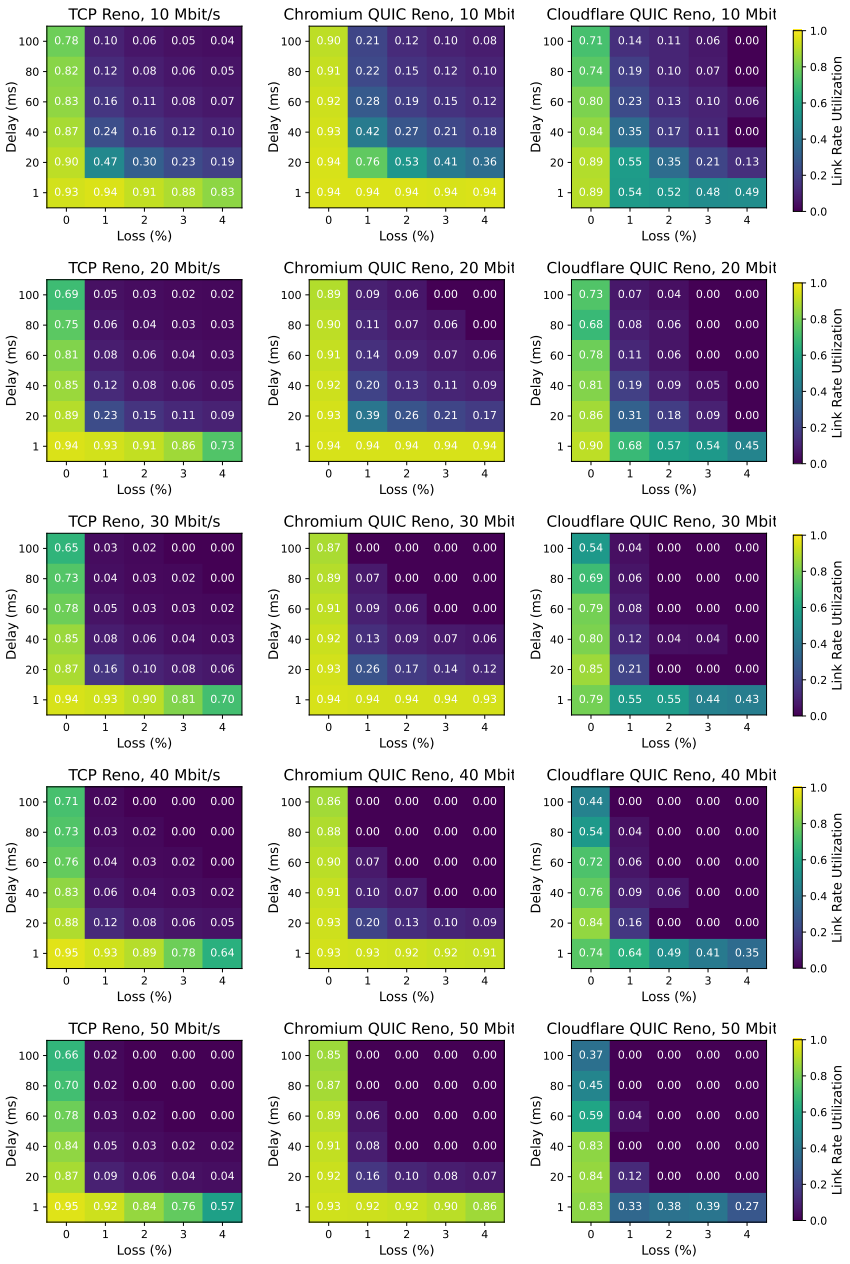
(a) Linux TCP. (b) Google `quiche`. (c) Cloudflare `quiche`. (d) `picoquic`.

Figure 5.10: BBRv1.



(a) Linux TCP. (b) Google `quiche`. (c) Cloudflare `quiche`. (d) `picoquic`.

Figure 5.11: CUBIC.



(a) Linux TCP. (b) Google `quiche`. (c) Cloudflare `quiche`.

Figure 5.12: Reno.

Chapter 6

Conclusion

This dissertation explored a new approach to resolving the tension between performance and ossification for secure transport protocols. This tension emerged over the last few decades starting with the growth of wireless networks and lossy, high-delay paths. TCP performance-enhancing proxies were deployed to ameliorate the poor performance of "loss-based" congestion control algorithms such as CUBIC in these settings, but PEPs have since become controversial for preventing existing transport protocols from evolving on the wire. This led to the deployment of "secure" transport protocols such as QUIC that encrypt their transport headers and forgo all proxy assistance.

In the **Sidekick** approach to in-network assistance, proxies and endpoints send information on an *adjacent* connection to the base connection *about* the encrypted packets that they have received. This information is called the **quACK**, applying set reconciliation techniques to be able to efficiently refer to random identifiers even at packet-scale settings (Chapter 2). When proxies and endpoints send quACKs using the Sidekick protocol (Chapter 3), possibly with a Packrat proxy (Chapter 4), we find that secure transport protocols are able to achieve performance benefits similar to those achieved by TCP using traditional PEPs. These benefits include earlier retransmissions, path-aware congestion responses, reduced energy usage at the endpoint, and reduced network congestion—all without modifying the wire format nor the packets of the underlying connection.

At times, it seems the world has already moved on from PEPs, designing new congestion control algorithms and transport protocols to supplant those of the 1990s connection-splitting era. But this dissertation has shown that even the latest "model-based" BBR algorithm remains vulnerable to lossy networks—and that in some network conditions, at least in emulation, proxies still significantly improve the long-lived throughput of modern QUIC (Chapter 5). In-network assistance from proxies continues to offer valuable performance benefits, but will we have learned our lesson about ossification? The Sidekick approach is one such proposal for reconciling the two. I hope this work encourages efforts to better establish the scope of this problem in real-world deployments, and to design more practical, deployable methods that reimagine how endpoints and the network can securely collaborate.

6.1 Discussion on real-world studies and deployment

After the publication of our NSDI '24 paper [140], we presented this work to various stakeholders in industry (Google, satellite and cellular network operators) and the Internet standards community at the IETF to gauge whether there was any practical interest. Here, I summarize some of my main takeaways and my view on the obstacles that must be overcome for such a proposal to come to life.

Do these stakeholders believe a problem exists and needs to be solved?

When we first approached industry with this problem about in-network assistance for secure transport protocols, we faced some skepticism about its premise. For example, studies from Google showed that QUIC empirically matched or exceeded TCP's performance in wide deployment [81], and others believed that BBR's minimization of loss as a congestion signal meant that connection-splitting was no longer relevant [70]. This inspired us to perform the measurement study in Chapter 5, which showed that BBR's behavior has evolved to benefit more from splitting over time and that perhaps the empirical studies of QUIC are missing certain less-studied classes of network settings.

Network operators provided mixed feedback despite widely deploying PEPs themselves. Cellular operators described link-layer retransmissions as rendering loss virtually nonexistent. On the other hand, satellite network operators were extremely enthusiastic. It is a valid concern that Sidekick only benefits network settings with random loss, though Chapter 4 evaluates some of the tradeoffs with link-layer approaches. Real-world studies in collaboration with network operators are required to truly understand the scope of the problem in practice. It would be valuable to look at both networks where PEPs are currently deployed (Class I in Chapter 5), as well as satellite, wireless ad-hoc, and other networks where reliable connectivity is still an unsolved problem (Classes II and III).

There must also be personal incentives for transport protocol developers and network operators to deploy a solution, even if they believe the problem exists. Network operators want their users to experience good performance as a signal of the network quality they pay for. Protocol developers may only care about good performance that covers a majority of use cases, even if the tail end suffers.

If so, do they believe Sidekick protocols are the right solution?

There are risks to deploying new solutions, especially when two independent parties need to collaborate. Unlike transparent PEPs, Sidekick protocols require both endpoints and proxies to deploy their half of the solution before they can observe performance benefits. When deployed unilaterally, the proxy pays a cost for in-line packet processing, while the endpoint has invested in integrating the complex coding theory of the quACK into their application. The status quo is simply easier to maintain.

Another risk is that the performance of Sidekick protocols in the real world is not well understood. Although Section 3.7 showed some empirical promise, the improvements were still less pronounced and more variable than those in emulation. We hypothesized that these results could be explained

by the greater variability in real networks, and that a dynamically configured quACK would improve performance. Such improvements can be made after an initial deployment.

A final obstacle in practice is that there needs to be a protocol standard so that the endpoint and proxy can communicate with each other when they are controlled by different entities. We found some common ground with the MASQUE and SCONEPRO communities at the IETF for signaling in-network information over an adjacent connection, but ultimately these proposals already had specific scopes in mind, and the Sidekick proposal had the limitations described here.

One workaround to standardization would be to evaluate Sidekick protocols from an organization that controls both entities. One such environment would be a home, office, or public Wi-Fi environment where we could run a proxy on an in-line device behind the wireless router in a building with clients that we control. Another environment would be with a business that controls both an application and some CDN/SFU infrastructure with in-network nodes that can act as proxies.

6.2 Future research directions

Building on the ideas developed in this dissertation, I now explore several open-ended directions for future research.

Performance engineering the quACK.

In Chapter 2, we detailed the performance engineering required to make set reconciliation practical at packet scales, including nanosecond per-packet overheads and communication sizes under 100 bytes. One limitation of the current IBLT construction is that decoding with high probability requires sending significantly more symbols than the number of missing elements, undermining the rateless property. Future work could explore new mapping functions or data structures that reduce the chance of collision when decoding small set sizes, or that consider a known distribution of packet identifiers, or even an interactive Sidekick protocol that uses more than one RTT for set reconciliation. Implementing quACK operations in specialized hardware is another open-ended idea for performance optimization and enabling in-line quACK processing in high-speed networks.

Utilizing the quACK in other packet-scale settings.

Could quACKs prove more broadly useful in other networking scenarios? In Chapter 3 and Chapter 4, we applied quACKs to Sidekick protocols so that path-aware endpoints could emulate the behavior of PEPs. However, quACKs could also be used for network packet analysis, aggregating statistics in real-time with minimal space and computation. It may also be interesting to explore the relation of quACKs to data sketches and streaming algorithms, other research areas at the intersection of networking and coding theory. One research problem is how to enforce differentiated services [51] based solely on the observed flow behavior, rather than trusting packet markings. Bloom-filter-like

data structures such as the one in the quACK are especially well-suited to settings like this that demand efficient per-packet operations with approximate statistics.

Sending in-network signals over an adjacent connection.

The general idea of sending in-network signals over an adjacent connection increases endpoint visibility into the network without altering the underlying connection. This generates various opportunities to improve performance from the perspective of the endpoint. A variation on the quACK over the packets that have been *received* could be a quACK over the packets that have been *sent*. This would allow the decoder to explicitly refer to missing packets by their identifiers, an example of another type of path-aware endpoint behavior enabled by quACKs.

Now imagine an omnipotent scheduler aware of the properties of every path segment and the application behavior desired by every endpoint in the network. The signals need not just be quACKs, but could also be the queue policy and state, the current fair flow rate, ECN, etc. What is a *fair* allocation of resources with this knowledge and how can we achieve this allocation (or something close) by signaling information about the network to the endpoints?

Multicast and using proxies to benefit the network.

How can proxies benefit not just applications but the network itself? In most of this work, we have described the advantages of Sidekick protocols in terms of application metrics such as throughput and latency. But in-network retransmissions and reducing the amount of time a connection needs to stay alive can also reduce network congestion, an understated advantage of PEPs.

In an initial exploration of IP multicast in Chapter 4, we showed that a modified Packrat proxy can deliver in-network retransmissions to multicast clients with minimal per-client memory overhead. Multicast inherently reduces network congestion by reducing redundant data delivery over the network. For many practical reasons, including reliability, IP multicast is not used over the Internet today [35]. It would be interesting to explore whether a network with some composition of Packrat proxies could enable a more practical yet reliable form of multicast.

6.3 Immediate impact on congestion control research

Independent of connection-splitting, the measurement study in Chapter 5 has several implications for congestion control research even in end-to-end settings. First, we urge researchers to refer to congestion control schemes by algorithm/implementation/version, not just "BBR" or even "QUIC BBRv1". The networking community once used "TCP" synonymously with its congestion control until many more algorithms evolved. Times have changed again, and we showed BBR to exhibit significantly different behavior over major version changes and in the many implementations of QUIC.

Second, we urge the community to create performance test suites for a congestion control implementation to be able to claim that it conforms to a particular standard. That is, an implementation should empirically achieve the intended behavior of the algorithm. The behavior can be described as a relative trend such as reacting progessively to random loss up to 10%, or an absolute behavior such as achieving the maximum bandwidth in a lossless, single-flow setting. These test suites can include our heatmaps, for example. The goal of establishing an empirical standard is to help implementations achieve more similar behavior in the same settings for a fairer Internet, or even to hold intentionally different implementations accountable and explain the differences.

Finally, the *split throughput heuristic* can be incorporated as a research tool for studying congestion control in split settings. It is challenging for QUIC to emulate the behavior of split QUIC connections without having an explicit connection splitter to benchmark against. The heuristic helps us understand potential performance benefits based on end-to-end changes even in theoretical split settings. It gives us a systematic approach to understanding split behavior when the path is split into multiple segments each using a different end-to-end scheme. More work is also required to understand the fairness of split congestion control when co-existing with end-to-end or other split schemes.

6.4 Final remarks

Sidekick protocols are a novel approach to in-network assistance for secure transport protocols that yield performance benefits similar to those of traditional PEPs, but leave the underlying protocol free to evolve. The performance benefits of connection-splitting remain relevant today especially in lossy network environments. In summary, this work contributes new tools and directions to reimagine how endpoints and proxies can securely collaborate to achieve a better and more performant network.

Bibliography

[1] 3rd Generation Partnership Project (3GPP). NR; NR and NG-RAN Overall Description; Stage 2. TS 38.300, 3GPP, 2025. Version 18.5.0.

[2] Mark Allman and Aaron Falk. On the effective evaluation of TCP. *ACM SIGCOMM Computer Communication Review*, 29(5):59–70, 1999.

[3] Emmanuel André, Nicolas Le Breton, Augustin Lemesle, Ludovic Roux, and Alexandre Gouaillard. Comparative study of WebRTC open source SFUs for video conferencing. In *2018 Principles, Systems and Applications of IP Telecommunications*, IPTComm '18, pages 1–8. IEEE, 2018.

[4] Apple Inc. iCloud Private Relay Overview. `https://www.apple.com/icloud/docs/iCloud_Private_Relay_Overview_Dec2021.pdf`, December 2021.

[5] Jakob Bæk Tejs Houen, Rasmus Pagh, and Stefan Walzer. Simple set sketching. In *Symposium on Simplicity in Algorithms*, SOSA '23, pages 228–241. SIAM, 2023.

[6] Ajay Bakre and B.R. Badrinath. Handoff and system support for indirect TCP/IP. In *Second USENIX Symposium on Mobile and Location-Independent Computing*, Ann Arbor, MI, April 1995. USENIX Association.

[7] Ajay Bakre and B.R. Badrinath. I-TCP: Indirect TCP for Mobile Hosts. In *Proceedings of 15th International Conference on Distributed Computing Systems*, 1995.

[8] Hari Balakrishnan, Srinivasan Seshan, Elan Amir, and Randy H. Katz. Improving TCP/IP performance over wireless networks. In *Proceedings of the 1st Annual International Conference on Mobile Computing and Networking*, MobiCom '95, page 2–11, New York, NY, USA, 1995. Association for Computing Machinery.

[9] Andrea Bittau, Daniel B. Giffin, Mark J. Handley, David Mazieres, Quinn Slack, and Eric W. Smith. Cryptographic Protection of TCP Streams (tcpcrypt). RFC 8548, Internet Engineering Task Force (IETF), May 2019.

[10] BitTorrent Foundation. BitTorrent (BTT) White Paper v0.8.7. https://www.bittorrent.com/btt/btt-docs/BitTorrent_(BTT)_White_Paper_v0.8.7_Feb_2019.pdf, February 2019.

[11] Ethan Blanton, Dr. Vern Paxson, and Mark Allman. TCP Congestion Control. RFC 5681, Internet Engineering Task Force (IETF), September 2009.

[12] John Border. Google QUIC over Satellite Links. Presentation, IETF PANRG interim. https://datatracker.ietf.org/meeting/interim-2020-panrg-01/materials/slides-interim-2020-panrg-01-sessa-google-quic-over-satellite-testing-update, June 2020.

[13] John Border, Bhavit Shah, Chi-Jiun Su, and Rob Torres. Evaluating QUIC's Performance Against Performance Enhancing Proxy over Satellite Link. In *2020 IFIP Networking Conference*, Networking '20, pages 755–760, 2020.

[14] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: programming protocol-independent packet processors. *SIGCOMM Computer Communication Review*, 44(3):87–95, July 2014.

[15] Robert Braden. Requirements for Internet Hosts – Communication Layers. RFC 1122, Internet Engineering Task Force (IETF), October 1989.

[16] Harald Burchardt, Nikola Serafimovski, Dobroslav Tsonev, Stefan Videv, and Harald Haas. VLC: Beyond point-to-point communication. *IEEE Communications Magazine*, 52(7):98–105, 2014.

[17] Carlo Caini, Rosario Firrincieli, and Daniele Lacamera. PEPsal: a Performance Enhancing Proxy designed for TCP satellite connections. In *2006 IEEE 63rd Vehicular Technology Conference*, volume 6, pages 2607–2611, 2006.

[18] Yi Cao, Arpit Jain, Kriti Sharma, Aruna Balasubramanian, and Anshul Gandhi. When to Use and When Not to Use BBR: An Empirical Analysis and Evaluation Study. In *Proceedings of the Internet Measurement Conference*, IMC '19, pages 130–136, 2019.

[19] Neal Cardwell. BBR Congestion Control Work at Google, March 2018. Presentation, IETF 101 London, Internet Congestion Control Research Group (ICCRG). https://datatracker.ietf.org/meeting/101/materials/slides-101-iccrg-an-update-on-bbr-work-at-google-00.

[20] Neal Cardwell. BBRv3: Algorithm Overview and Google's Public Internet Deployment. Presentation, IETF 119 Brisbane, Congestion Control Working Group (CCWG). https://datatracker.ietf.org/meeting/119/materials/slides-119-ccwg-bbrv3-overview-and-google-deployment-00, March 2024.

[21] Neal Cardwell and Yuchung Cheng. BBR Congestion Control. Presentation, IETF 97 Seoul, Internet Congestion Control Research Group (ICCRG). `https://www.ietf.org/proceedings/97/slides/slides-97-iccrg-bbr-congestion-control-01.pdf`, November 2016.

[22] Neal Cardwell, Yuchung Cheng, C. Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. BBR: Congestion-Based Congestion Control. *Communications of the ACM*, 60(2):58–66, 2017.

[23] Neal Cardwell, Ian Swett, and Joseph Beshay. BBR Congestion Control. Internet Draft draft-cardwell-ccwg-bbr-00, Internet Engineering Task Force (IETF), September 2024.

[24] Vinton Cerf, Yogen Dalal, and Carl Sunshine. Specification of Internet Transmission Control Program. RFC 675, Internet Engineering Task Force (IETF), December 1974.

[25] Yuchung Cheng, Neal Cardwell, Nandita Dukkipati, and Priyaranjan Jha. The RACK-TLP Loss Detection Algorithm for TCP. RFC 8985, Internet Engineering Task Force (IETF), February 2021.

[26] Dah-Ming Chiu and Raj Jain. Analysis of the increase and decrease algorithms for congestion avoidance in computer networks. *Computer Networks and ISDN systems*, 17(1):1–14, 1989.

[27] David D. Clark. The design philosophy of the DARPA internet protocols. In *Symposium Proceedings on Communications Architectures and Protocols*, SIGCOMM '88, page 106–114, New York, NY, USA, 1988. Association for Computing Machinery.

[28] Cloudflare, Inc. quiche. GitHub, `https://github.com/cloudflare/quiche`.

[29] Bryce Cronkite-Ratcliff, Aran Bergman, Shay Vargaftik, Madhusudhan Ravi, Nick McKeown, Ittai Abraham, and Isaac Keslassy. Virtualized Congestion Control. In *Proceedings of the 2016 ACM SIGCOMM Conference*, SIGCOMM '16, page 230–243, New York, NY, USA, 2016. Association for Computing Machinery.

[30] Ana Custura, Tom Jones, and Gorry Fairhurst. Impact of Acknowledgements using IETF QUIC on Satellite Performance. In *2020 10th Advanced Satellite Multimedia Systems Conference and the 16th Signal Processing for Space Communications Workshop*, ASMS/SPSC, pages 1–8, 2020.

[31] Ana Custura, Tom Jones, Raffaello Secchi, and Gorry Fairhurst. Reducing the acknowledgement frequency in IETF QUIC. *International Journal of Satellite Communications and Networking*, 41(4):315–330, 2023.

[32] Soumyadeep Datta and Fraida Fund. Replication: "When to Use and When Not to Use BBR". In *Proceedings of the 2023 ACM on Internet Measurement Conference*, IMC '23, pages 30–35, 2023.

[33] Paul Davern, Noor Nashid, Cormac J. Sreenan, and Ahmed H. Zahran. HTTPEP: a HTTP performance enhancing proxy for satellite systems. *International Journal of Next-Generation Computing*, 2(3), 2011.

[34] Quentin De Coninck and Olivier Bonaventure. Multipath QUIC: Design and Evaluation. In *Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies*, CoNEXT '17, page 160–166, New York, NY, USA, 2017. Association for Computing Machinery.

[35] Christophe Diot, Brian Neil Levine, Bryan Lyles, Hassan Kassem, and Doug Balensiefen. Deployment issues for the IP multicast service and architecture. *IEEE Network*, 14(1):78–88, 2000.

[36] Yevgeniy Dodis, Rafail Ostrovsky, Leonid Reyzin, and Adam Smith. Fuzzy Extractors: How to Generate Strong Keys from Biometrics and Other Noisy Data. *SIAM Journal on Computing*, 38(1):97–139, January 2008.

[37] Fahad R. Dogar and Peter Steenkiste. Architecting for edge diversity: supporting rich services over an unbundled transport. In *Proceedings of the 8th International Conference on Emerging Networking Experiments and Technologies*, CoNEXT '12, page 13–24, New York, NY, USA, 2012. Association for Computing Machinery.

[38] DPDK: Data Plane Development Kit. `https://www.dpdk.org/`, September 2023.

[39] Martin Duke. QUIC Version 2. RFC 9369, Internet Engineering Task Force (IETF), May 2023.

[40] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. The design and operation of CloudLab. In *2019 USENIX Annual Technical Conference*, USENIX ATC '19, pages 1–14, Renton, WA, July 2019. USENIX Association.

[41] Korian Edeline and Benoit Donnet. A Bottom-Up Investigation of the Transport-Layer Ossification. In *2019 Network Traffic Measurement and Analysis Conference*, TMA '19, pages 169–176, 2019.

[42] HISTORY.com Editors. World Wide Web (WWW) launches in the public domain. History. `https://www.history.com/this-day-in-history/april-30/world-wide-web-launches-in-public-domain`, May 2025.

[43] David Eppstein and Michael T. Goodrich. Straggler Identification in Round-Trip Data Streams via Newton's Identities and Invertible Bloom Filters. *IEEE Transactions on Knowledge and Data Engineering*, 23(2):297–306, February 2011.

[44] Gorry Fairhurst and Lloyd Wood. Advice to link designers on link Automatic Repeat reQuest (ARQ). RFC 3366, Internet Engineering Task Force (IETF), September 2002.

[45] Viktor Farkas, Balázs Héder, and Szabolcs Nováczki. A Split Connection TCP Proxy in LTE Networks. In Róbert Szabó and Attila Vidács, editors, *Information and Communication Technologies*, pages 263–274, Berlin, Heidelberg, 2012. Springer.

[46] Bryan Ford and Janardhan R. Iyengar. Breaking up the transport logjam. In *Proceedings of the 7th ACM Workshop on Hot Topics in Networks*, HotNets '08, pages 85–90, New York, NY, USA, 2008. Association for Computing Machinery.

[47] Michael T. Goodrich and Michael Mitzenmacher. Invertible Bloom Lookup Tables. In *2011 49th Annual Allerton Conference on Communication, Control, and Computing*, Allerton, pages 792–799. IEEE, 2011.

[48] Google. QUICHE. GitHub, `https://github.com/google/quiche/`.

[49] Prateesh Goyal, Mohammad Alizadeh, and Hari Balakrishnan. Rethinking Congestion Control for Cellular Networks. In *Proceedings of the 16th ACM Workshop on Hot Topics in Networks*, HotNets '17, page 29–35, New York, NY, USA, 2017. Association for Computing Machinery.

[50] Jim Griner, John Border, Markku Kojo, Zach D. Shelby, and Gabriel Montenegro. Performance Enhancing Proxies Intended to Mitigate Link-Related Degradations. RFC 3135, Internet Engineering Task Force (IETF), June 2001.

[51] Daniel B. Grossman. New Terminology and Clarifications for Diffserv. RFC 3260, Internet Engineering Task Force (IETF), April 2002.

[52] Sangtae Ha, Injong Rhee, and Lisong Xu. CUBIC: A New TCP-Friendly High-Speed TCP Variant. *SIGOPS Operating Systems Review*, 42(5):64–74, July 2008.

[53] Mark J. Handley, Jitendra Padhye, Sally Floyd, and Joerg Widmer. TCP Friendly Rate Control (TRFC): Protocol Specification. RFC 5348, Internet Engineering Task Force (IETF), September 2008.

[54] Michael Hauben and Ronda Hauben. *Netizens: On the History and Impact of Usenet and the Internet*. Wiley-IEEE Computer Society Pr, 1997. Chapter 7: History of ARPANET.

[55] David A. Hayes, David Ros, and Özgü Alay. On the Importance of TCP Splitting Proxies for Future 5G mmWave Communications. In *2019 IEEE 44th LCN Symposium on Emerging Topics in Networking*, LCN Symposium, pages 108–116, 2019.

[56] Keqiang He, Eric Rozner, Kanak Agarwal, Yu (Jason) Gu, Wes Felter, John Carter, and Aditya Akella. AC/DC TCP: Virtual Congestion Control Enforcement for Datacenter Networks. In *Proceedings of the 2016 ACM SIGCOMM Conference*, SIGCOMM '16, page 244–257, New York, NY, USA, 2016. Association for Computing Machinery.

[57] Michio Honda, Yoshifumi Nishida, Costin Raiciu, Adam Greenhalgh, Mark Handley, and Hideyuki Tokuda. Is it still possible to extend TCP? In *Proceedings of the 2011 ACM SIGCOMM Conference on Internet Measurement Conference*, IMC '11, page 181–194, New York, NY, USA, 2011. Association for Computing Machinery.

[58] Christian Huitema. picoquic. GitHub, `https://github.com/private-octopus/picoquic`.

[59] Geoff Huston. Just one QUIC bit. APNIC. `https://blog.apnic.net/2018/03/28/just-one-quic-bit/`, March 2018.

[60] IEEE Computer Society. IEEE Standard for Information technology–Local and metropolitan area networks–Specific requirements–Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications - Amendment 8: Medium Access Control (MAC) Quality of Service Enhancements. IEEE Std 802.11e-2005, IEEE, November 2005.

[61] IEEE Computer Society. IEEE Standard for Local and metropolitan area networks – Media Access Control (MAC) Service Definition. IEEE Std 802.1AC-2016, IEEE, March 2017.

[62] Internet Engineering Task Force (IETF). IETF 119: Congestion Control Working Group 2024-03-21 03:00. `https://www.youtube.com/watch?v=ZVqQiA7h-W8`, March 2024. Q&A: 1:37:30.

[63] Jana Iyengar and Ian Swett. QUIC Loss Detection and Congestion Control. RFC 9002, Internet Engineering Task Force (IETF), May 2021.

[64] Jana Iyengar, Ian Swett, and Mirja Kühlewind. QUIC Acknowledgement Frequency. Internet Draft draft-ietf-quic-ack-frequency-07, Internet Engineering Task Force (IETF), October 2023.

[65] Jana Iyengar and Martin Thomson. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000, Internet Engineering Task Force (IETF), May 2021.

[66] Janardhan R. Iyengar and Bryan Ford. Flow splitting with fate sharing in a next generation transport services architecture. *CoRR*, abs/0912.0921, 2009.

[67] Van Jacobson and Michael J. Karels. Congestion Avoidance and Control. In *SIGCOMM 1988*, Stanford, CA, August 1988.

[68] Aman Kapoor, Aaron Falk, Theodore Faber, and Yuri Pryadkin. Achieving Faster Access to Satellite Link Bandwidth. In *Proceedings of the 25TH IEEE International Conference on Computer Communications*, INFOCOM '06, pages 1–6, 2006.

[69] Mark G. Karpovsky, Lev B. Levitin, and Ari Trachtenberg. Data verification and reconciliation with generalized error-control codes. *IEEE Transactions on Information Theory*, 49(7):1788–1793, 2003.

[70] Frode Kileng. Comments at IETF 121 PANRG session. "there's been a sharp decline of PEPs in mobile networks in the last years. BBR is a cause for this. (When AWS switched to BBR 2-3 years ago, throughput of moblle networks with PEPs either declined or a steady trend, while those without had a significant increase in throughput. I.e. as measured bye 3rd party benchmarks, e.g. Tutela)" `https://zulip.ietf.org/#narrow/stream/287-panrg/topic/ietf-121/near/144090`.

[71] Dzmitry Kliazovich, Simone Redana, and Fabrizio Granelli. Cross-layer error recovery in wireless access networks: The ARQ proxy approach. *International Journal of Communication Systems*, 25(4):461–477, April 2012.

[72] Florian Klingler, Falko Dressler, and Christoph Sommer. The impact of head of line blocking in highly dynamic WLANs. *IEEE Transactions on Vehicular Technology*, 67(8):7664–7676, 2018.

[73] S. Koenig, Daniel Lopez-Diaz, Jochen Antes, Florian Boes, R. Henneberger, Arnulf Leuther, Axel Tessmann, R. Schmogrow, D. Hillerkuss, R. Palmer, Thomas Zwick, Christian Koos, W. Freude, Oliver Ambacher, Juerg Leuthold, and Ingmar Kallfass. Wireless sub-THz communication system with high data rate. *Nature Photonics*, 7:977–981, October 2013.

[74] Mike Kosek, Hendrik Cech, Vaibhav Bajpai, and Jörg Ott. Exploring Proxying QUIC and HTTP/3 for Satellite Communication. In *2022 IFIP Networking Conference*, Networking '22, pages 1–9, 2022.

[75] Mike Kosek, Tanya Shreedhar, and Vaibhav Bajpai. Beyond QUIC v1: A First Look at Recent Transport Layer IETF Standardization Efforts. *IEEE Communications Magazine*, 59(4):24–29, 2021.

[76] Mike Kosek, Benedikt Spies, and Jörg Ott. Secure Middlebox-Assisted QUIC. In *2023 IFIP Networking Conference*, Networking '23, pages 1–9. IEEE, 2023.

[77] Zsolt Krämer, Mirja Kühlewind, Marcus Ihlar, and Attila Mihály. Cooperative performance enhancement using QUIC tunneling in 5G cellular networks. In *Proceedings of the Applied Networking Research Workshop*, ANRW '21, page 49–51, New York, NY, USA, 2021. Association for Computing Machinery.

[78] Zsolt Krämer, Sándor Molnár, Marcus Pieskä, and Attila Mihály. A Lightweight Performance Enhancing Proxy for Evolved Protocols and Networks. In *2020 IEEE 25th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks*, CAMAD '20, pages 1–6, 2020.

[79] Nicolas Kuhn, Emmanuel Lochin, François Michel, and Michael Welzl. Forward Erasure Correction (FEC) Coding and Congestion Control in Transport. RFC 9265, Internet Engineering Task Force (IETF), July 2022.

[80] Nicolas Kuhn, François Michel, Ludovic Thomas, Emmanuel Dubois, and Emmanuel Lochin. QUIC: Opportunities and threats in SATCOM. In *2020 10th Advanced Satellite Multimedia Systems Conference and the 16th Signal Processing for Space Communications Workshop*, ASMS/SPSC, pages 1–7, 2020.

[81] Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, Jeff Bailey, Jeremy Dorfman, Jim Roskind, Joanna Kulik, Patrik Westin, Raman Tenneti, Robbie Shade, Ryan Hamilton, Victor Vasiliev, Wan-Teh Chang, and Zhongyi Shi. The QUIC Transport Protocol: Design and Internet-Scale Deployment. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, SIGCOMM '17, pages 183–196, 2017.

[82] Andrew Lanxon. Android vs. iPhone: 15 Years of Innovation Through Rivalry. CNET, a Ziff Davis company. `https://www.cnet.com/tech/mobile/smartphone-showdown-15-years-of-android-vs-iphone/`, April 2024.

[83] Hoang D. Le and Anh T. Pham. Link-layer retransmission-based error-control protocols in FSO communications: A survey. *IEEE Communications Surveys & Tutorials*, 24(3):1602–1633, 2022.

[84] Ka-Cheong Leung, Victor OK Li, and Daiqin Yang. An overview of packet reordering in transmission control protocol (TCP): problems, solutions, and challenges. *IEEE Transactions on Parallel and Distributed Systems*, 18(4):522–535, 2007.

[85] Tong Li, Kai Zheng, Ke Xu, Rahul Arvind Jadhav, Tao Xiong, Keith Winstein, and Kun Tan. TACK: Improving Wireless Transport Performance by Taming Acknowledgments. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM '20, page 15–30, New York, NY, USA, 2020. Association for Computing Machinery.

[86] Yizhou Li, Xingwang Zhou, Mohamed Boucadair, Jianglong Wang, and Fengwei Qin. LOOPS (Localized Optimizations on Path Segments) Problem Statement and Opportunities for Network-Assisted Performance Enhancement. Internet Draft draft-li-tsvwg-loops-problem-opportunities-06, Internet Engineering Task Force (IETF), July 2020.

[87] libcurl - the multiprotocol file transfer library. `https://curl.se/libcurl/`, September 2023.

[88] Anna Maria Mandalari, Andra Lutu, Bob Briscoe, Marcelo Bagnulo, and Ozgu Alay. Measuring ECN++: Good News for ++, Bad News for ECN over Mobile. *IEEE Communications Magazine*, 56(3):180–186, 2018.

[89] Aitor Martin and Naeem Khademi. On the Suitability of BBR Congestion Control for QUIC Over GEO SATCOM Networks. In *Proceedings of the Workshop on Applied Networking Research*, ANRW '22, pages 1–8, New York, NY, USA, 2022. Association for Computing Machinery.

[90] Robin Marx, Joris Herbots, Wim Lamotte, and Peter Quax. Same Standards, Different Decisions: A Study of QUIC and HTTP/3 Implementation Diversity. In *Proceedings of the Workshop on the Evolution, Performance, and Interoperability of QUIC*, pages 14–20, 2020.

[91] Matt Mathis and Jamshid Mahdavi. Deprecating the TCP Macroscopic Model. *ACM SIGCOMM Computer Communication Review*, 49(5):63–68, 2019.

[92] Matthew Mathis. Reflections on the TCP Macroscopic Model. *ACM SIGCOMM Computer Communication Review*, 39(1):47–49, 2008.

[93] Matthew Mathis, Jeffrey Semke, Jamshid Mahdavi, and Teunis Ott. The Macroscopic Behavior of the TCP Congestion Avoidance Algorithm. *ACM SIGCOMM Computer Communication Review*, 27(3):67–82, 1997.

[94] Steven McCanne and Van Jacobson. The BSD packet filter: a new architecture for user-level packet capture. In *Proceedings of the USENIX Winter 1993 Technical Conference*, USENIX '93, page 2, USA, 1993. USENIX Association.

[95] Carolyn McMillan. Lo and behold: The internet. UC Newsroom. `https://www.universityofcalifornia.edu/news/lo-and-behold-internet`, October 2019.

[96] Attila Mihály, Szilveszter Nádas, Sándor Molnár, Zsolt Krämer, Robert Skog, and Marcus Ihlar. Supporting multi-domain congestion control by a lightweight PEP. In *2017 International Conference on Internet of Things, Embedded Systems and Communications*, IINTEC, pages 105–110, 2017.

[97] Yaron Minsky, Ari Trachtenberg, and Richard Zippel. Set reconciliation with nearly optimal communication complexity. *IEEE Transactions on Information Theory*, 49(9):2213–2218, 2003.

[98] Ayush Mishra, Xiangpeng Sun, Atishya Jain, Sameer Pande, Raj Joshi, and Ben Leong. The Great Internet TCP Congestion Control Census. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 3(3):1–24, 2019.

[99] Peter L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44(170):519–521, 1985.

[100] John Nagle. Congestion Control in IP/TCP Internetworks. RFC 896, Internet Engineering Task Force (IETF), January 1984.

[101] Yong Niu, Yong Li, Depeng Jin, Li Su, and Athanasios V. Vasilakos. A survey of millimeter wave communications (mmWave) for 5G: opportunities and challenges. *Wireless Networks*, 21:2657–2676, 2015.

[102] Surendra Pandey, Raja Moses Manoj Kumar Eda, and Debabrata Das. Efficient Reordering-Reassembly PDCP and RLC Window Management Algorithm in 5G and Beyond. In *2020 IEEE International Conference on Electronics, Computing and Communication Technologies*, CONECCT, pages 1–6, 2020.

[103] Giorgos Papastergiou, Gorry Fairhurst, David Ros, Anna Brunstrom, Karl-Johan Grinnemo, Per Hurtig, Naeem Khademi, Michael Tüxen, Michael Welzl, Dragana Damjanovic, and Simone Mangiante. De-Ossifying the Internet Transport Layer: A Survey and Future Perspectives. *IEEE Communications Surveys & Tutorials*, 19(1):619–639, 2017.

[104] The PARI Group, Institut de Mathématiques de Bordeaux, UMR 5251 du CNRS, Université de Bordeaux, 351 Cours de la Libération, F-33405 Talence Cedex, France. *User's Guide to PARI/GP (version 2.11.1)*, 2018.

[105] Abhinav Pathak, Angela Wang, Cheng Huang, Albert Greenberg, Y. Charlie Hu, Randy Kern, Jin Li, and Keith Ross. Measuring and Evaluating TCP Splitting for Cloud Services. In *Proceedings of the 11th International Conference on Passive and Active Measurement*, PAM '10, pages 41–50, 2010.

[106] S. Paul, E. Ayanoglu, T.F. La Porta, K.-W.H. Chen, K.E. Sabnani, and R.D. Gitlin. An asymmetric protocol for digital cellular communications. In *Proceedings of INFOCOM'95*, volume 3, pages 1053–1062 vol. 3, 1995.

[107] Colin Perkins, Magnus Westerlund, and Joerg Ott. Media Transport and Use of RTP in WebRTC. RFC 8834, Internet Engineering Task Force (IETF), January 2021.

[108] Adithya Abraham Philip, Rukshani Athapathu, Ranysha Ware, Fabian Francis Mkocheko, Alexis Schlomer, Mengrou Shou, Zili Meng, Srinivasan Seshan, and Justine Sherry. Prudentia: Findings of an Internet Fairness Watchdog. In *Proceedings of the ACM SIGCOMM 2024 Conference*, SIGCOMM '24, pages 506–520, 2024.

[109] Adithya Abraham Philip, Ranysha Ware, Rukshani Athapathu, Justine Sherry, and Vyas Sekar. Revisiting TCP Congestion Control Throughput Models & Fairness Properties at Scale. In *Proceedings of the 21st ACM Internet Measurement Conference*, IMC '21, pages 96–103, 2021.

[110] Alain Pirovano and Fabien Garcia. A New Survey on Improving TCP Performances Over Geostationary Satellite Link. *Network and Communication Technologies*, 2(1):pp–xxx, 2013.

[111] Michele Polese, Marco Mezzavilla, Menglei Zhang, Jing Zhu, Sundeep Rangan, Shivendra Panwar, and Michele Zorzi. milliProxy: A TCP proxy architecture for 5G mmWave cellular systems. In *2017 51st Asilomar Conference on Signals, Systems, and Computers*, pages 951–957, 2017.

[112] Costin Raiciu, Christoph Paasch, Sebastien Barre, Alan Ford, Michio Honda, Fabien Duchene, Olivier Bonaventure, and Mark Handley. How hard can it be? Designing and implementing a deployable multipath TCP. In *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation*, NSDI '12, page 29, USA, 2012. USENIX Association.

[113] Florentin Rochet, Emery Assogba, and Olivier Bonaventure. TCPLS: Closely Integrating TCP and TLS. In *Proceedings of the 19th ACM Workshop on Hot Topics in Networks*, HotNets '20, page 45–52, New York, NY, USA, 2020. Association for Computing Machinery.

[114] IEEE Standards Association (IEEE SA). The Evolution of Wi-Fi Technology and Standards. IEEE. `https://standards.ieee.org/beyond-standards/the-evolution-of-wi-fi-technology-and-standards/`, May 2023.

[115] J. H. Saltzer, D. P. Reed, and D. D. Clark. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, November 1984.

[116] Patrick Sattler, Juliane Aulbach, Johannes Zirngibl, and Georg Carle. Towards a tectonic traffic shift? Investigating Apple's new relay network. In *Proceedings of the 22nd ACM Internet Measurement Conference*, IMC '22, pages 449–457, 2022.

[117] David Schinazi. Proxying UDP in HTTP. RFC 9298, Internet Engineering Task Force (IETF), August 2022.

[118] David Schinazi. The MASQUE Proxy. Internet Draft draft-schinazi-masque-proxy-05, Internet Engineering Task Force (IETF), February 2025.

[119] David Schinazi and Lucas Pardue. HTTP Datagrams and the Capsule Protocol. RFC 9297, Internet Engineering Task Force (IETF), August 2022.

[120] Marten Seemann. QUIC Interop Runner. `https://interop.seemann.io/`, January 2025.

[121] Justine Sherry, Chang Lan, Raluca Ada Popa, and Sylvia Ratnasamy. BlindBox: Deep Packet Inspection over Encrypted Traffic. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, SIGCOMM '15, page 213–226, New York, NY, USA, 2015. Association for Computing Machinery.

[122] Yeong-Jun Song, Geon-Hwan Kim, Imtiaz Mahmud, Won-Kyeong Seo, and You-Ze Cho. Understanding of BBRv2: Evaluation and Comparison with BBRv1 Congestion Control Algorithm. *IEEE Access*, 9:37131–37145, 2021.

[123] Starlink. Connecting the Unconnected. `https://www.starlink.com/connecting-the-unconnected`. Accessed: June 2, 2025.

[124] W. Richard Stevens. TCP Slow Start, Congestion Avoidance, Fast Retransmit, and Fast Recovery Algorithms. RFC 2001, Internet Engineering Task Force (IETF), January 1997.

[125] Elias Summermatter and Christian Grothoff. Byzantine fault tolerant set reconciliation. GNUnet Living Standards Document. `https://lsd.gnunet.org/lsd0003/`, August 2022.

[126] Ludovic Thomas, Emmanuel Dubois, Nicolas Kuhn, and Emmanuel Lochin. Google QUIC Performance Over a Public SATCOM Access. *International Journal of Satellite Communications and Networking*, 37(6):601–611, 2019.

[127] Dr. Joseph D. Touch. Transport Options for UDP. Internet Draft draft-ietf-tsvwg-udp-options-28, Internet Engineering Task Force (IETF), November 2023.

[128] W3Techs. Usage statistics of QUIC for websites. Web Technology Surveys. `https://w3techs.com/technologies/details/ce-quic`. Accessed: April 2025.

[129] Gerry Wan, Fengchen Gong, Tom Barbette, and Zakir Durumeric. Retina: analyzing 100GbE traffic on commodity hardware. In *Proceedings of the ACM SIGCOMM 2022 Conference*, SIGCOMM '22, page 530–544, New York, NY, USA, 2022. Association for Computing Machinery.

[130] Ranysha Ware, Matthew K. Mukerjee, Srinivasan Seshan, and Justine Sherry. Modeling BBR's Interactions with Loss-Based Congestion Control. In *Proceedings of the Internet Measurement Conference*, IMC '19, pages 137–143, 2019.

[131] Ranysha Ware, Adithya Abraham Philip, Nicholas Hungria, Yash Kothari, Justine Sherry, and Srinivasan Seshan. CCAnalyzer: An Efficient and Nearly-Passive Congestion Control Classifier. In *Proceedings of the ACM SIGCOMM 2024 Conference*, SIGCOMM '24, pages 181–196, 2024.

[132] Magnus Westerlund and Stephan Wenger. RTP Topologies. RFC 7667, Internet Engineering Task Force (IETF), November 2015.

[133] Keith Winstein and Hari Balakrishnan. Mosh: An Interactive Remote Shell for Mobile Clients. In *2012 USENIX Annual Technical Conference*, USENIX ATC '12, pages 177–182. USENIX Association, June 2012.

[134] Shinae Woo and KyoungSoo Park. Scalable TCP session monitoring with symmetric receive-side scaling. *KAIST, Daejeon, Korea, Tech. Rep*, 144, 2012.

[135] Lei Yang. riblt. GitHub, `https://github.com/yangl1996/riblt/`.

[136] Lei Yang, Yossi Gilad, and Mohammad Alizadeh. Practical rateless set reconciliation. In *Proceedings of the ACM SIGCOMM 2024 Conference*, SIGCOMM '24, pages 595–612, 2024.

[137] Gina Yuan. QuACK. GitHub, `https://github.com/ygina/quack`.

[138] Gina Yuan. Sidekick. GitHub, `https://github.com/ygina/sidekick`.

[139] Gina Yuan, Thea Rossman, and Keith Winstein. Internet Connection Splitting: What's Old is New Again. In *2025 USENIX Annual Technical Conference*, USENIX ATC '25, 2025.

[140] Gina Yuan, Matthew Sotoudeh, David K. Zhang, Michael Welzl, David Mazières, and Keith Winstein. Sidekick: In-Network Assistance for Secure End-to-End Transport Protocols. In *21st USENIX Symposium on Networked Systems Design and Implementation*, NSDI '24, pages 1813–1830, 2024.

[141] Gina Yuan, David K. Zhang, Matthew Sotoudeh, Michael Welzl, and Keith Winstein. Sidecar: In-Network Performance Enhancements in the Age of Paranoid Transport Protocols. In *Proceedings of the 21st ACM Workshop on Hot Topics in Networks*, HotNets '22, pages 221–227, 2022.

[142] Danesh Zeynali, Emilia N. Weyulu, Seifeddine Fathalli, Balakrishnan Chandrasekaran, and Anja Feldmann. Promises and Potential of BBRv3. In *International Conference on Passive and Active Network Measurement*, PAM '24, pages 249–272. Springer, 2024.

[143] Johannes Zirngibl, Philippe Buschmann, Patrick Sattler, Benedikt Jaeger, Juliane Aulbach, and Georg Carle. It's Over 9000: Analyzing Early QUIC Deployments with the Standardization on the Horizon. In *Proceedings of the 21st ACM Internet Measurement Conference*, IMC '21, page 261–275, New York, NY, USA, 2021. Association for Computing Machinery.

[144] Zoom Video Communications, Inc. Zoom Encryption White Paper. `https://explore.zoom.us/docs/doc/Zoom%20Encryption%20Whitepaper.pdf`, August 2021.